CSCI 566 Midterm Review

Yue Zhao, Fall 2026. Core material from Blocks 1-7, traced through small worked examples.

USC Viterbi School of Engineering

Route Through the Review

Seven sections move from evaluation and classical methods to graphs and shared computation.

1	Evaluation and Classical ML	4-36
2	Forward Computation and CNNs	37-77
3	Losses, Gradients and Updates	78-111
4	Reliable Training and Transfer	112-145
5	Recurrent Memory	146-173
6	Attention and Transformers	174-196
7	Graphs and Shared Computation	197-233

Two Study Paths for the Midterm Review

Class follows a core route through selected and new slides. The full deck keeps every step for self-study.

PowerPoint: Midterm Core Review (112 pages)

Evaluation	4-9, 14, 16-20, 22-26, 35-36
Forward + CNNs	37, 41-42, 46-47, 67-75, 77
Gradients	78-81, 87-97, 99-101, 104-107, 111
Training	112-113, 115, 122-124, 127, 130-133, 135-136, 139-140, 142, 144
RNN / LSTM	146, 148, 151, 155-156, 168, 170, 172-173
Attention	174, 178, 182, 184-186, 193-196
Graphs	197-198, 202-203, 209-210, 214-215, 219-222, 226, 229, 232-233

Opening pages 1-3 are included in the 112-page route.

Evaluation and Classical ML

Data splits, generalization, and simple predictors set the standard for judging any model.

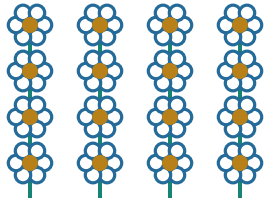
1. Training data fits parameters, validation data selects settings, and test data gives a final check.
2. Fit preprocessing statistics on training data only, then apply them unchanged to validation and test.
3. Generalization means low error on new data from the same source as the training data.
4. kNN predicts by a vote among nearby labels, and k-means clusters by minimizing a squared distance objective.

Splitting Data by Plant: Training, Validation, Test

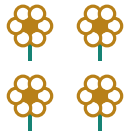
Illustrative split of 120 daisy and tulip photos, grouped by plant

Daisy or tulip?

120 photos: 24 plants, 5 photos each







Training: 16 plants / 80 photos

Fit weights

Validation: 4 plants / 20 photos

Select depth or checkpoint

Test: 4 plants / 20 photos

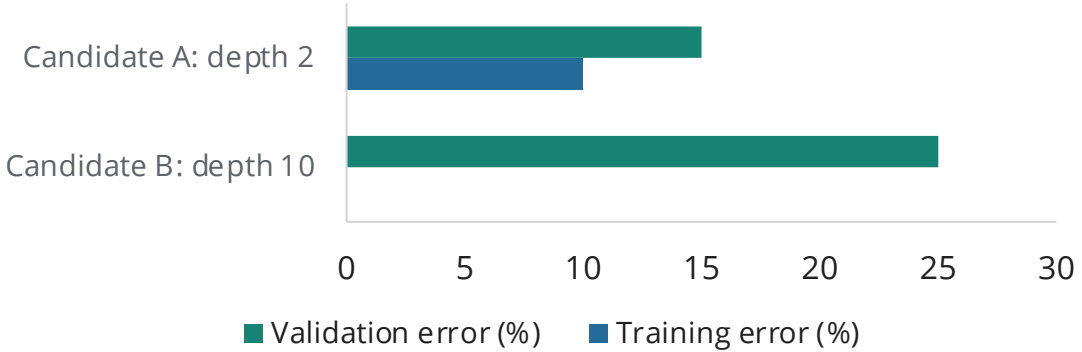
Evaluate once, after selection

All 5 photos of a plant stay together

Generalize to unseen plants, same source

Choosing a Model with Validation, Then Testing Once

Two decision trees fit on the same 80 training photos (illustrative results)



Training: 8/80 errors (10%)

Validation: 3/20 errors (15%)

Training: 0/80 errors (0%)

Validation: 5/20 errors (25%)

Select A: lower validation error → **Choice frozen**

A on test: 4/20 errors (20%)

Test is not used to switch candidate

USC Viterbi School of Engineering

Fit Once, Apply Everywhere

Centering a flower-photo feature using only training data (illustrative example)

Average pixel intensity per photo (arbitrary units)



All 80 training photos (daisy and tulip) → **Fit: mean of feature over all 80 training photos = 100 (illustrative)**

Stored rule: x → x - 100

Training photo: 120 → **−100** → **20**

Validation photo: 130 → **−100** → **30**

Test photo: 90 → **−100** → **-10**

Outputs are centered feature values, not prediction errors

Check: Training, Validation, and Test

Train fits weights and preprocessing, validation chooses settings and checkpoint, and test evaluates the final selected model.

A Try several learning rates, keep the one with the best test accuracy, and report that accuracy.

B Fit on train, choose learning rate and checkpoint on validation, then evaluate that choice on test.

C Compute normalization statistics from all data including test, then train and tune on validation.

D Add the test set to training for more data, then choose settings on validation and report validation.

Answer B: Fit, Choose, Then Evaluate

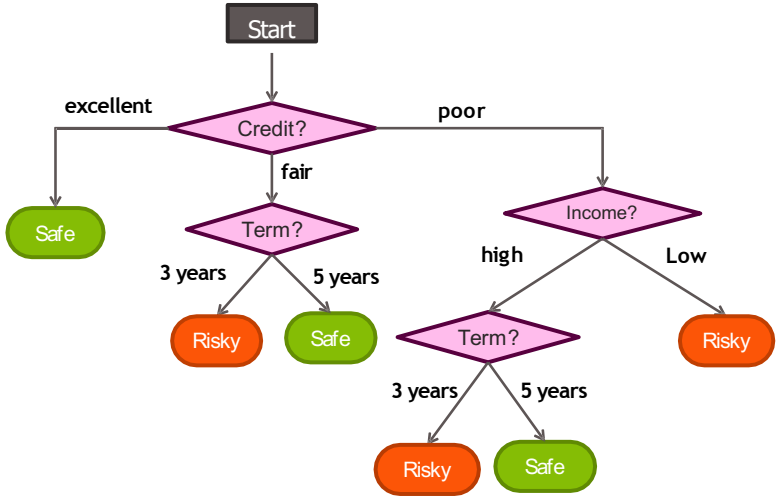
Only plan B fits on train, chooses on validation, and keeps the test set unseen until the final selected model is evaluated.

Answer: B

A	Incorrect. Choosing settings with test accuracy makes the test set a selection tool, so its score no longer checks unseen data.
B	**Correct. Each split keeps its taught role, and the test set stays unseen until it evaluates the final selected model.**
C	Incorrect. Preprocessing is fitted on the training set only. Using test statistics leaks test information into the model.
D	Incorrect. Fitting weights on test data removes the held-out check, and a validation score used for choosing is not a final result.

Decision Tree

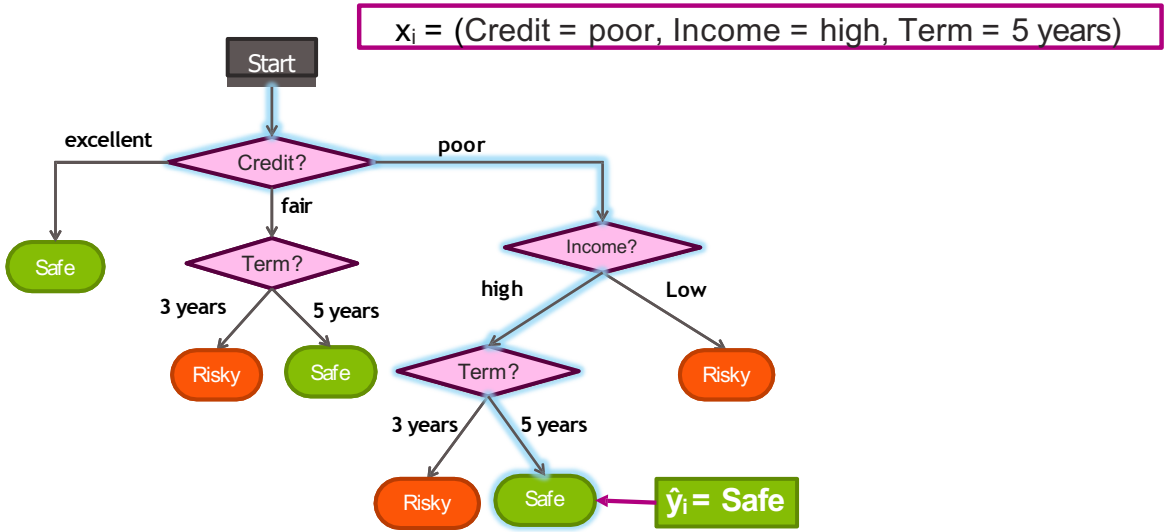
This module ... decision trees



Credit to Stanford CS 229

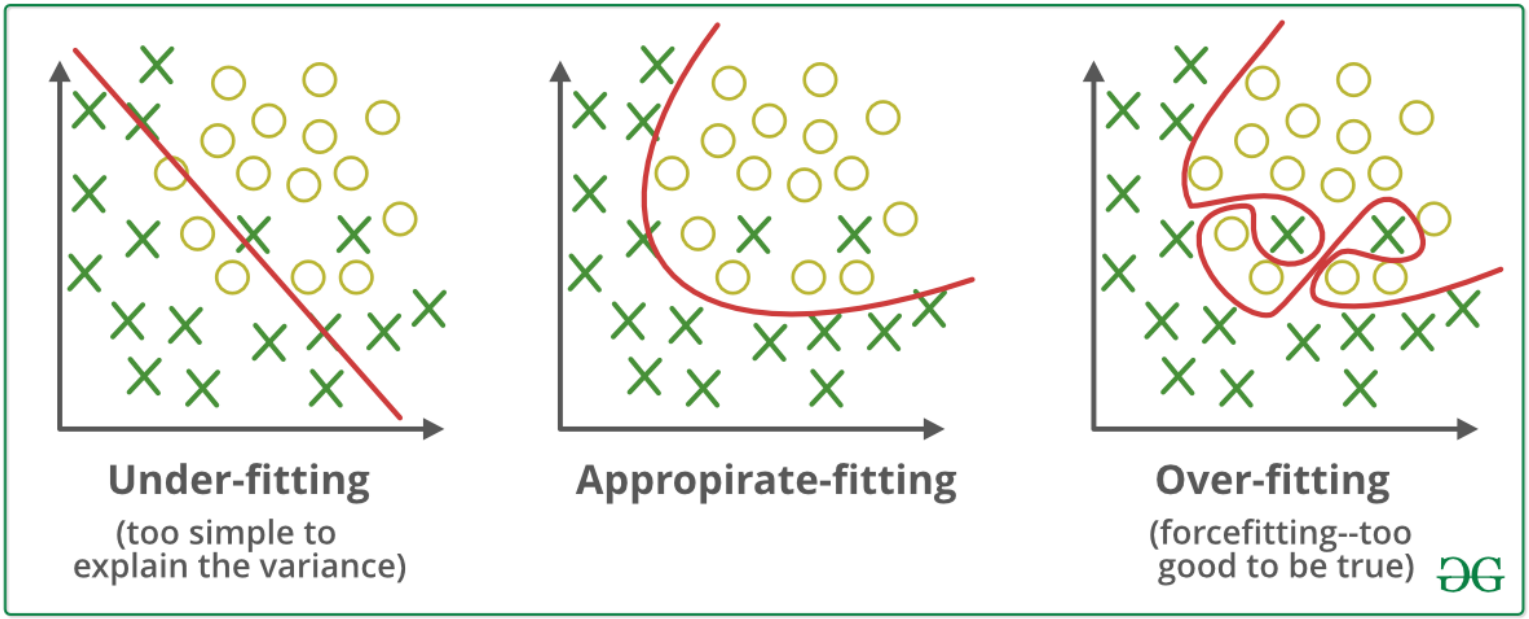
Decision Tree

Scoring a loan application



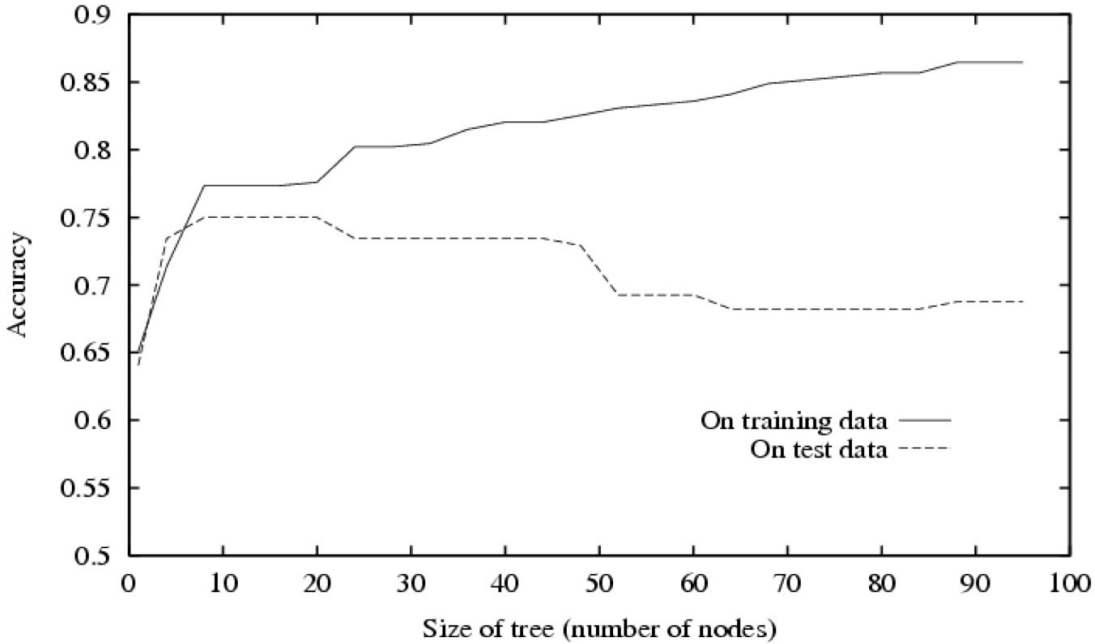
Credit to Stanford CS 229

Overfitting and Underfitting Revisited



We favor the case where the ML model appropriately depict the underlying data

DT Generalization – Overfitting Revisited

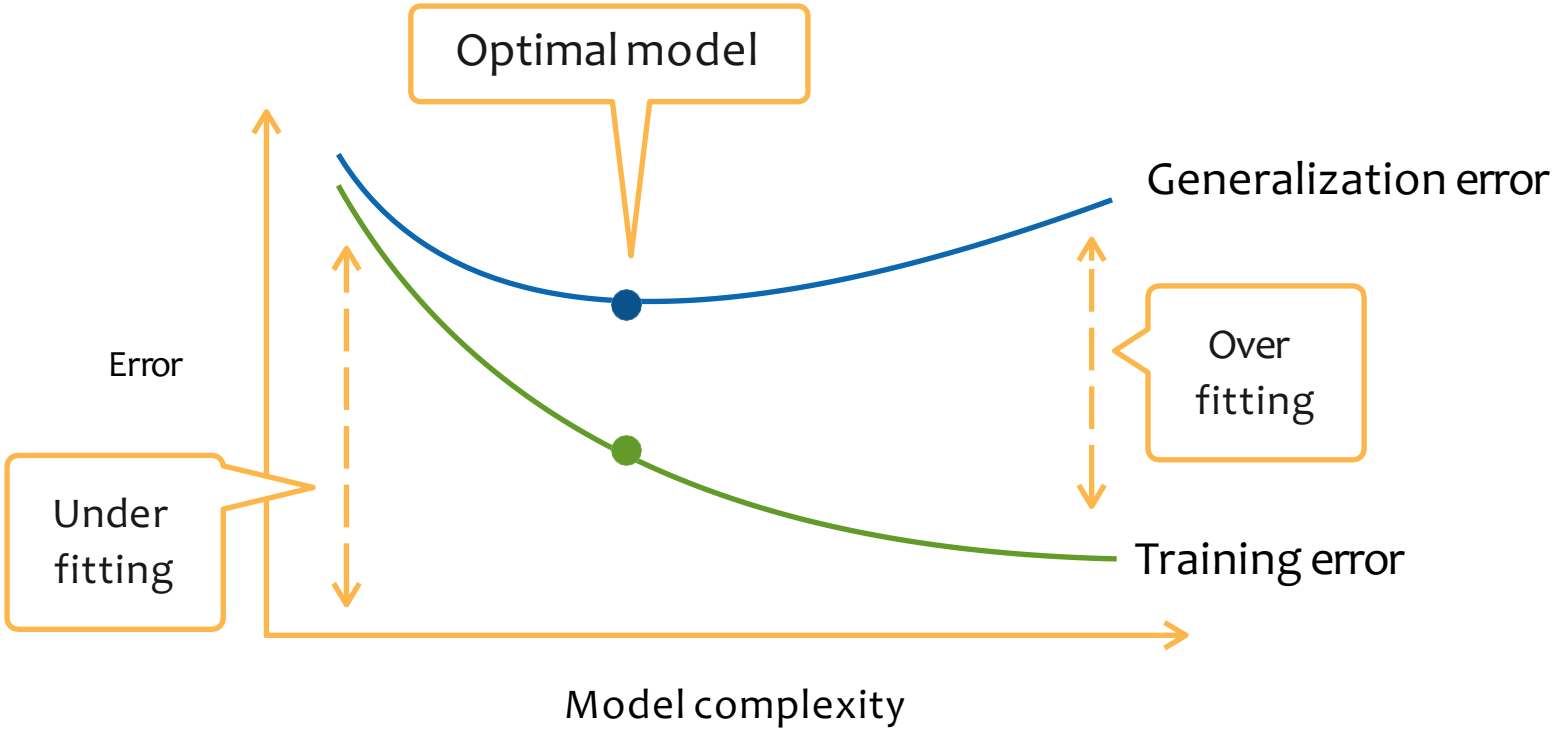


Credit to CMU 10601

Figure from Tom Mitchell

USC Viterbi School of Engineering

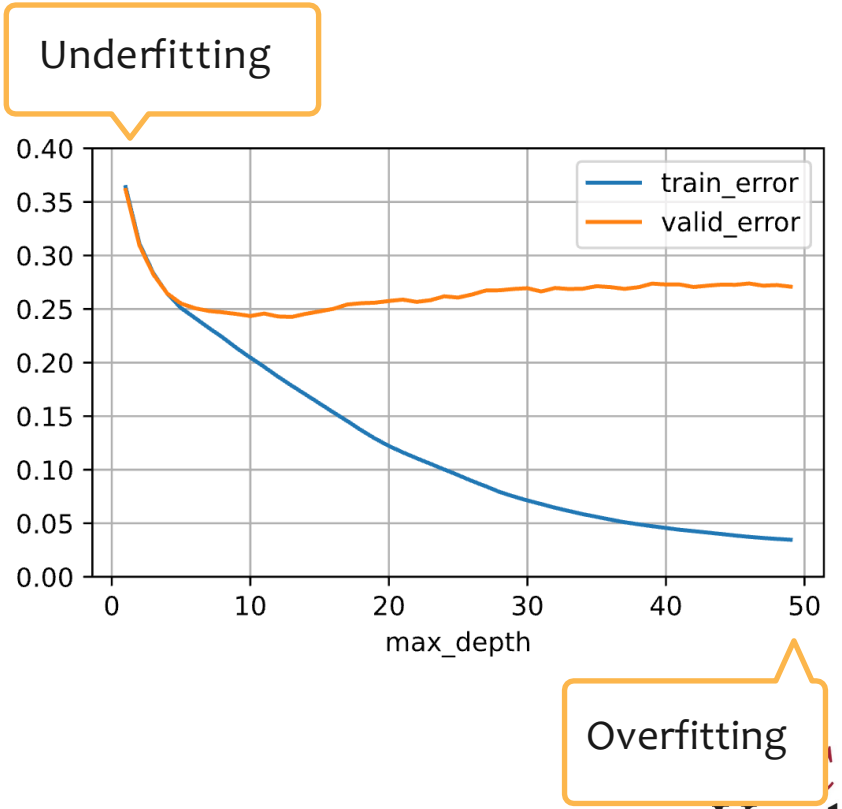
Model Complexity



USC Viterbi School of Engineering

Model Complexity Example: Decision Tree

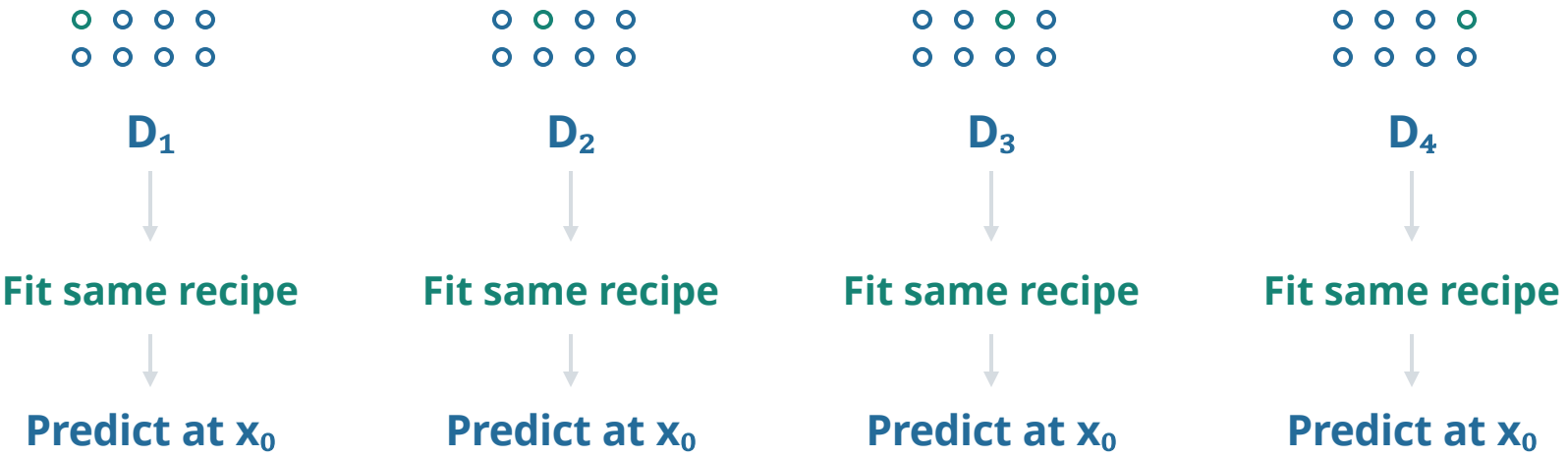
- The tree size can be controlled by the number of levels
- Use scikit-learn **DecisionTreeRegressor(max_depth =n)** on house sales data



Viterbi School of Engineering

Repeating Training on Fresh Data Samples

To study a training recipe, imagine drawing four new training sets from one source and asking each fitted model to predict at the same input x0.



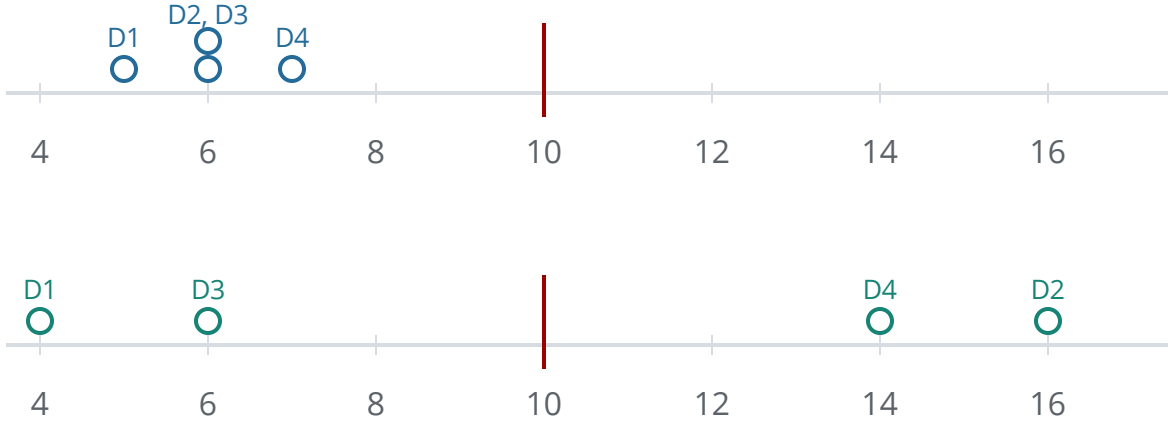
D1 to D4 are four training sets drawn independently from the same source.

Bias and Variance on Two Number Lines

At one fixed input x0, each recipe is fit on four training sets D1–D4 from one source. Dots are unitless toy predictions, and the noise-free target is 10.

Bias: mean − target

Variance: spread across refits



Mean 6; target 10

Mean 10; target 10

A: mean misses the target. B: mean on target, larger spread.

Reading Train and Validation Errors as Clues

When train and validation data come from the same source, the pattern of their errors suggests a first thing to try, not a proven cause.

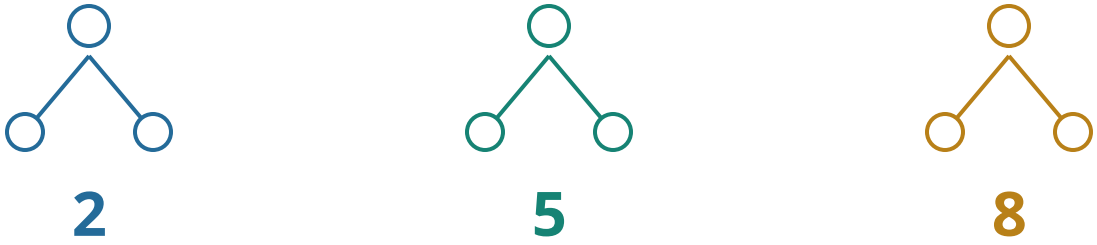
Observed Pattern	Possible Cause	First Things to Try
Training + validation errors both high	Too rigid, weak features, or incomplete training	Inspect fit and features; consider more capacity
Training low; validation much higher	Model fits training-specific details	More data, regularization, or a simpler model

Averaging an ensemble may reduce variability, but correlated member errors limit the benefit.

Averaging an Ensemble of Trees

Three trees fitted on resampled data give different predictions, and the ensemble reports their mean.

Same input sent to all three trees



(2 + 5 + 8) / 3 = 5

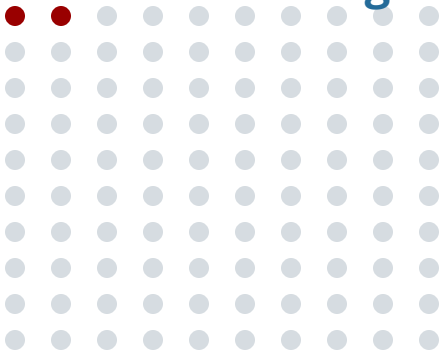
Resampling the training data changes the trees that are learned.

Averaging aims to reduce sensitivity, but correlated errors limit it.

When 98% Accuracy Catches No Defects

Fabricated inspection data: 1,000 parts, of which 20 are defective (positive) and 980 are good (negative). A rule that always predicts good never flags a part.

20 defective / 980 good



Each dot = 10 parts

Predict every part as good

980 / 1,000 = 98% accuracy

0 / 20 defects caught

Recall = 0%

Precision is undefined because no part is flagged, and 0/0 has no value.

Metrics for Binary Classification

- Accuracy: # correct predictions / # examples

```
sum(y == y_hat) / y.size
```

- Precision: # True positive / # (True positive + False positive)

```
sum((y_hat == 1) & (y == 1)) / sum(y_hat == 1)
```

- Recall: # True positive / # Positive examples

```
sum((y_hat == 1) & (y == 1)) / sum(y == 1)
```

- Be careful of division by 0
- One metric that balances precision and recall
  - F1: the harmonic mean of precision and recall: $2pr/(p + r)$

USC Viterbi School of Engineering

Precision and Recall from the Same Counts

Same 1,000 fabricated parts. Positive means defective and predicted positive means flagged for inspection. The counts are TP 16, FP 24, FN 4 and TN 956.

Actual status ↓ / Prediction →

Parts	Flagged +	Passed −
Defective +	TP = 16 Caught	FN = 4 Missed
Good −	FP = 24 False alarms	TN = 956 Correctly passed

Precision

16 / (16 + 24) = 40%

Of 40 flagged, 16 defective

Recall

16 / (16 + 4) = 80%

Of 20 defective, 16 caught

Both fractions share the numerator 16 but divide by different groups: the 40 flagged parts versus the 20 defective parts.

Moving the Flagging Threshold

Same 1,000 fabricated parts with 20 defects. A part is flagged when its defect score is at least the threshold. The counts are constructed for illustration.

Same 1,000 parts, including 20 defects

Threshold	Flagged	True / False Alarms	Precision	Recall
0.2 (lower)	80	19 / 61	23.75%	95%
0.8 (higher)	12	9 / 3	75%	45%

Missed defects: 1 at threshold 0.2; 11 at threshold 0.8

Lowering the threshold can add both true and false alarms; recall cannot fall, but precision can move either way.

Check: Which Metric Leads for a Defect Flagger

A factory flags rare defects. Missing a defect costs much more than a false alarm, which triggers one manual review. Which metric should lead threshold selection?

A Accuracy, since most parts are good and it combines all four counts.

B Precision, since every flagged part should turn out to be defective.

C Specificity, since correctly passing good parts is the most common case.

D Recall, while tracking precision so the manual review load stays manageable.

Answer D: Recall Leads, Precision Is Tracked

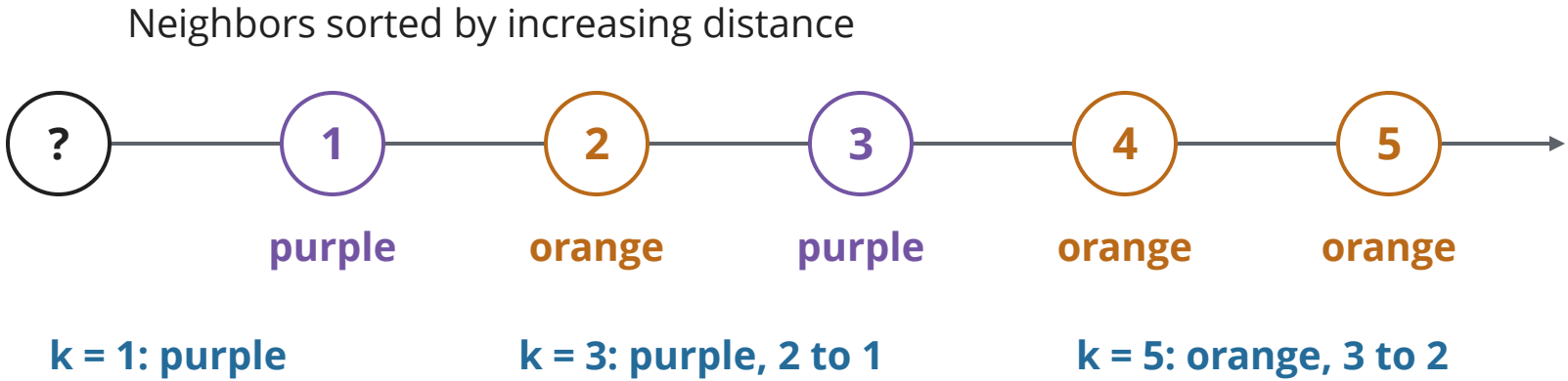
Defect flagger: a missed defect ships to a customer, while a false alarm triggers one manual review.

Answer: D

A	Wrong: with rare defects, accuracy can stay high even when many defects are missed.
B	Wrong: precision limits false alarms, the cheaper error here, and can tolerate misses.
C	Wrong: specificity counts good parts and can be high while defects slip through.
D	**Correct: recall tracks the costly misses, and precision shows the review workload.**

k-Nearest Neighbor Voting

The prediction depends on k, because a larger k lets more neighbors vote.



Sorted by increasing distance, the nearest labels are purple, orange, purple, orange, orange.

Votes here are unweighted and distances are distinct; choose k by validation results.

k Nearest Neighbor Algorithm

def set_hyperparameters(k, d)**:**
    Store k
    Store d(·, ·)

def train($\mathcal{D}$)**:**
    Store $\mathcal{D}$

def h(x')**:**
    Let S = the set of k points in $\mathcal{D}$ nearest to x'
        according to distance function
        d(**u**, **v**)
    Let v = majority_vote(S)
    return v

Credit to CMU 10601

k Nearest Neighbor Algorithm

Distance Functions:

- KNN requires a **distance function**

$$d : \mathbb{R}^M \times \mathbb{R}^M \rightarrow \mathbb{R}$$

- The most common choice is **Euclidean distance**

$$d(\boldsymbol{u}, \boldsymbol{v}) = \sqrt{\sum_{m=1}^{M} (u_m - v_m)^2}$$

- But there are other choices (e.g. **Manhattan distance**)

$$d(\boldsymbol{u}, \boldsymbol{v}) = \sum_{m=1}^{M} |u_m - v_m|$$

Credit to CMU 10601

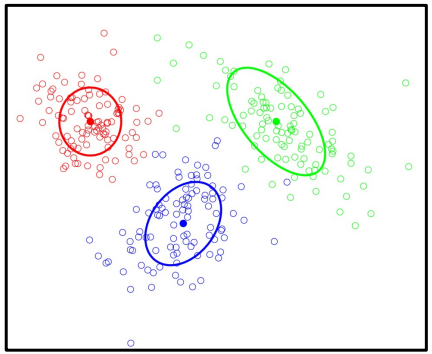
Clustering

Clustering

No labels provided
…uncover cluster structure from input alone

Input: docs as vectors x_i
Output: cluster labels z_i

An unsupervised learning task



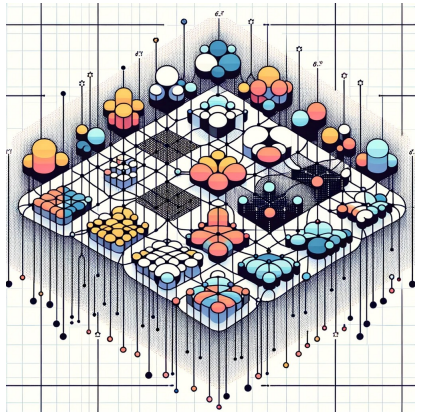
Credit to Stanford CS 229

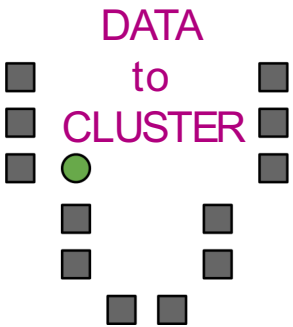
Clustering: k-means

k-means

Assume

- Score= distance to cluster center (smaller better)





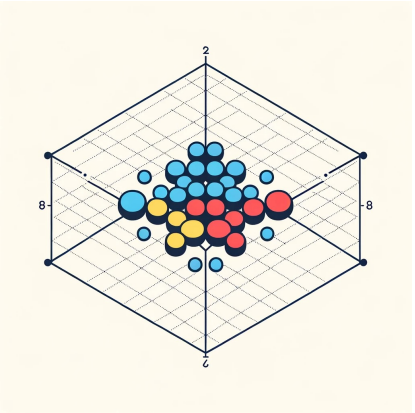
Credit to Stanford CS 229

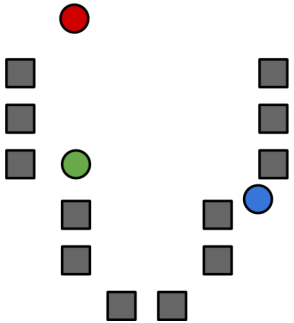
Clustering: k-means

k-means algorithm

1. Initialize cluster centers

   $\mu_1, \mu_2, \ldots, \mu_k$





Credit to Stanford CS 229

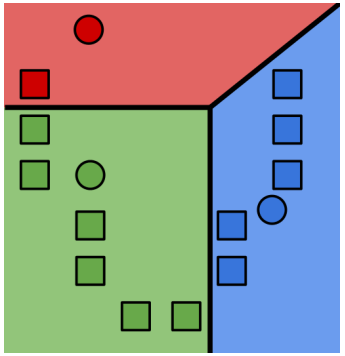
Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center

$$z_i \leftarrow \arg\min_j ||\mu_j - \mathbf{x}_i||_2^2$$

Inferred label for obs i, whereas supervised learning has given label y_i



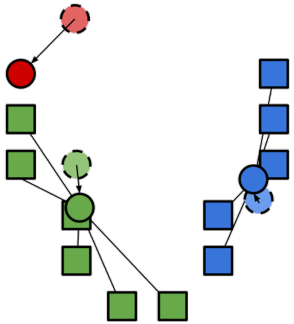
Credit to Stanford CS 229

Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations

$$\mu_j = \frac{1}{n_j} \sum_{i:z_i=j} \mathbf{x}_i$$

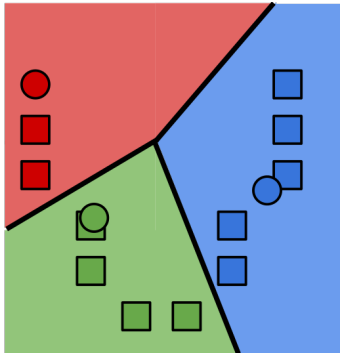


Credit to Stanford CS 229

Clustering: k-means

k-means algorithm

1. Initialize cluster centers
2. Assign observations to closest cluster center
3. Revise cluster centers as mean of assigned observations
4. Repeat 2.+3. until convergence



Credit to Stanford CS 229

The k-Means Objective

k-means scores a clustering by the total squared distance from points to their assigned centers.

Assigned points: 1, 5, 6. Center: 4.



$$(1 + 5 + 6) / 3 = 4$$

$$J = (1 - 4)^2 + (5 - 4)^2 + (6 - 4)^2 = 14$$

With assignments fixed, the mean minimizes the sum of squared distances.

The next page compares two complete runs on the same data and k.

Two Finished k-Means Runs, Two Different Answers

Same data {1, 3, 5, 9}, same k = 2. Repeating the assignment and mean steps changes neither run.

Run A



$J = 1 + 1 + 4 + 4 = 10$

Run B



$J = 4 + 0 + 4 + 0 = 8$

Run B has lower squared-distance total, 8 versus 10, so it wins this comparison.

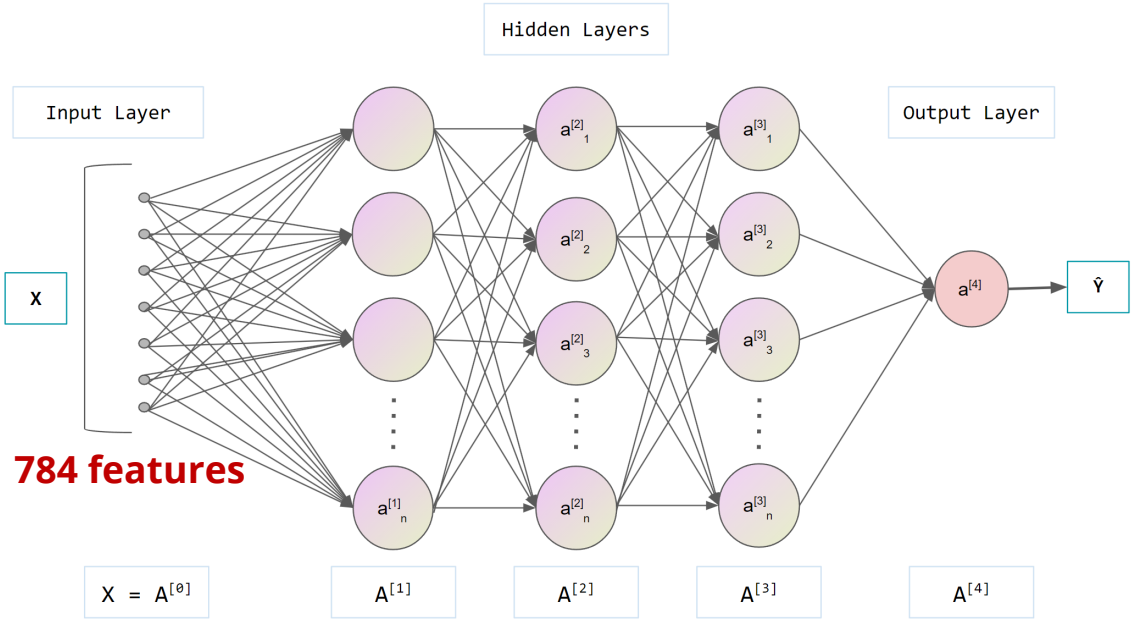
Comparing two converged runs does not prove global optimality.

Forward Computation and CNNs

Tracking shapes and counting parameters makes each forward layer concrete.

1. Track tensor shapes through each layer, and separate trainable parameters from computed activations.
2. Stacking affine layers without a nonlinearity still gives one affine map.
3. In CNNs, channels and spatial positions play different roles in counting and in computation.
4. Some source slides use row vectors xW; new formulas here use column vectors Wx, and parameter counts are the same.

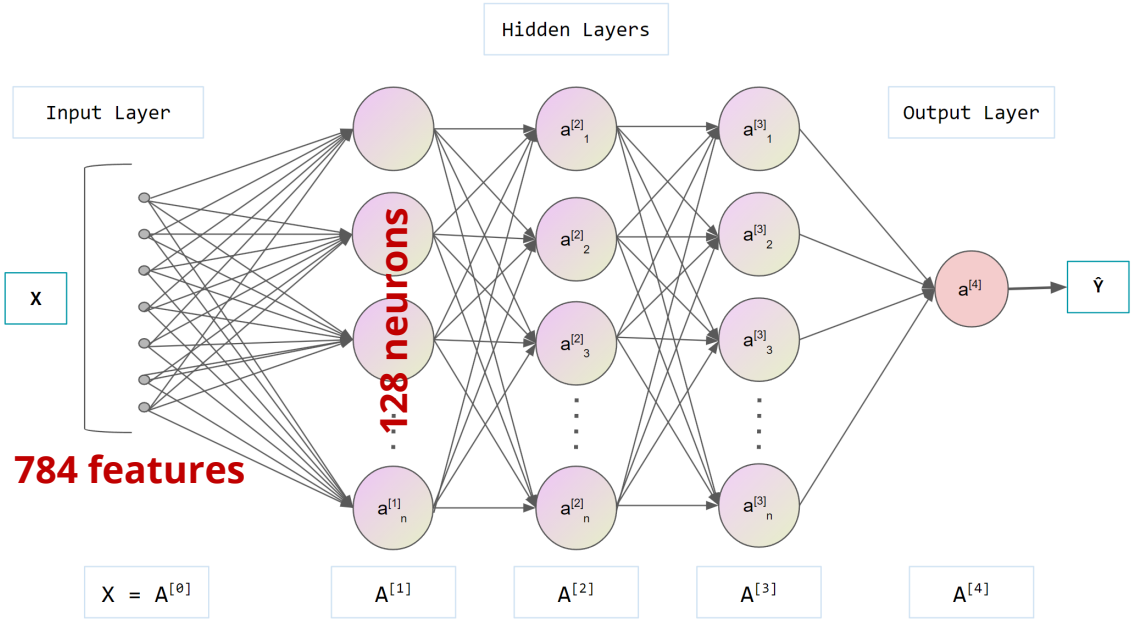
Neural Network Data Shape Explained



Assume each sample is a 28x28 pixels image = 784 features

USC Viterbi School of Engineering

Neural Network Data Shape Explained



The first hidden layer

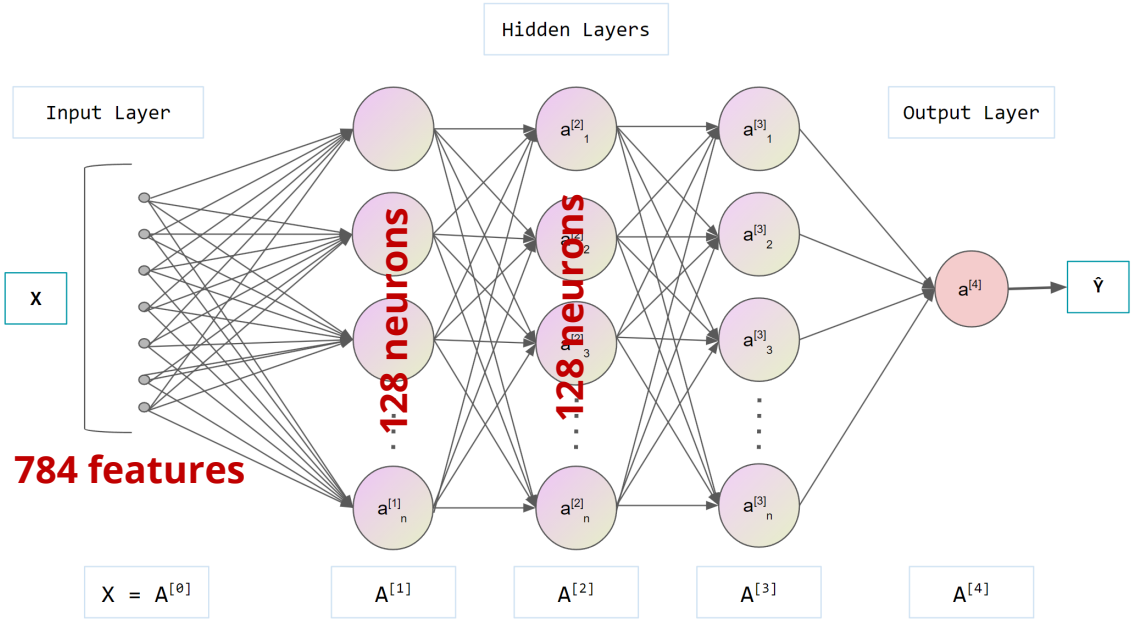
A[1] has 128 neurons

Thus, the weight matrix **W1** for the first layer will be 784x128

After the first layer, it will have

[1x784] · [784x128] = **1x128** outputs

Neural Network Data Shape Explained



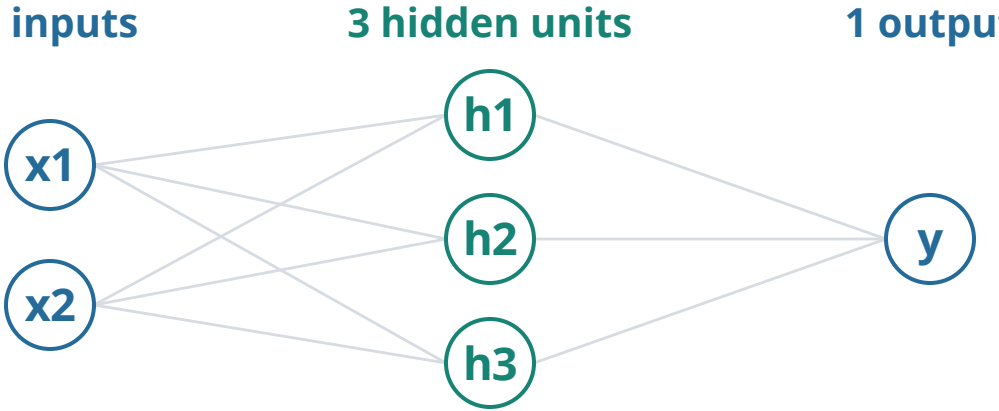
After the first layer, it will have

[1x784] · [784x128] = **1x128** outputs

The second layer also gets 128 neurons, and the weight matrix **W2** for the second layer will be 128x128

Counting Parameters in a Dense Network

A network with widths 2, 3, and 1 has 13 trainable parameters in its weights and biases.



Column vectors: Wx + b

W_1: 3 × 2 = 6

b_1: 3

W_2: 1 × 3 = 3

b_2: 1

6 + 3 + 3 + 1 = 13 parameters

Inputs and activations are recomputed per example, and ReLU has no trainable parameters.

Stacked Affine Layers Collapse

Without a nonlinearity between them, two affine layers are equivalent to a single affine layer.



$$y = (W_2W_1)x + (W_2b_1 + b_2)$$

$$h = 3x + 2, \quad y = 2h - 1 \quad \Rightarrow \quad y = 6x + 3$$

From h = W1x + b1 and y = W2h + b2, we get y = W2W1x + W2b1 + b2.

The combined weight is W2W1 and the combined bias is W2b1 + b2.

Neural networks: why is max operator important?

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the **activation function.**

Q: What if we try to build a neural network without it?

$$f = W_2 W_1 x$$

Credit to Stanford CS 231n

Neural networks: why is max operator important?

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the **activation function.**

Q: What if we try to build a neural network without one?

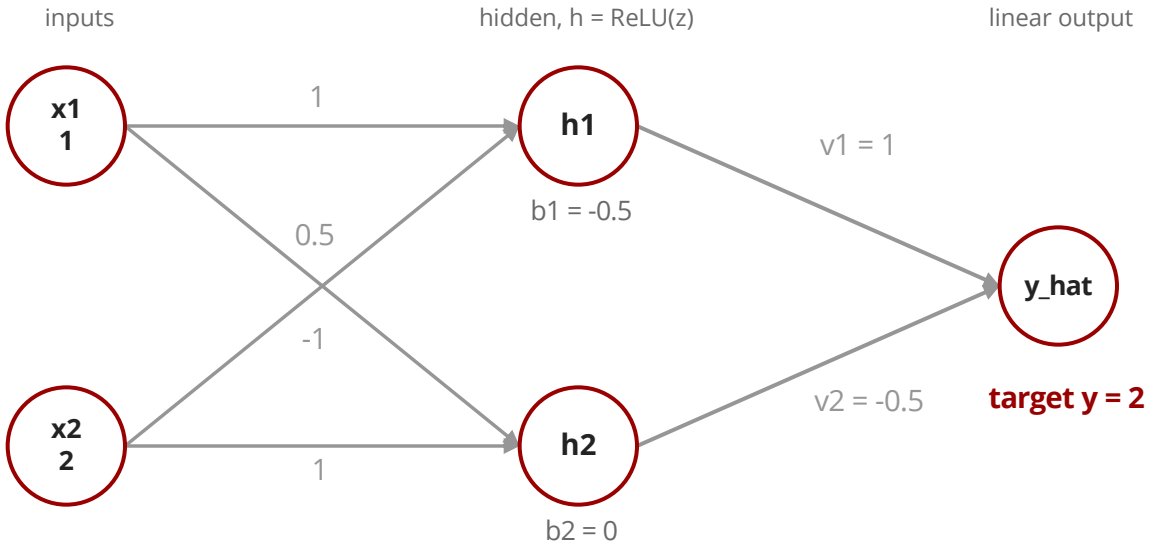
$$f = W_2 W_1 x \qquad W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, f = W_3 x$$

A: We end up with a linear classifier again!

Neural networks need this non linearity!!

Credit to Stanford CS 231n

Worked Example: Read the Numbers on the Network



All values live on the object that uses them:

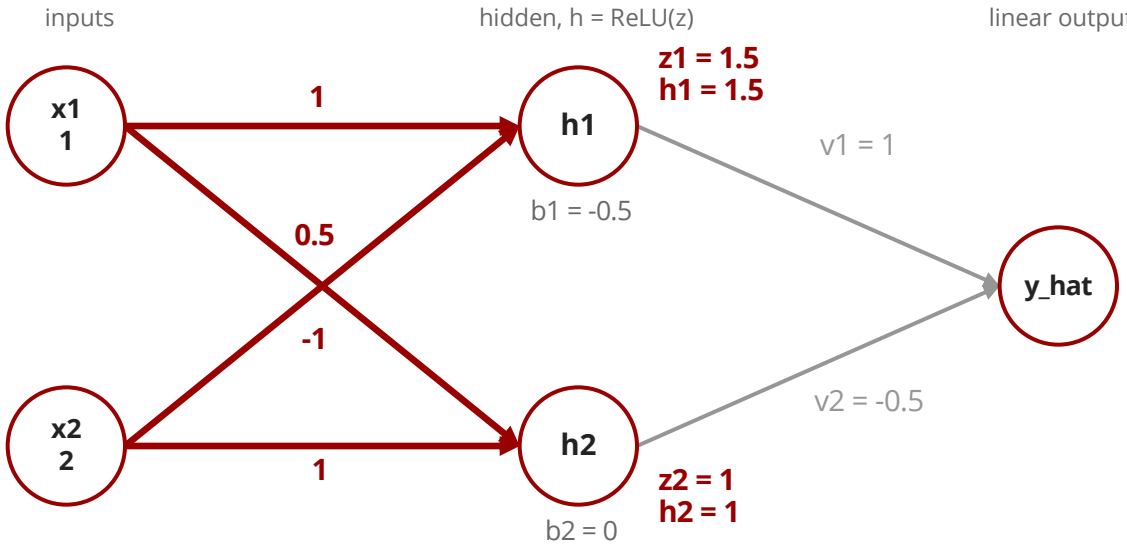
inputs in the input nodes
weights on their edges
biases beside hidden nodes
target beside the output

Read each multiplication from an edge and each sum at the receiving node.

One fixed diagram will carry the full forward, loss, gradient, and update sequence.

USC Viterbi School of Engineering

Forward Pass: Hidden Units



At h1

z1 = 1(1) + 0.5(2) - 0.5
= 1.5
h1 = ReLU(1.5) = 1.5

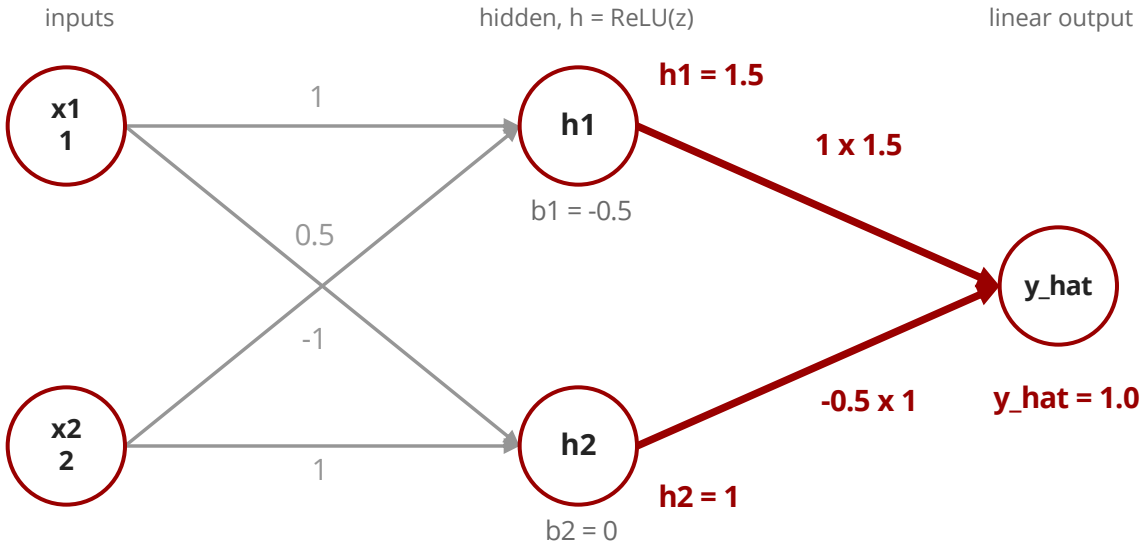
At h2

z2 = -1(1) + 1(2) + 0
= 1
h2 = ReLU(1) = 1

Each highlighted edge contributes one product to the receiving hidden unit.

USC Viterbi School of Engineering

Forward Pass: Linear Output



Both hidden activations now travel along their output edges.

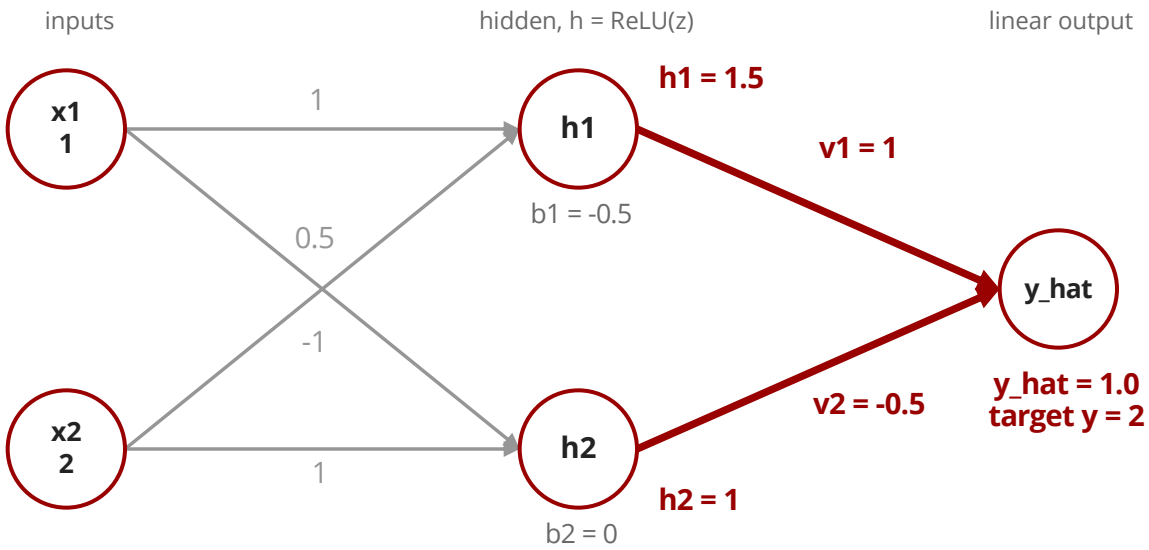
y_hat = 1(1.5) + (-0.5)(1) + 0 = 1.0

The output layer is linear in this example.

The output value is a weighted sum of the two hidden activations.

USC Viterbi School of Engineering

Loss at the Output



Prediction and target meet at the output.

$$L = (\hat{y} - y)^2 = (1 - 2)^2 = 1$$

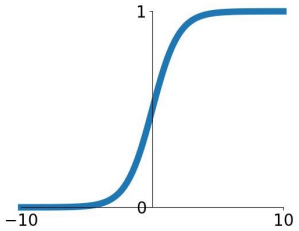
The loss measures the output error before any parameter is updated.

USC Viterbi School of Engineering

Activation functions

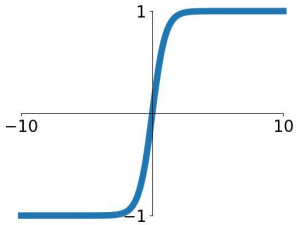
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

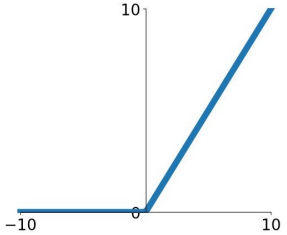


tanh

$$\tanh(x)$$



$$\max(0, x)$$



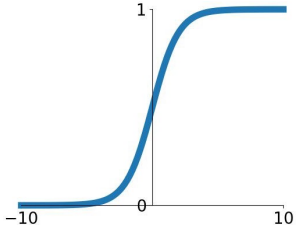
Sigmoid Function

- **Characteristics**: Sigmoid squashes the input values into a range between 0 and 1. It's historically been used for binary classification.
- **When to use**: In the output layer for binary classification tasks. When you need probabilities as outputs, since its output can be interpreted as a probability.
- **Limitations**: Prone to **vanishing gradient problem (discussed later)**. Outputs are not zero-centered.

Activation functions

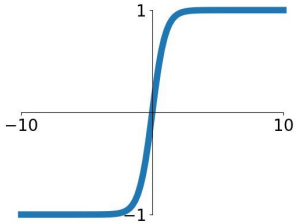
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



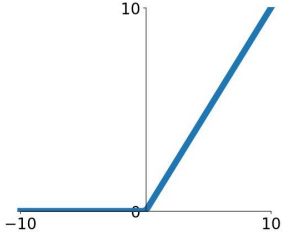
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



ReLU:

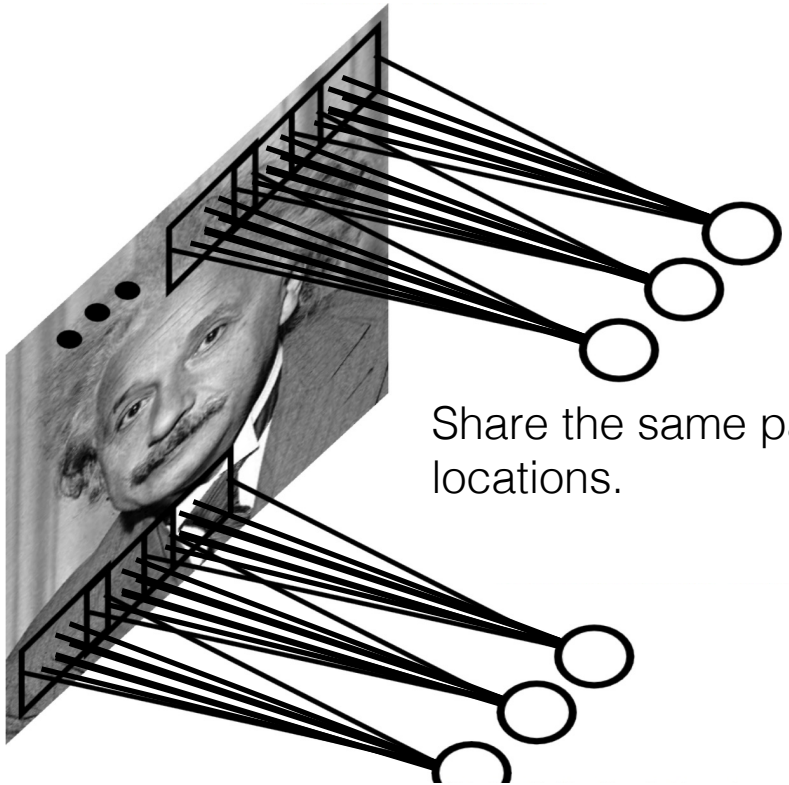
Characteristics: ReLU is a piece-wise linear function that outputs the input directly if positive, otherwise, it will output zero.

When to use: In most hidden layers, as it helps with the vanishing gradient problem and speeds up training. Good default choice for deep neural networks.

Limitations:

Can suffer from "dying ReLU" problem, where neurons can become inactive on all current training examples.

Convolutional layer

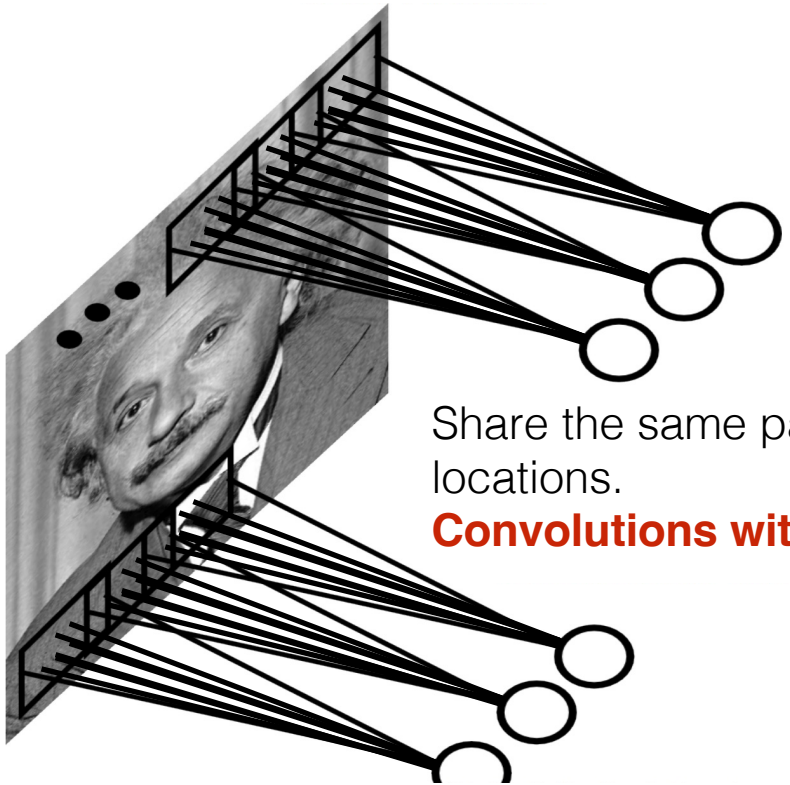


Share the same parameters across different locations.

Slide credit: Marc'Aurelio Ranzato

USC Viterbi School of Engineering

Convolutional layer

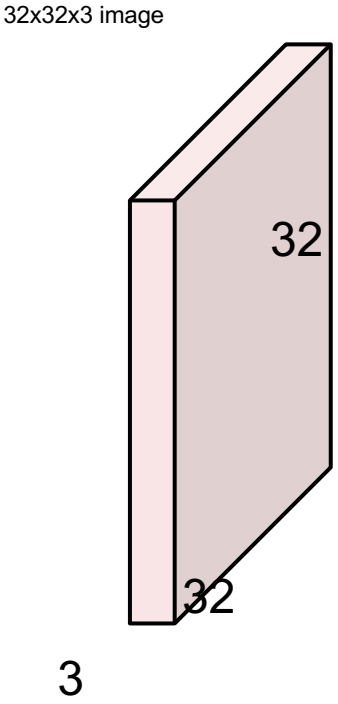


Share the same parameters across different locations.

Convolutions with learned kernels (weights)

Slide credit: Marc'Aurelio Ranzato

Convolutional layer





Convolve the filter with the image

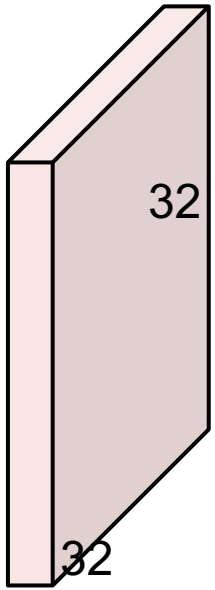
i.e. “slide over the image spatially, computing dot products”

Slide credit: CS 231N

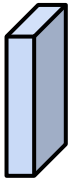
Convolutional layer

Standard filters extend the full depth of the input volume

32x32x3 image



5x5x3 filter

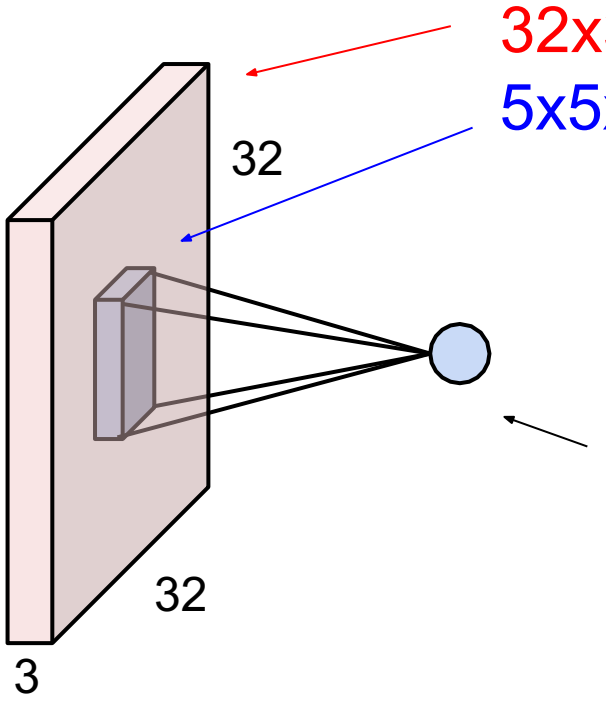


Convolve the filter with the image

i.e. “slide over the image spatially, computing dot products”

Slide credit: CS 231N

Convolutional layer

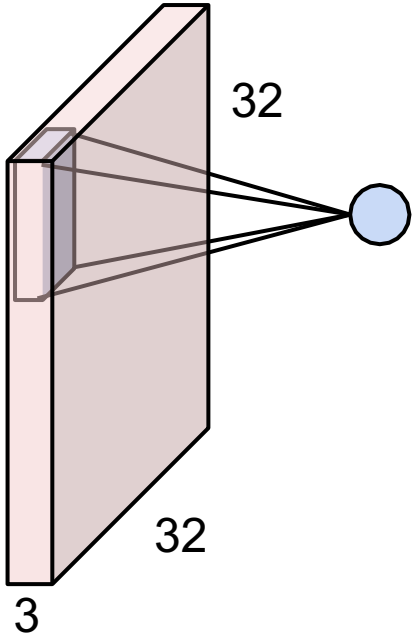


1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image (i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

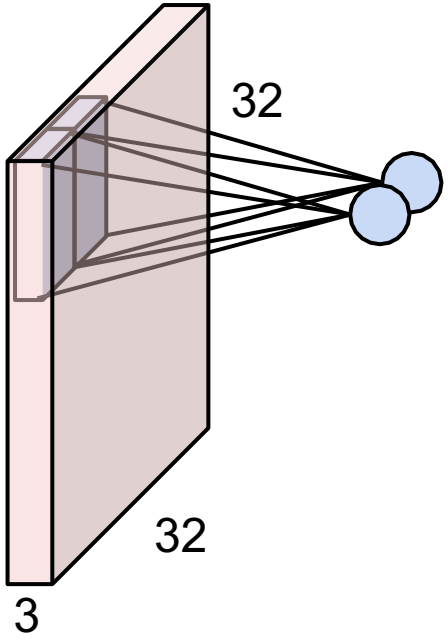
Slide credit: CS 231N

Convolutional layer



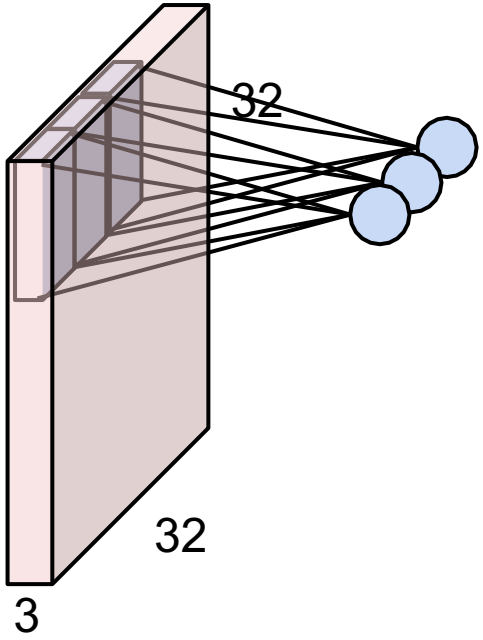
Slide credit: CS 231N

Convolutional layer



Slide credit: CS 231N

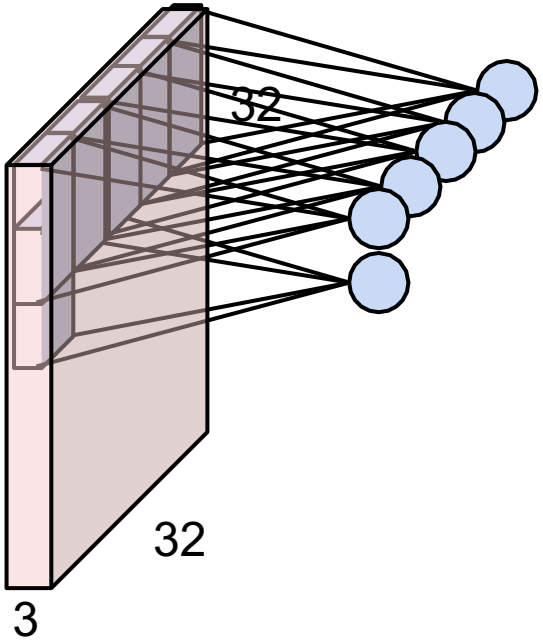
Convolutional layer



Slide credit: CS 231N

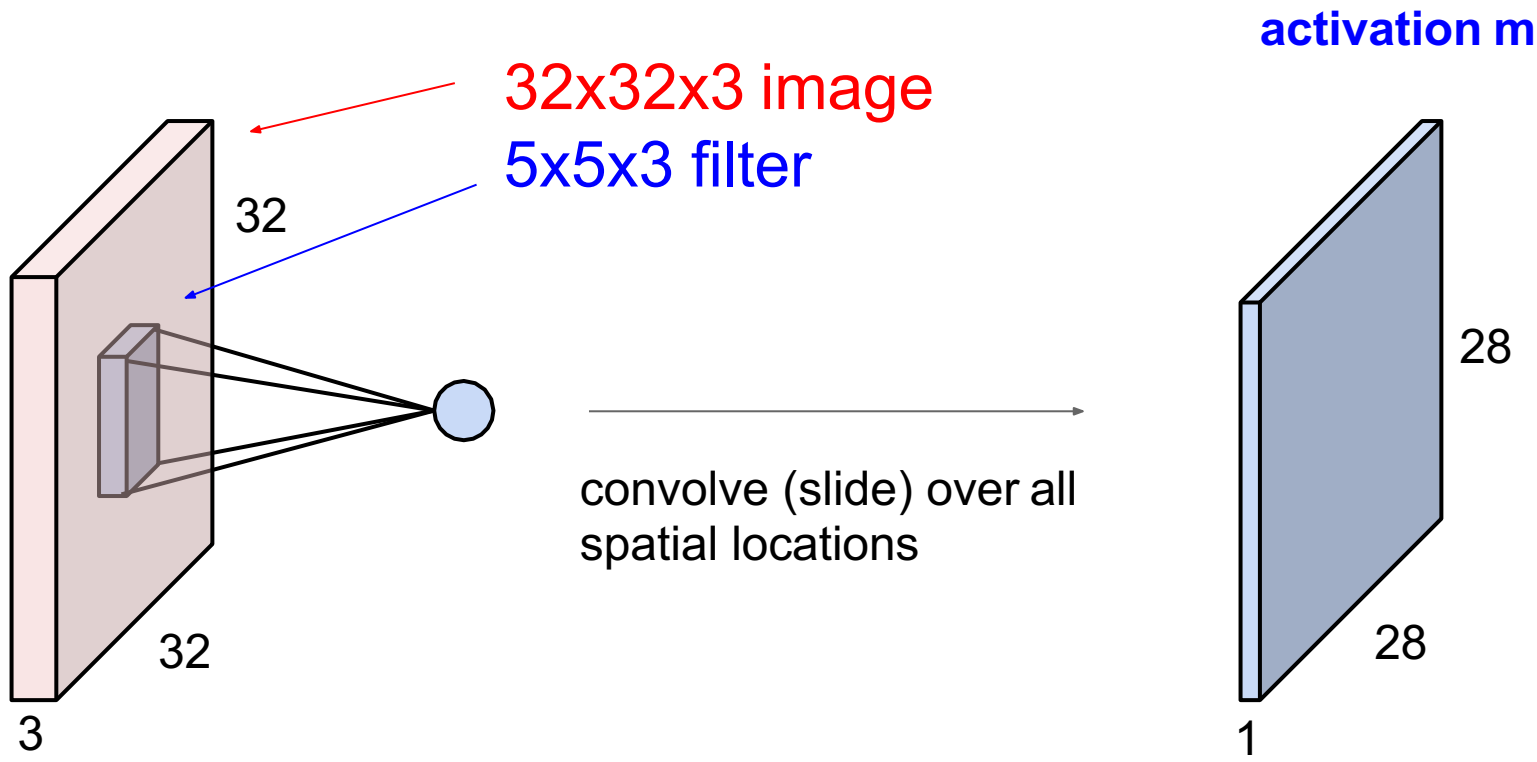
USC Viterbi School of Engineering

Convolutional layer



Slide credit: CS 231N

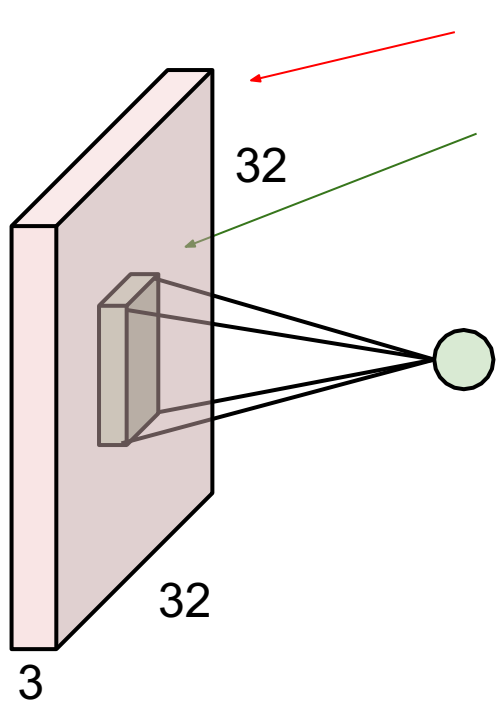
Convolutional layer



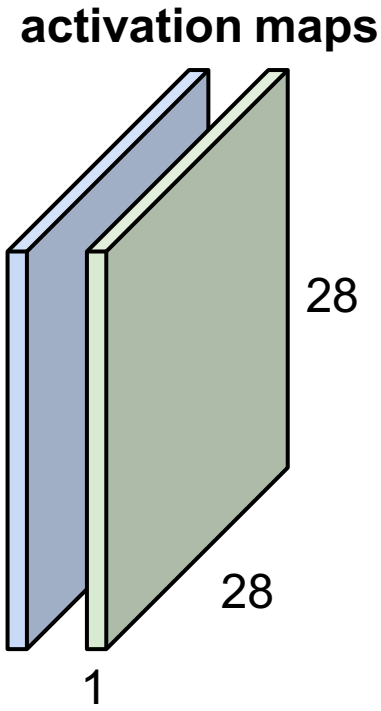
Slide credit: CS 231N

Convolutional layer

consider a second, green filter

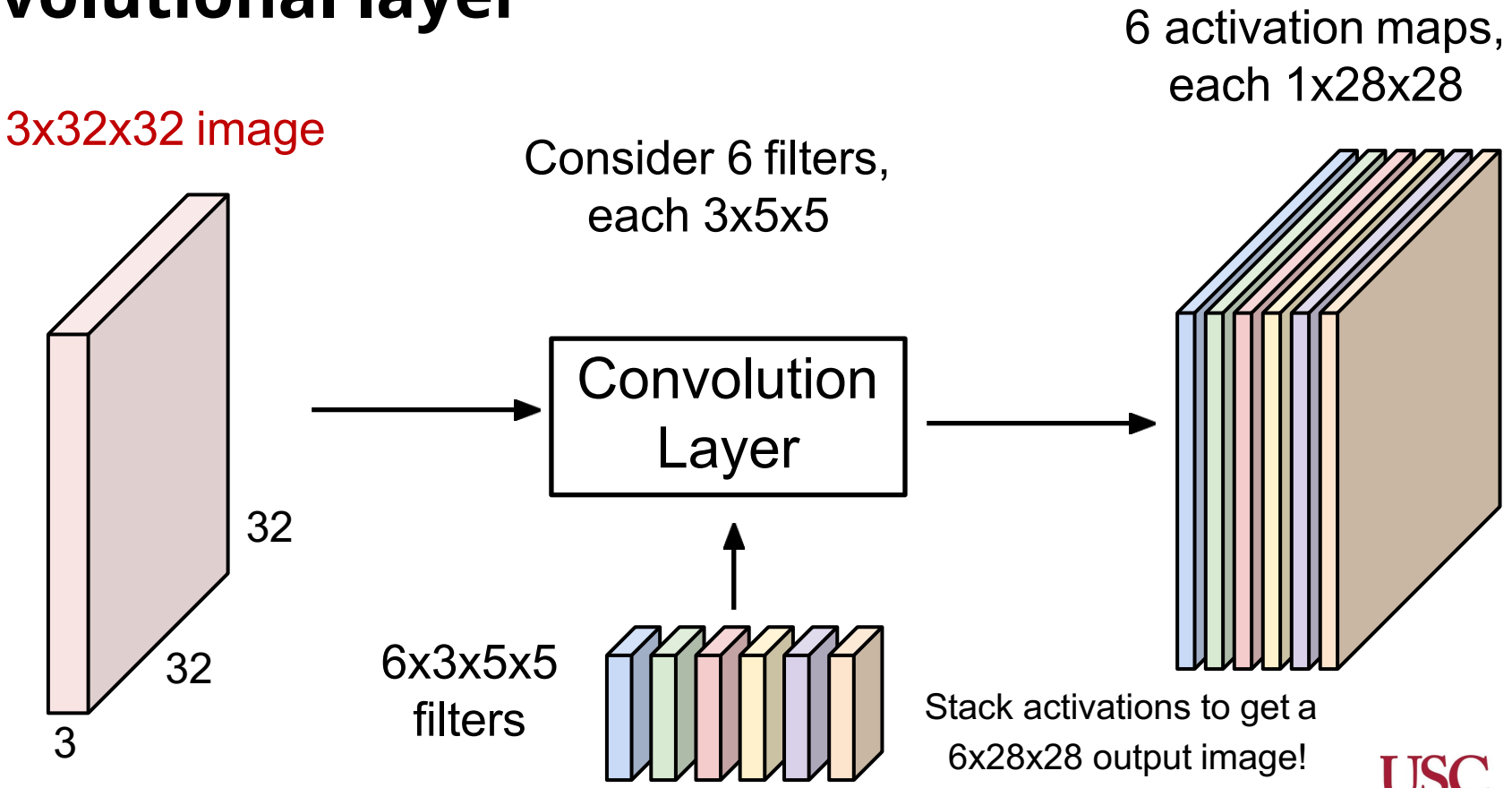


convolve (slide) over all spatial locations



Slide credit: CS 231N

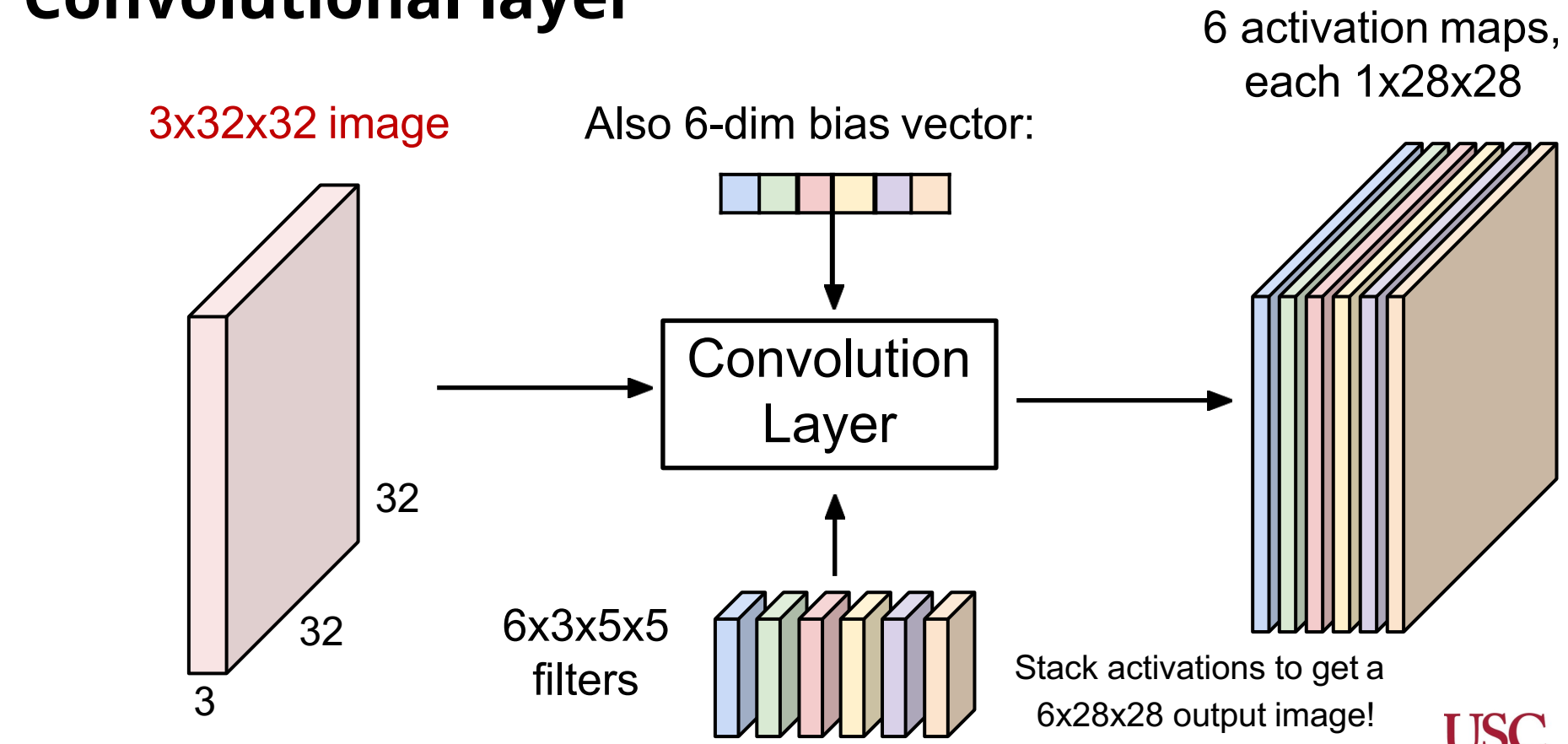
Convolutional layer



Slide inspiration: Justin Johnson

Slide credit: CS 231N

Convolutional layer



Slide inspiration: Justin Johnson

Slide credit: CS 231N

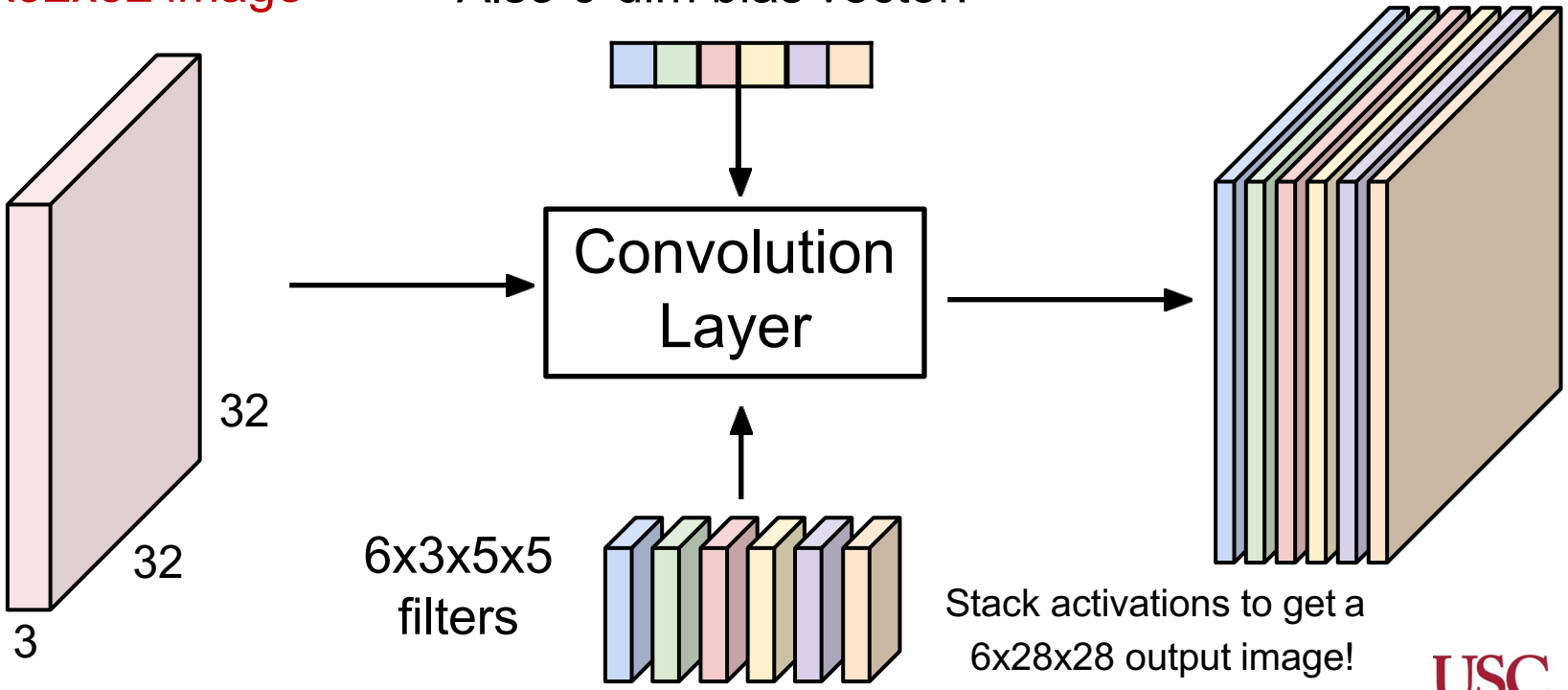
USC Viterbi School of Engineering

Convolutional layer

28x28 grid, at each point a 6-dim vector

3x32x32 image

Also 6-dim bias vector:

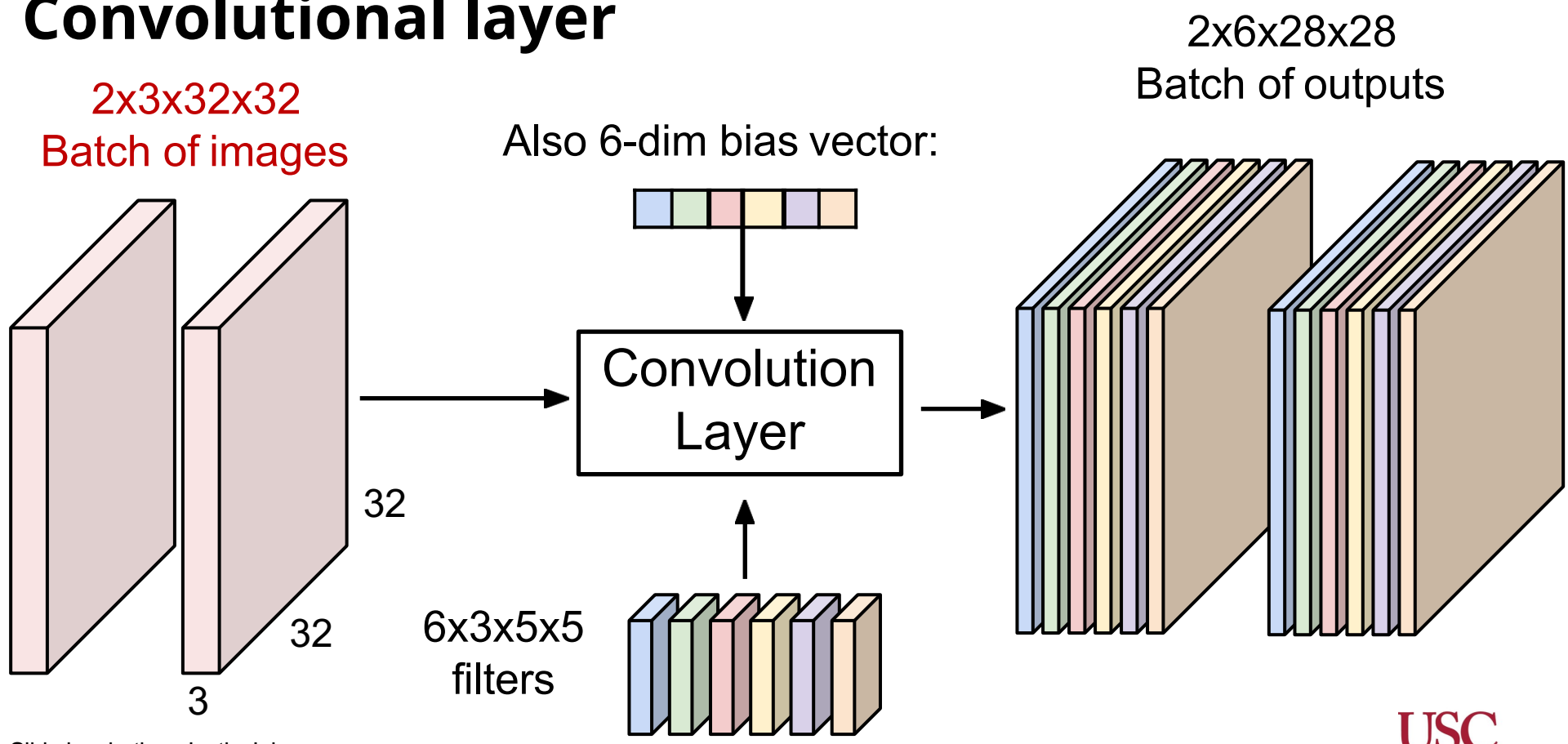


Stack activations to get a 6x28x28 output image!

Slide inspiration: Justin Johnson

Slide credit: CS 231N

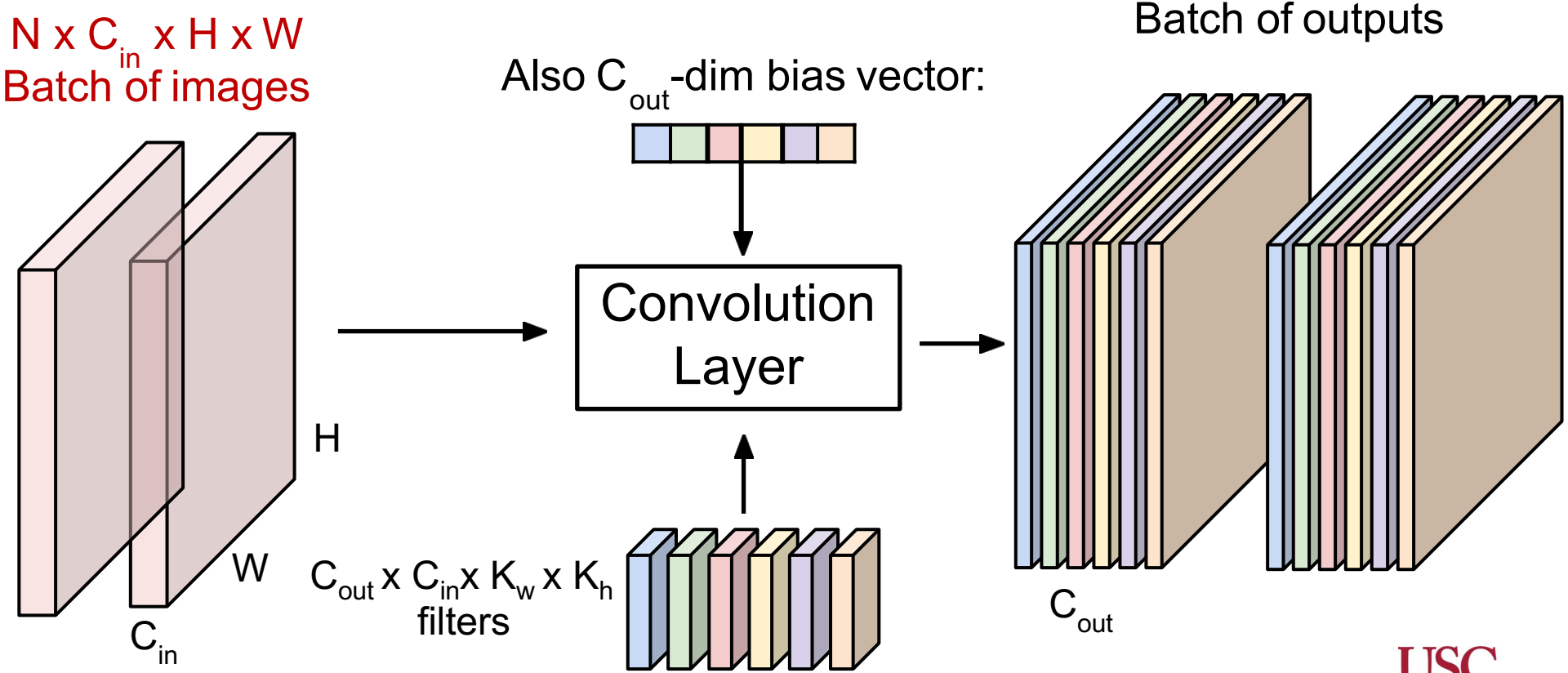
Convolutional layer



Slide inspiration: Justin Johnson

Slide credit: CS 231N

Convolutional layer

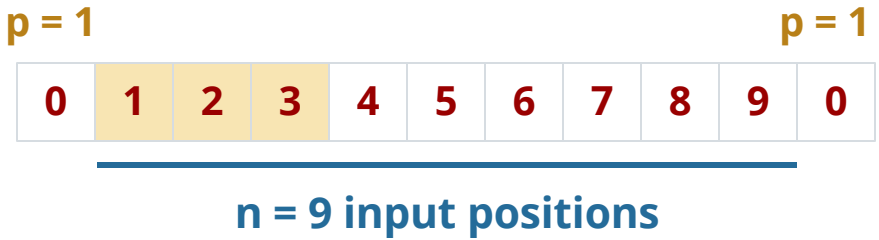


Slide inspiration: Justin Johnson

Slide credit: CS 231N

Four Symbols for One Convolution Row

A standard convolution with input width n = 9, padding p = 1, kernel size k = 3 and stride s = 2.



Here: k = 3, s = 2

Lowercase n: width
Uppercase N: often batch size

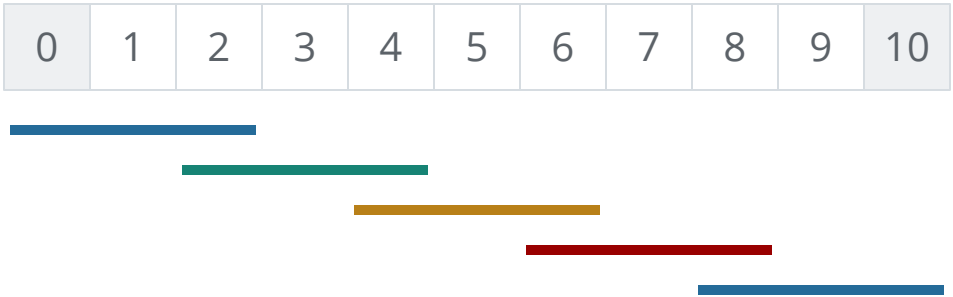
n	Input width before padding
p	Zero cells added on each side
k	Window width
s	Step between window starts

Lowercase n is a spatial length, while uppercase N often denotes batch size.

How Many Windows Fit? Output Size

With n = 9, p = 1, k = 3 and s = 2, windows start at padded indices 0, 2, 4, 6 and 8, giving 5 outputs.

n = 9, p = 1, k = 3, s = 2



Padded indices 0…10
Gray padding adds two cells.
Valid starts: 0, 2, 4, 6, 8

Each window covers k = 3 adjacent cells.

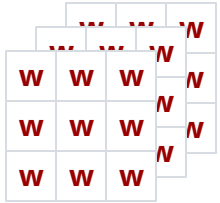
Output width = (n + 2p − k) / s + 1 = 5

(9 + 2 − 3) / 2 + 1 = 4 moves + the first window = 5 outputs.

Counting Convolution Parameters

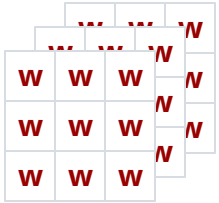
Two 3 x 3 filters over 3 input channels use 56 parameters, regardless of input size.

Filter 1



3 × 3 × 3 = 27 weights
+ 1 bias = 28

Filter 2



3 × 3 × 3 = 27 weights
+ 1 bias = 28

2 filters = 2 output channels

2 × (27 + 1) = 56 parameters

A larger input image reuses the same 56 parameters at more positions.

Kernel Size and Filter Count

Standard dense 2D convolution with 3 input channels, 2 filters and one bias per filter, using stride 1 and padding chosen to keep the spatial size.

Kernel Size	Spatial Span	Parameters	What It Reads
1×1	1×1	8	Channels at one site
3×3	3×3	56	9 spatial sites
5×5	5×5	152	25 spatial sites

All rows: 3 input channels, 2 filters; one bias per filter.

2 → 4 filters doubles output channels, with the same spatial span.

A 1×1 Convolution Mixes Channels at One Spot

A standard dense 1×1 convolution with 2 input channels, 1 output channel, weights (1, 2) and bias −1. Here x1 and x2 are channel entries at one position.

Position 1: two channels

2	5

$1 \times 2 + 2 \times 5 - 1$ → **11**

Position 2: two channels

4	1

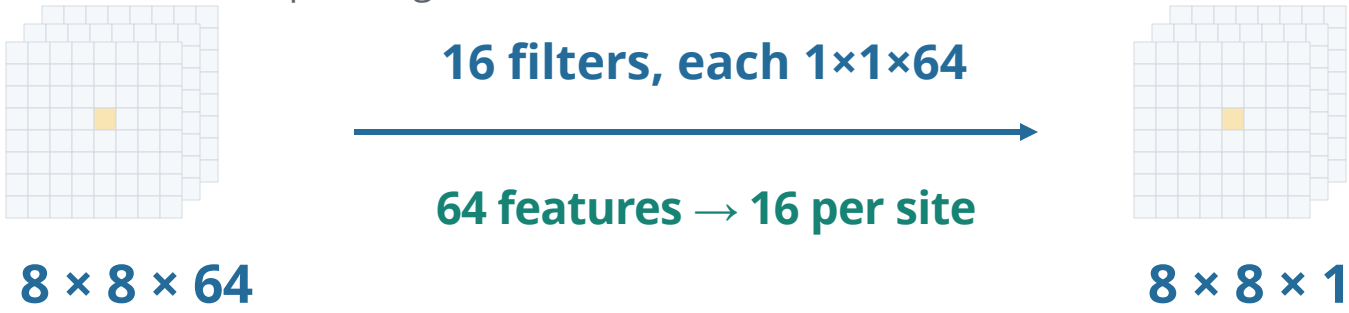
$1 \times 4 + 2 \times 1 - 1$ → **5**

The 1×1 filter never reads neighboring positions; it only forms a learned combination of the channels stacked at one spot.

Why 1×1 Convolutions Shrink Channels First

A feature map of shape 8×8×64 (height × width × channels) passes through 16 filters of size 1×1 with stride 1 and padding 0, giving 8×8×16.

H × W × C; stride 1, padding 0



Path to 32 Channels	Parameters (Biases Included)
Direct 3×3: 64 → 32	18,464
1×1: 64 → 16, then 3×3: 16 → 32	1,040 + 4,640 = 5,680

Fewer parameters; the two paths can represent different functions.

Check: Adding Filters to a 3×3 Layer

A 3×3 convolution (stride 1, padding 1) maps an 8×8×64 tensor (H×W×C) to 8×8×16. Only the filter count changes, from 16 to 24. What changes?

A Output channels become 24; output height, width, and each filter's 3×3×64 shape stay the same.

B Each filter grows spatially, so it covers more neighboring positions per window.

C Output height and width grow, while the output channel count stays at 16.

D Each filter now reads 24 input channels instead of all 64.

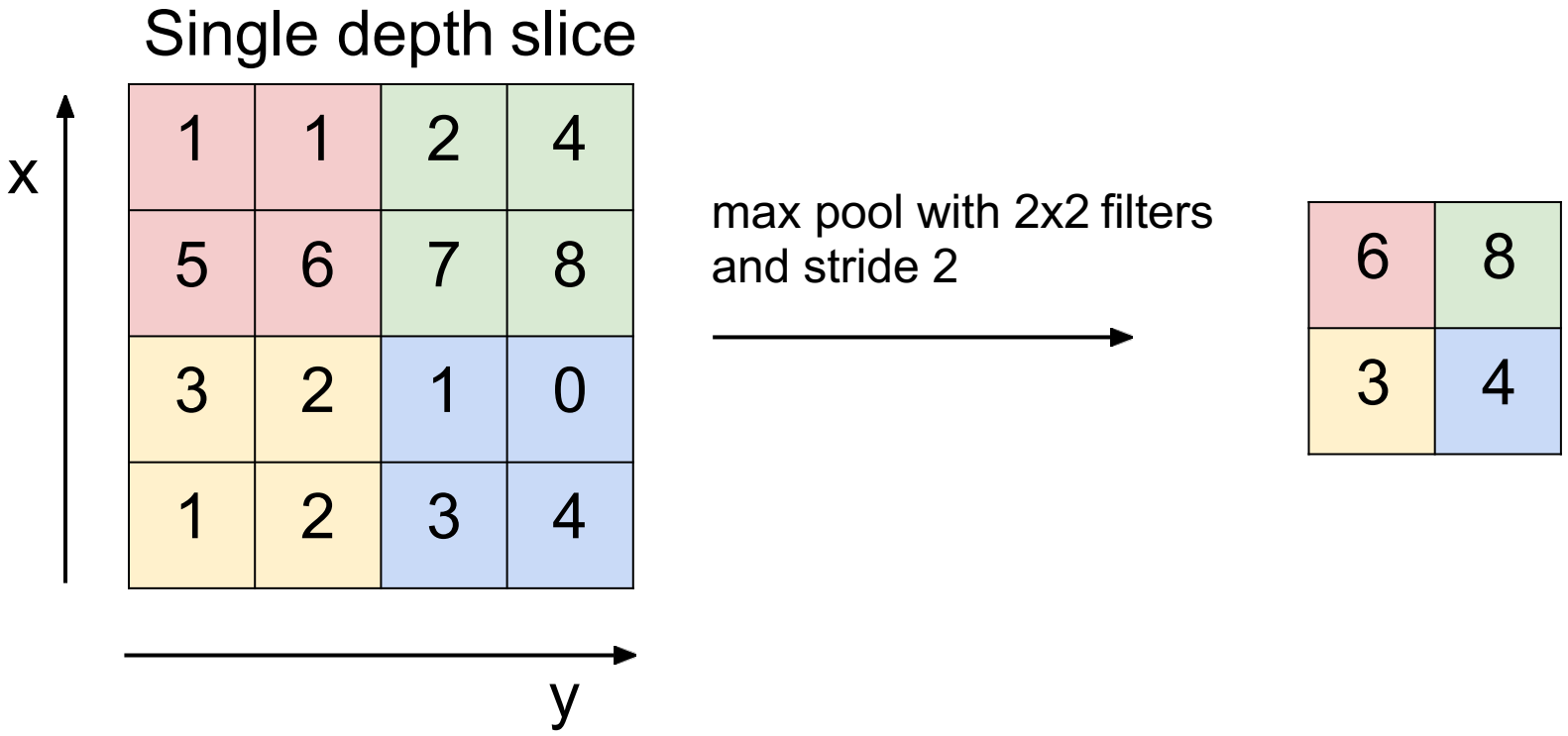
Answer A: Filter Count Sets Output Channels

A 3×3 layer (stride 1, padding 1) on an 8×8×64 input (H×W×C) changes only its filter count, from 16 to 24.

Answer: A

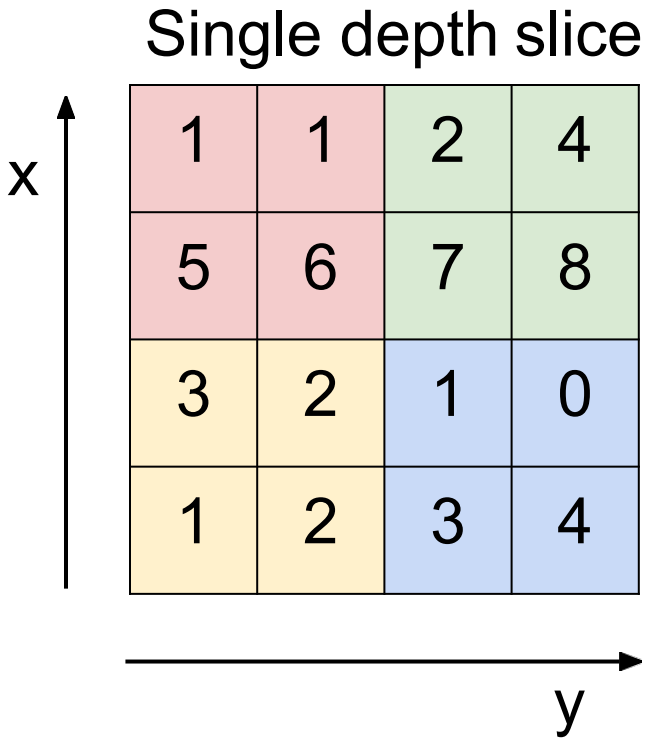
A	**Correct: one filter makes one output channel, and kernel, stride and padding keep the 8×8 size.**
B	Wrong: each filter still reads the same 3×3 neighborhood when only filter count changes.
C	Wrong: height and width depend on kernel, stride and padding, none of which changed.
D	Wrong: in standard convolution every filter spans all input channels, here still 64.

Max Pooling



Slide credit: CS 231N

Max Pooling



max pool with 2x2 filters
and stride 2

6	8
3	4

- No learnable parameters
- Some invariance to **small** shifts

Slide credit: CS 231N

Max Pooling Is Not Linear

Additivity is a necessary test of linearity: a linear map must pass it. Max pooling fails, so max is not linear.

u	**2**	0

max(u) + max(v) = 2 + 3 = 5

v	0	**3**

max(u + v) = max(2, 3) = 3

Linearity requires f(u + v) = f(u) + f(v); max gives 3 instead of 5.

A mean over fixed entries is linear, and neither max nor mean pooling has trainable parameters.

Losses, Gradients and Updates

Training turns a score into a loss, a loss into gradients, and gradients into parameter updates.

1. A loss converts model scores into a number that training tries to reduce.
2. Backpropagation applies the chain rule, multiplying local derivatives along each path.
3. When a value feeds several branches, its gradient is the sum of the branch contributions.
4. The optimizer is a separate step that uses the computed gradients to update parameters.

Classifying One Spam Email by Its Score

A model outputs a real-valued score s (not a probability) and predicts spam when s ≥ 0. This email is known spam, so the target is y = 1.

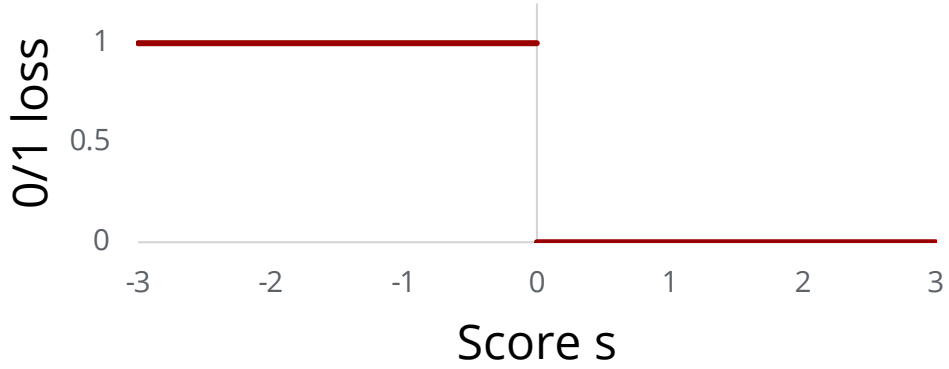
Known spam email: target y = 1

Model Score s	Prediction (Threshold 0)	0/1 Mistake Loss
−2	Not spam	1
−1	Not spam	1
+2	Spam	0

Moving the score from −2 to −1 brings it closer to spam, yet the 0/1 loss stays at 1.

Why a Mistake Count Gives No Gradient

Same known-spam email: target y = 1, real-valued score s, and spam if s ≥ 0. The red chart counts mistakes.



Known spam: y = 1

Flat regions: slope = 0

Jump at s = 0: slope undefined

A small score change within either flat region leaves the hard loss unchanged.

Next: return to the numeric prediction and squared loss from Block 3.

Squared Loss Gives a Local Direction

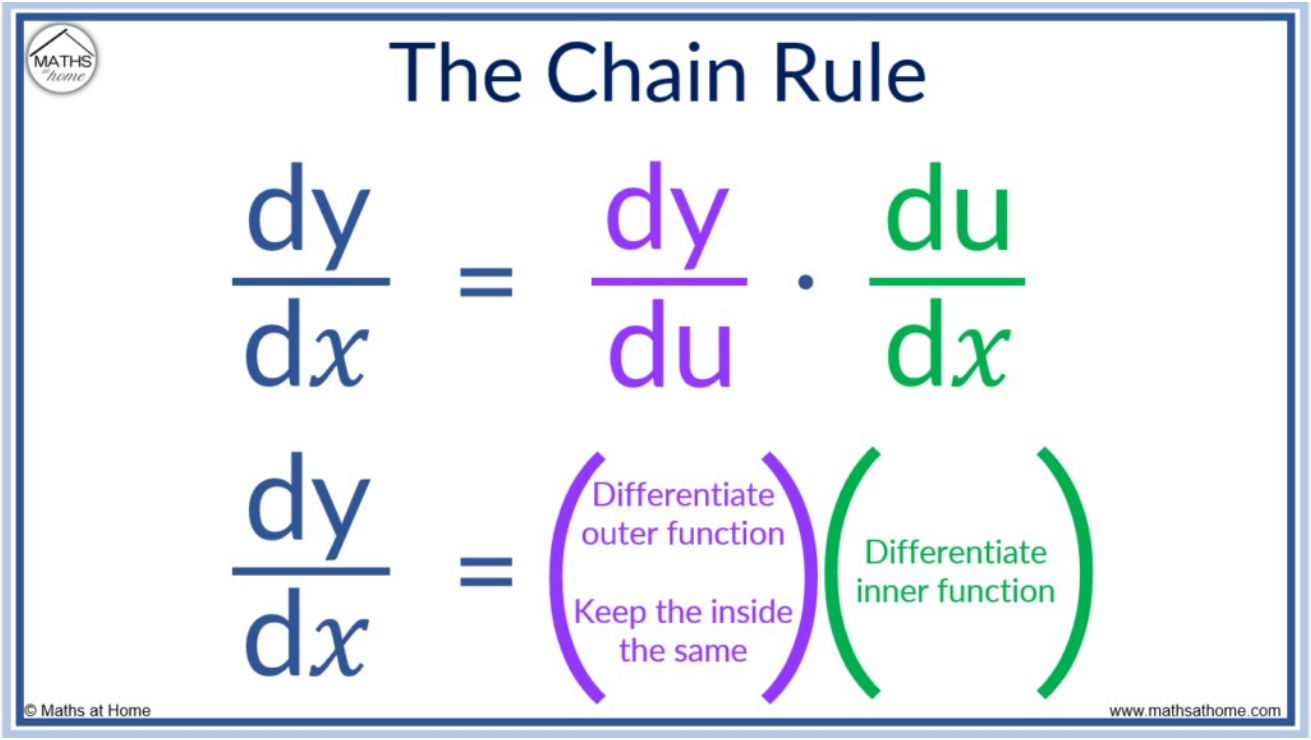
We now switch to numeric prediction from Block 3: target $t = 2$, prediction $\hat{y}$, and loss $L = (\hat{y} - t)^2$.

Numeric target $t = 2$; $L = (\hat{y} - t)^2$

Prediction $\hat{y}$	Squared Loss L	Local Slope $\partial L/\partial \hat{y}$
0	4	−4
1	1	−2
3	1	+2

A negative slope calls for raising $\hat{y}$ a little; a positive slope calls for lowering it.

Chain Rule Revisited



USC Viterbi School of Engineering

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

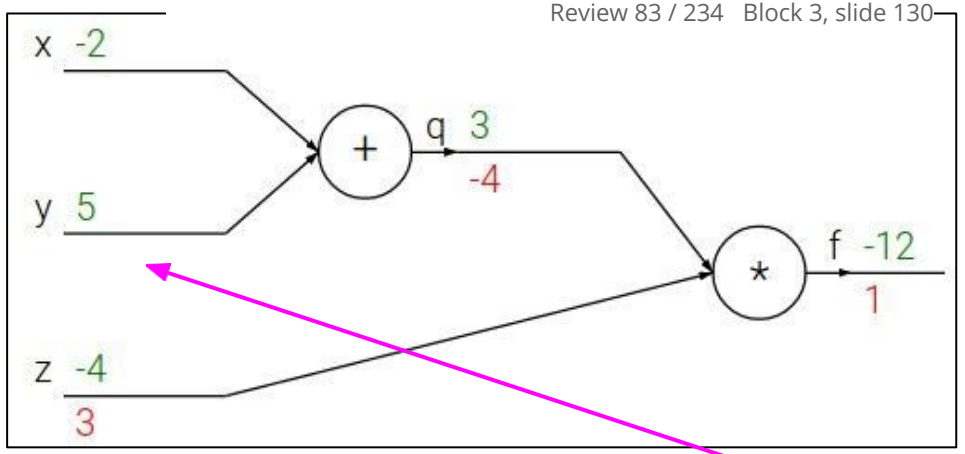
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

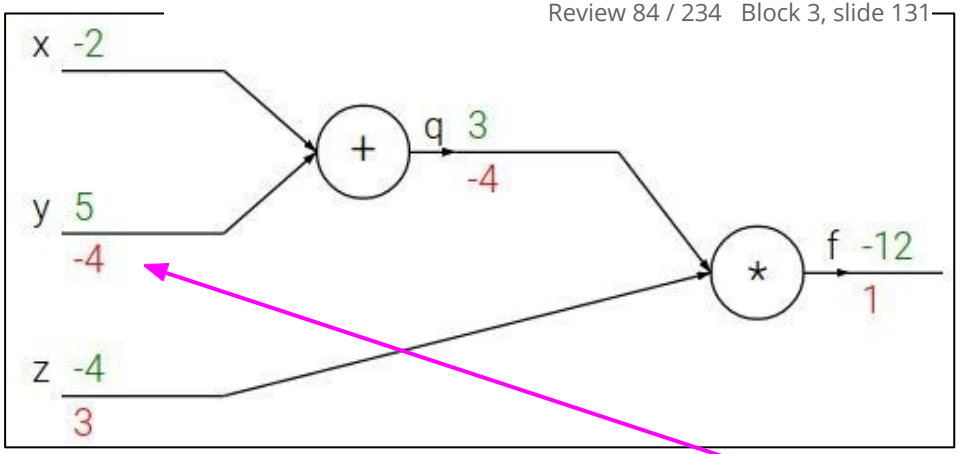
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

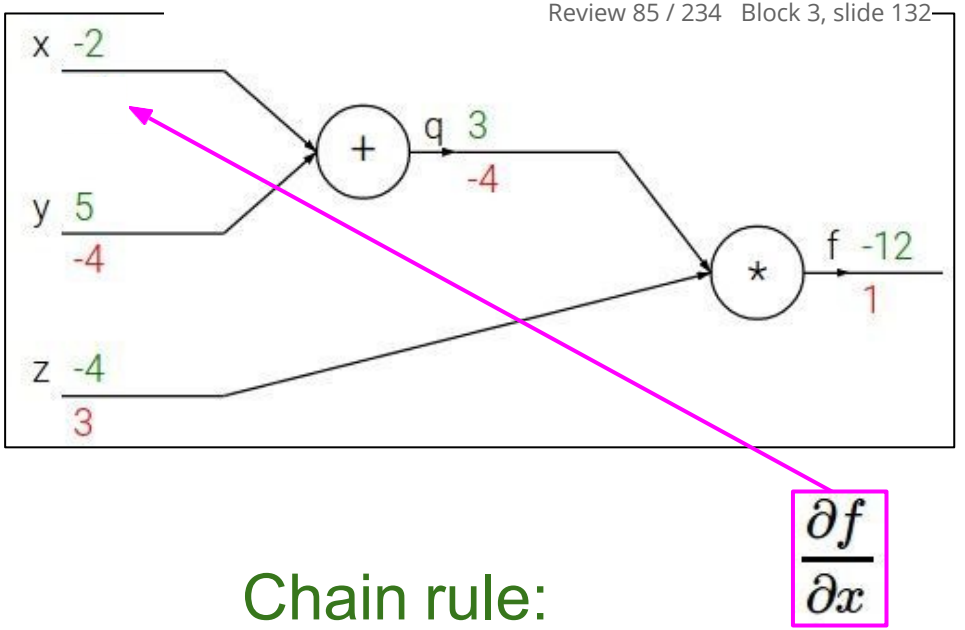
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

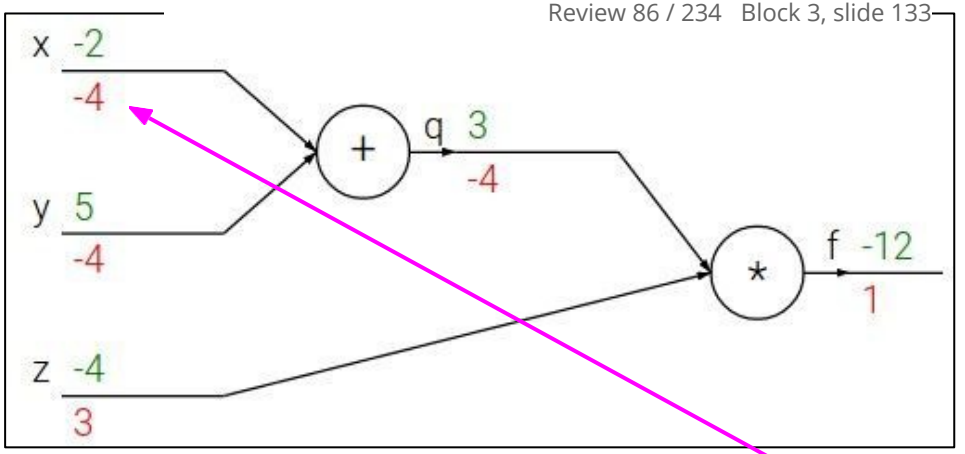
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

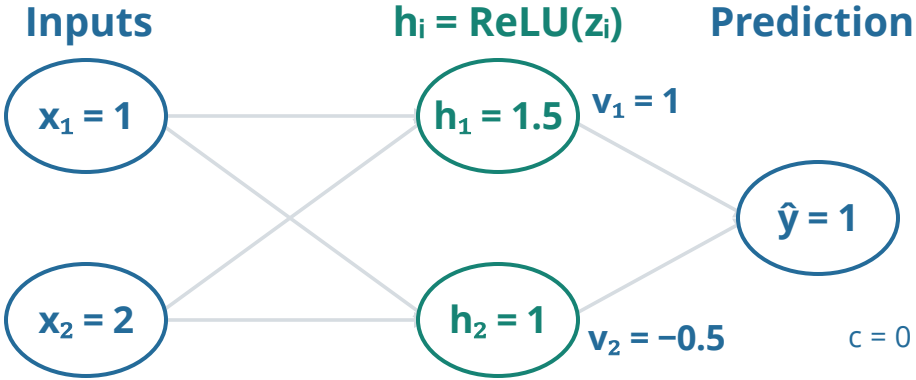
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

A 2-2-1 Network for a Regression Toy

Inputs x1 = 1 and x2 = 2 (no physical units) feed two ReLU hidden units and one output ŷ. The network is trained toward target t = 2 with squared error.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$$z_i = \Sigma_j w_{ij} x_j + b_i$$

$$h_i = \max(0, z_i)$$

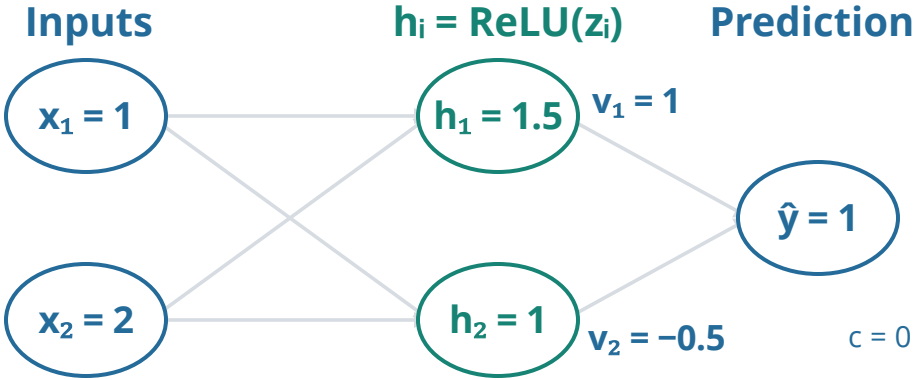
$$\hat{y} = v_1 h_1 + v_2 h_2 + c$$

$$L = (\hat{y} - t)^2$$

W: input → hidden
v: hidden → output

Forward Pass with the Initial Weights

Same 2-2-1 network and initial parameters, with inputs x1 = 1, x2 = 2 and target t = 2. Each hidden unit computes z_i, then h_i = max(0, z_i).



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$z_1 = 1\times1 + 0.5\times2 - 0.5$
$= 1.5$; $h_1 = 1.5$

$z_2 = -1\times1 + 1\times2 + 0$
$= 1$; $h_2 = 1$

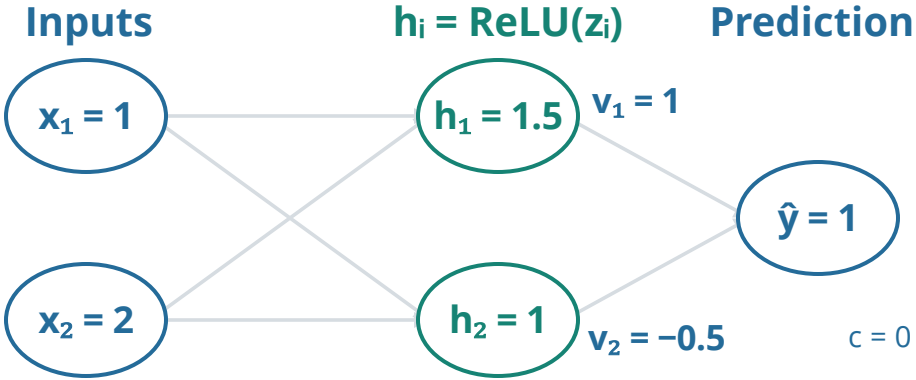
$\hat{y} = 1\times1.5 - 0.5\times1 + 0 = 1$

$L = (1 - 2)^2 = 1$

USC Viterbi School of Engineering

Start Backpropagation at the Prediction Error

Same network at its initial weights: $\hat{y} = 1$, $t = 2$ and $L = (\hat{y} - t)^2$, with $h1 = 1.5$, $h2 = 1$, $v1 = 1$, $v2 = -0.5$ and $c = 0$.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$L = (\hat{y} - t)^2$, $t = 2$

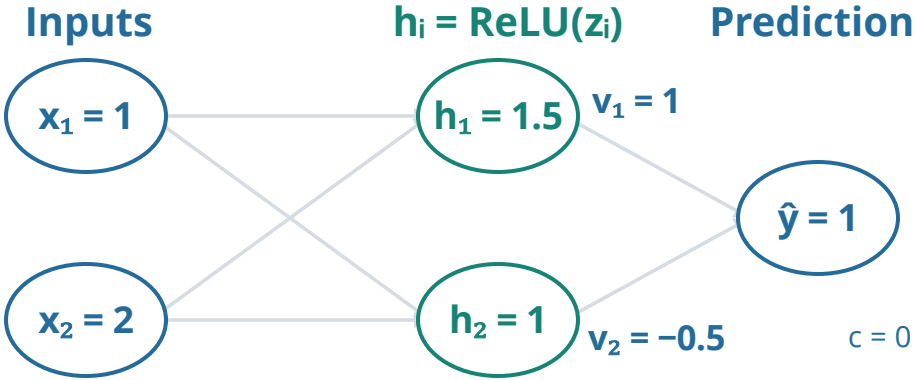
$e = \hat{y} - t = 1 - 2 = -1$

$\partial L / \partial \hat{y} = 2e = -2$

Negative slope: raising $\hat{y}$ a little lowers the loss.

Gradients for the Output Weights and Bias

The upstream signal is $\partial L/\partial \hat{y} = -2$, with $\hat{y} = v1 \cdot h1 + v2 \cdot h2 + c$, $h1 = 1.5$ and $h2 = 1$. Each gradient is $\partial L/\partial \hat{y}$ times what the parameter multiplies.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

Original $\hat{y} = 1$; $L = (\hat{y} - 2)^2$

$\partial L/\partial \hat{y} = -2$

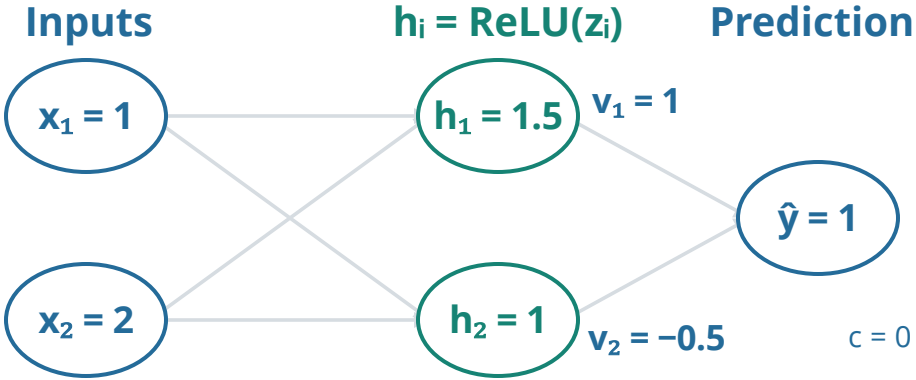
$\partial L/\partial v_1 = (-2)h_1 = -3$

$\partial L/\partial v_2 = (-2)h_2 = -2$

$\partial L/\partial c = (-2)\times 1 = -2$

Passing the Signal Back to the Hidden Units

Same context: $\partial L/\partial \hat{y} = -2$, v1 = 1, v2 = −0.5, z1 = 1.5 and z2 = 1. Define $\delta_i = \partial L/\partial z_i$, the loss sensitivity to hidden unit i's sum.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$\partial L/\partial h_1 = (-2)\times 1 = -2$

$\partial L/\partial h_2 = (-2)\times(-0.5) = +1$

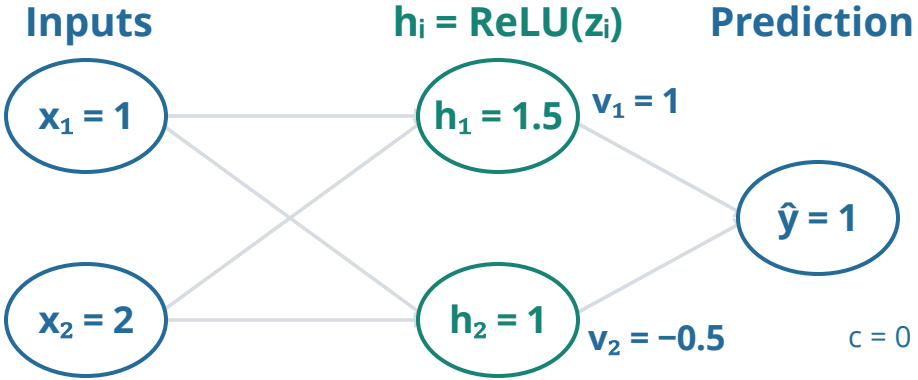
$z_1 = 1.5$, $z_2 = 1$: ReLU' = 1

$\delta_i \equiv \partial L/\partial z_i$

$\delta_1 = -2$; $\delta_2 = +1$

Gradients for the First-Layer Weights and Biases

Same context: inputs x = [1, 2] and hidden signals δ = [−2, 1], where δ_i = ∂L/∂z_i. Weight w_ij multiplies input j into hidden unit i.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1	0.5	−0.5
Hidden 2	−1	1	0

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$\delta_i \equiv \partial L/\partial z_i$; $\delta = (-2, +1)$

$\partial L/\partial w_{ij} = \delta_i \times x_j$

$\partial L/\partial W = [\ -2, -4; +1, +2\ ]$

$\partial L/\partial b_i = \delta_i$
$\partial L/\partial b = (-2, +1)$

Column 2 = 2 × column 1 because $x_2 = 2x_1$.

All Nine Gradients from One Forward Pass

Every gradient below comes from the original forward pass ($\hat{y} = 1$, $t = 2$, $L = 1$). The columns are parameter, old value and gradient $\partial L/\partial$parameter.

Parameter	Old Value	Gradient $\partial L/\partial$Parameter
w_{11}	1	−2
w_{12}	0.5	−4
w_{21}	−1	+1
w_{22}	1	+2
b_1	−0.5	−2
b_2	0	+1
v_1	1	−3
v_2	−0.5	−2
c	0	−2

w_{ij}: input $j \rightarrow$ hidden i; b_i: hidden bias; v_i: hidden $i \rightarrow$ output; c: output bias.

At $x = (1, 2)$, target $t = 2$: original prediction $\hat{y} = 1$; $L = (\hat{y} - t)^2$.

One Gradient-Descent Step with η = 0.05

Each parameter moves by new = old − η · gradient, with positive learning rate η = 0.05 and the nine gradients from the original forward pass.

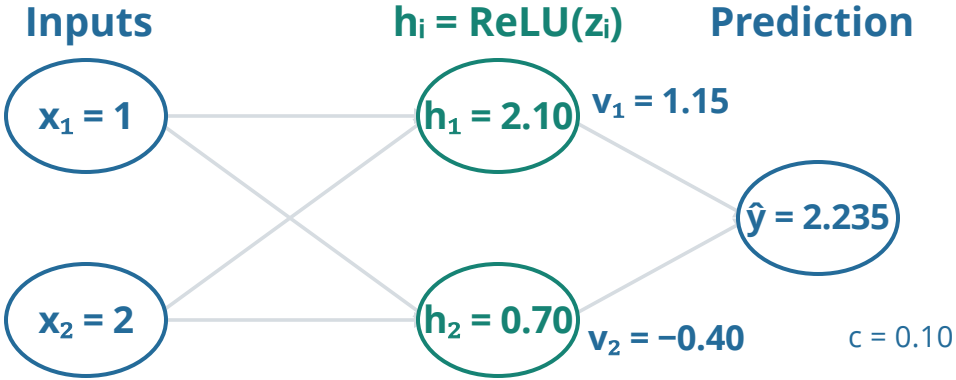
Parameter	Old	Gradient	New = Old − 0.05×Gradient
w_{11}	1	−2	1.10
w_{12}	0.5	−4	0.70
w_{21}	−1	+1	−1.05
w_{22}	1	+2	0.90
b_1	−0.5	−2	−0.40
b_2	0	+1	−0.05
v_1	1	−3	1.15
v_2	−0.5	−2	−0.40
c	0	−2	0.10

η = 0.05: learning rate. All gradients use the original forward pass.

At x = (1, 2), target t = 2: original prediction $\hat{y} = 1$; $L = (\hat{y} - t)^2$.

Rerunning the Network with Updated Weights

Same inputs x1 = 1, x2 = 2 and target t = 2, now with the updated parameters. Every activation is freshly recomputed and compared with its old value.



W (Weights)	Input x_1	Input x_2	Bias b
Hidden 1	1.10	0.70	−0.40
Hidden 2	−1.05	0.90	−0.05

w_{ij}: input j → hidden i; c: output bias; t=2; $L=(\hat{y}-t)^2$

$h_1 = \mathrm{ReLU}(1.10 + 1.40 - 0.40) = 2.10$

$h_2 = \mathrm{ReLU}(-1.05 + 1.80 - 0.05) = 0.70$

$\hat{y} = 1.15 \times 2.10 - 0.40 \times 0.70 + 0.10 = 2.235$

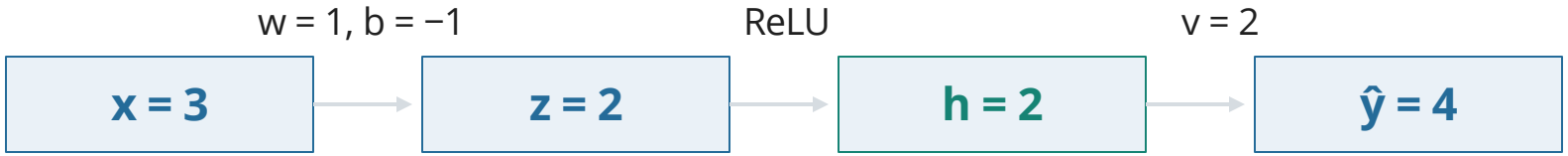
$L = (2.235 - 2)^2 = 0.055225$

Old loss: 1

USC Viterbi School of Engineering

Chain Rule Through One Hidden Unit

A single path from weight to loss gives a gradient of 12 as a product of four factors.



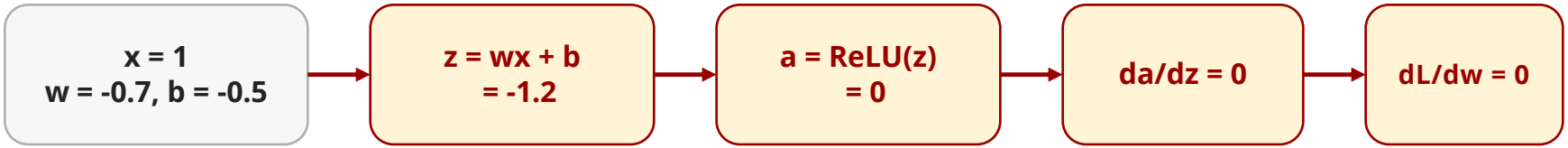
$t = 2, \quad L = \tfrac{1}{2}(\hat{y} - t)^2 = 2$ $\qquad$ $e = \hat{y} - t = 2$

$$\frac{\partial L}{\partial w} = \underbrace{(4-2)}_{\text{error}} \times \underbrace{2}_{\text{output weight}} \times \underbrace{1}_{\text{ReLU'}} \times \underbrace{3}_{\text{input}} = 12$$

$L = e^2$: $\partial L/\partial \hat{y} = 2e$ $\qquad$ $L = \tfrac{1}{2}e^2$: $\partial L/\partial \hat{y} = e$

All derivatives use the original values, computed before any update.

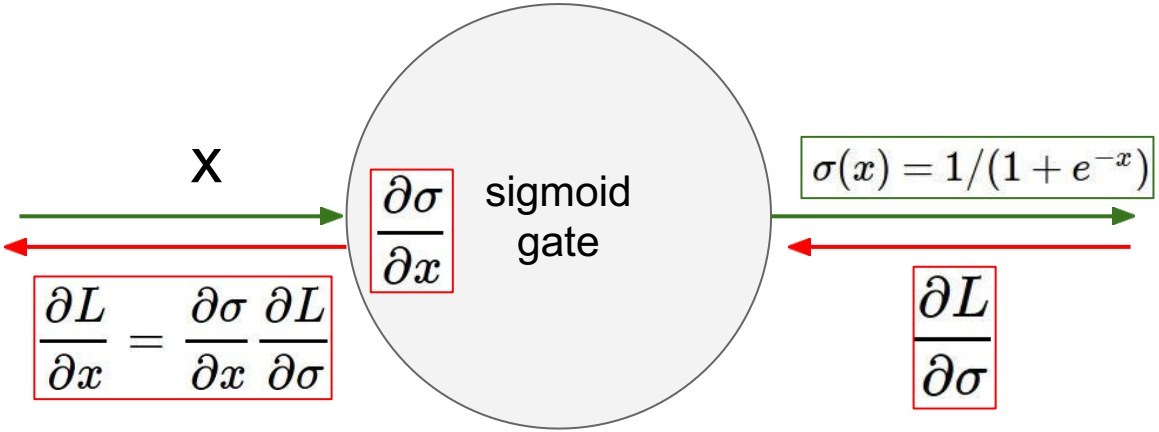
Dead ReLU: A Numeric Path to Zero Gradient

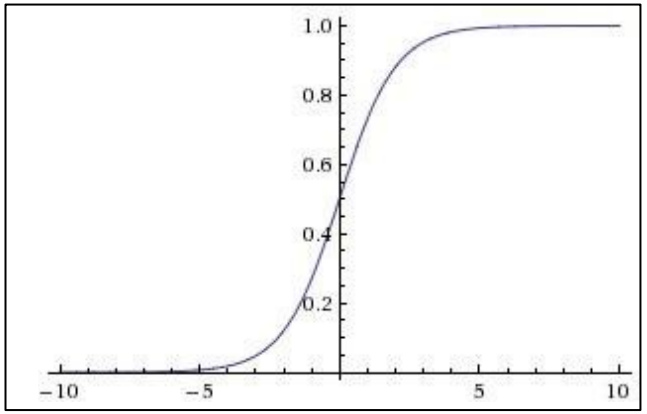


If this unit stays on the negative side for relevant training examples, the local gradient cannot move its incoming weights.

The issue is a whole negative region with zero local derivative, not the single point z = 0.

A dead ReLU outputs zero and blocks the gradient on the data that keep its pre-activation negative.





What happens when x = -10?

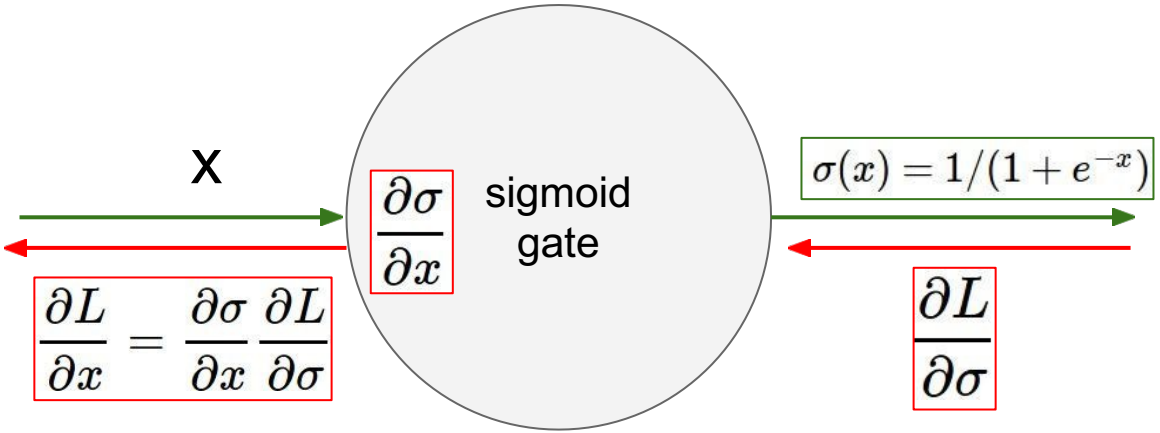
What happens when x = 0?

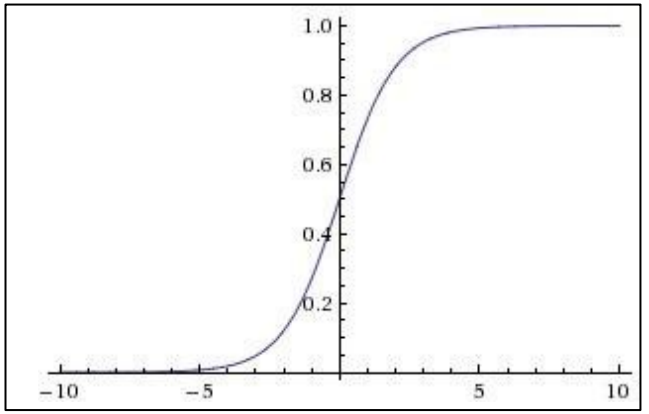
What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

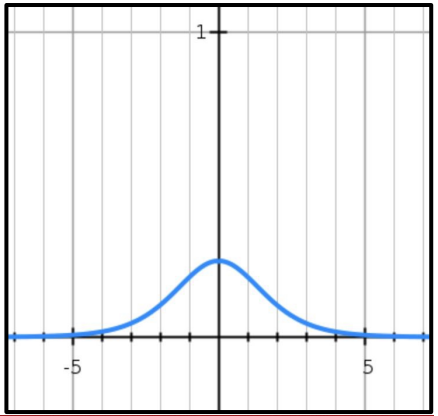
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)) = 1(1-1) = 0$$

USC Viterbi School of Engineering





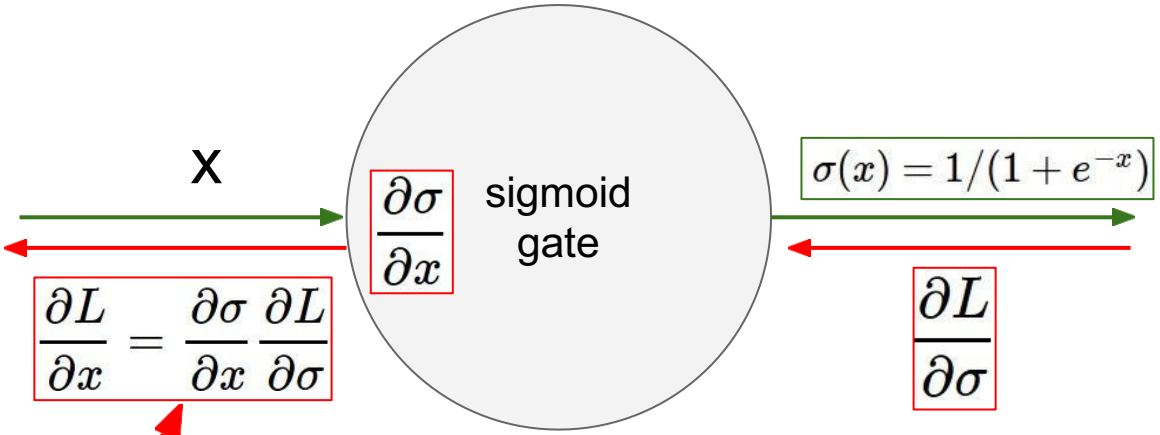
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

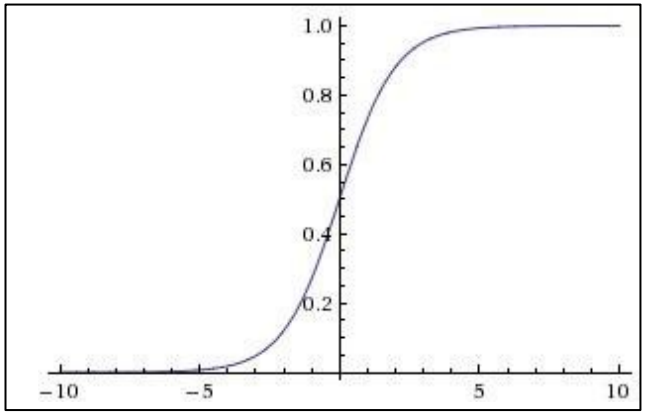


What happens when x = -10?

What happens when x = 0?

What happens when x = 10?





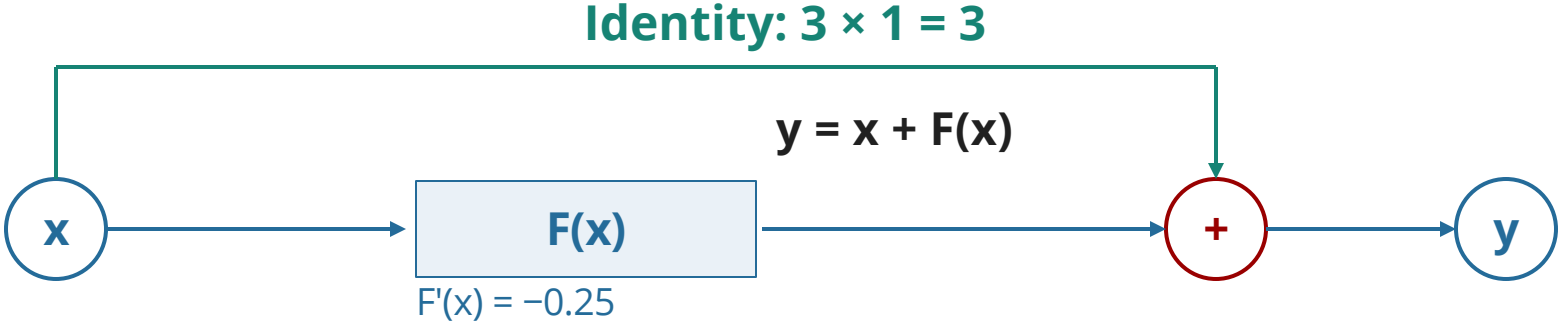
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

Why is this a problem?

Then the gradients flowing back are tiny (about 4.5e-5 here) and the weights barely move

Gradients Through a Residual Connection

In y = x + F(x), the gradient to x is the sum of an identity path and a branch path.



∂L/∂x = 3 − 0.75 = 2.25

The branch cancels part of the identity: 1 + F'(x) = 0.75, so 2.25 < 3.

Gradient clipping: **solution** for exploding gradient

- Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

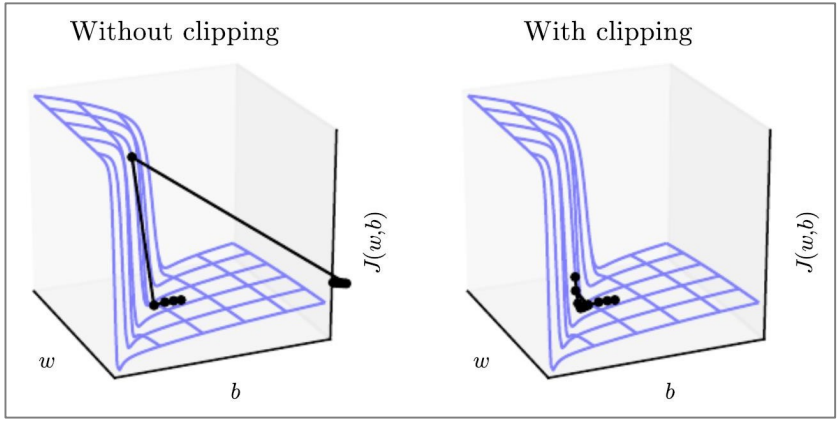
Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$
end if

- Intuition: take a step in the same direction, but a smaller step

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. http://proceedings.mlr.press/v28/pascanu13.pdf

Gradient clipping: **solution** for exploding gradient

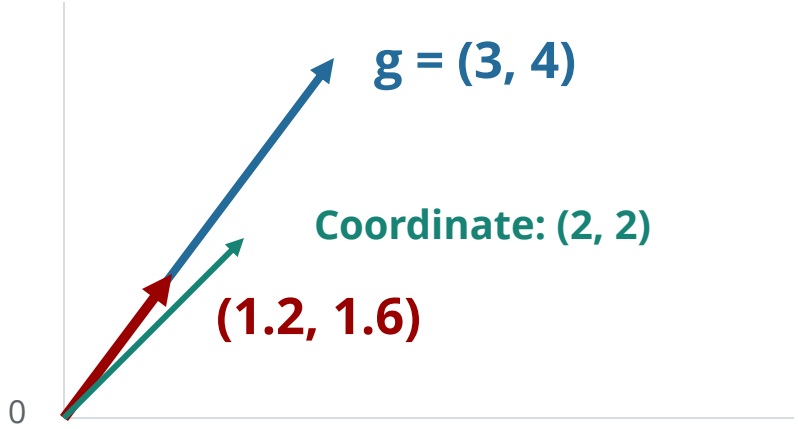


- This shows the loss surface of a simple RNN (hidden state is a scalar not a vector)
- The "cliff" is dangerous because it has steep gradient
- On the left, gradient descent takes two very big steps due to steep gradient, resulting in climbing the cliff then shooting off to the right (both bad updates)
- On the right, gradient clipping reduces the size of those steps, so effect is less drastic

Source: "Deep Learning", Goodfellow, Bengio and Courville, 2016. Chapter 10.11.1. https://www.deeplearningbook.org/contents/rnn.html

Global Gradient Norm Clipping

Clipping rescales the whole gradient vector so its norm is at most the threshold.



$\|g\|_2 = 5, \; c = 2$

Scale = 2 / 5

$g' = \min(1, c/\|g\|_2)\, g$

Global clipping applies one scale, so the gradient direction is kept.

Unchanged if norm <= threshold; coordinate clipping caps each entry.

Gradient Direction and Step Size

The gradient and the learning rate play different roles.

Current weight: 2.0 Gradient: −0.5 Learning rate: 0.1

$$\text{New weight} = 2.0 - 0.1 \times (-0.5) = 2.05$$

The gradient is local. The chosen step must still be checked.

The learning rate controls how far we move.

Course teaching example. Sources and assumptions in speaker notes.

Check: Reading the Sign of a Gradient

Gradient descent uses a positive learning rate η and the update $w \leftarrow w - \eta \cdot (dL/dw)$. For one weight w, backpropagation reports $dL/dw < 0$. What does the update do to w?

A Decreases w, because a negative slope means raising w raises the loss locally.

B Leaves w unchanged, because a negative slope signals a minimum.

C Increases w, because raising w lowers the loss locally.

D Flips the sign of η, so the step follows the gradient direction.

Answer C: Negative Slope Means Increase w

The update is $w \leftarrow w - \eta \cdot (dL/dw)$ with $\eta > 0$, and the reported derivative dL/dw is negative.

Answer: C

A Wrong: a negative slope means the loss falls locally as w rises, not that it grows.

B Wrong: a minimum has zero slope; a nonzero slope means a small move can lower the loss.

C **Correct: $-\eta \cdot (dL/dw)$ is positive, so w rises, though a finite step still needs checking.**

D Wrong: η stays positive, and the minus sign in the update already points downhill.

Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Full sum expensive when **N** is large!

Approximate sum using a **minibatch** of examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent

while True:
  data_batch = sample_training_data(data, 256) # sample 256 examples
  weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
  weights += - step_size * weights_grad # perform parameter update
```

Momentum Reuses Recent Gradient Direction

Updates can zigzag even when the long-term direction is consistent.



Momentum smooths the update direction over time.

Course teaching example. Sources and assumptions in speaker notes.

SGD + Momentum:

continue moving in the general direction as the previous iterations

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
  dx = compute_gradient(x)
  x -= learning_rate * dx
```

SGD+Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$x_{t+1} = x_t - \alpha v_{t+1}$$

- Build up “velocity” as a running mean of gradients
- Rho gives “friction”; typically rho=0.9 or 0.99

Sutskever et al, “On the importance of initialization and momentum in deep learning”, ICML 2013

Momentum Remembers Past Gradients

Same current gradient g = 2, different stored histories, so the update steps can differ.

m_new = 0.5 m_old + g, w_new = w − 0.1 m_new

m_old = 0 → m_new = 2 → **Δw = −0.2**

m_old = 4 → m_new = 4 → **Δw = −0.4**

Convention: m_new = 0.5 m_old + g, then w_new = w - 0.1 m_new.

With g = 2 and m_old = 4, m_new = 4 and the step is -0.4, so history matters.

Reliable Training and Transfer

Choices about initialization, normalization, regularization, and frozen layers shape how training behaves.

1. Identical starts keep symmetric hidden units identical under deterministic updates; random starts break symmetry.
2. Normalization layers can behave differently in training mode and evaluation mode.
3. Regularization strength is a hyperparameter, so validation data decides whether it helps generalization.
4. In transfer learning, you choose which pretrained layers stay frozen and which remain trainable.

Why Identical Hidden Units Stay Identical

Suppose two hidden units start with exactly the same parameters.

Same start

Same inputs and parameters produce the same outputs.

Same update

Under symmetric connections, they receive the same gradients.

Different random starts let hidden units learn different features.

Course teaching example. Sources and assumptions in speaker notes.

Weight initialization

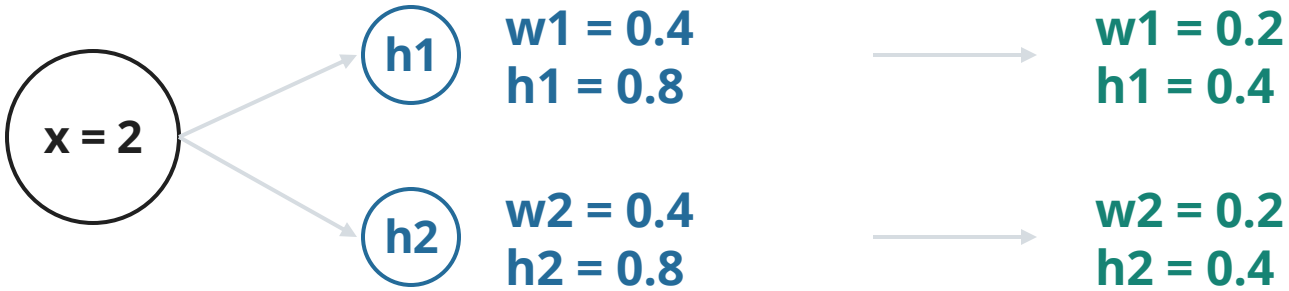
- Small random numbers
- Xavier
- He / Kaiming: ReLU-aware scale

Slide credit: CS 231N

Why Equal Starting Weights Stay Equal

Two identical hidden units receive equal gradients, so one SGD step keeps them doing the same computation.

Task: two identical hidden units, h = w times x



dL/dh = 1 each, so dL/dw = 2 each

SGD, rate 0.1: 0.4 minus 0.1 times 2 = 0.2

Different start: w1 = 0.3, w2 = 0.7; h1 = 0.6, h2 = 1.4 (different computations)

Normalization Inside the Network

Input preprocessing happens once. Hidden activations change during training.



Normalization also changes the optimization behavior.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

BatchNorm: A Complete Numeric Example

One feature across four examples: 2, 4, 6 and 8.

Input	2	4	6	8
Subtract mean 5	−3	−1	1	3
Divide by √5	−1.34	−0.45	0.45	1.34

Variance = 5. For this illustration, $\gamma = 1$, $\beta = 0$; ε is omitted.

Training learns γ and β after this normalization.

Course schematic. Sources and assumptions in speaker notes.

Normalization Adjusts Mean and Scale

Centering and scaling do not force an arbitrary distribution to become Gaussian.

What changes

Subtract the mean.
Divide by a scale estimate.

What does not follow

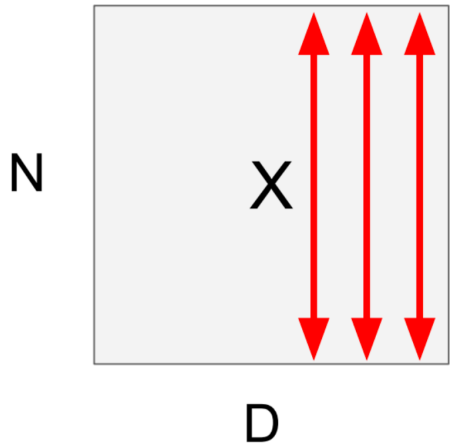
The resulting values need not follow a bell-shaped distribution.

BatchNorm then learns an additional scale and shift.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

Batch normalization

- Compute the empirical mean and variance independently for each dimension and normalize.



Batch normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

[Ioffe and Szegedy, 2015]

Batch normalization

- At test time
  - The mean and variance are not computed based on the batch.
  - Use stored running means and variances from training.

BatchNorm Behaves Differently at Evaluation

The flower classifier may receive one new photo at a time.

During training

Use mini-batch statistics.
Update running estimates.
Learn scale and shift.

During evaluation

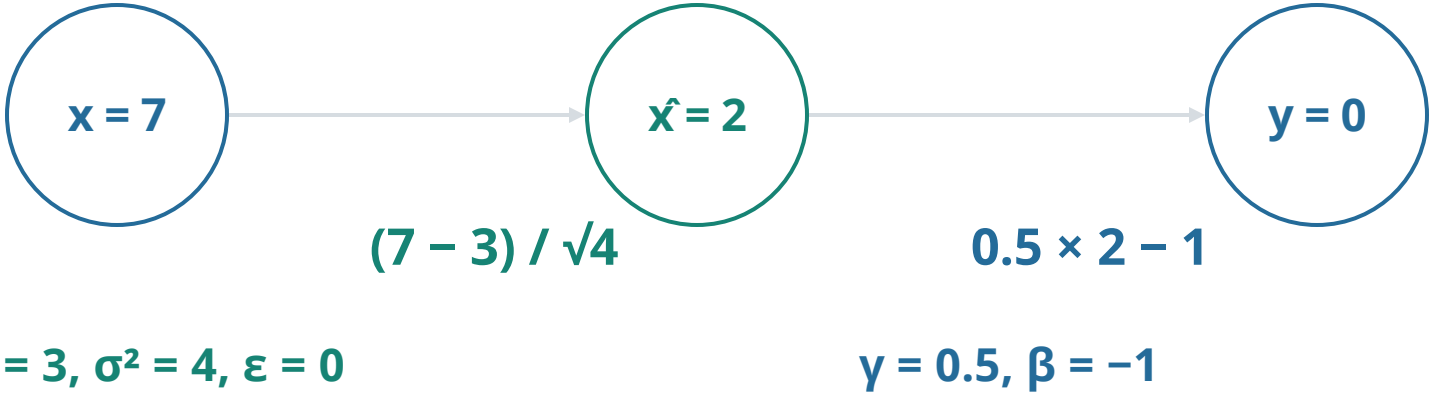
Usually use stored running statistics.
Keep the same learned scale and shift.

Switch the model to evaluation mode when evaluating.

BatchNorm: Ioffe & Szegedy (2015); Santurkar et al. (2018).

Batch Normalization in Two Stages

Batch normalization first standardizes with statistics, then applies a learned scale and shift.



Stage 1: with μ = 3, σ² = 4 and ε = 0, the input 7 becomes (7 − 3) / 2 = 2.

In standard evaluation, stored running mean and variance replace the batch statistics; γ and β stay.

BatchNorm at Evaluation Time

Stored running statistics give one photo the same normalized activation, whatever else is in the batch.

Same photo, x = 7

Wrong training mode: batch [7, 3]

7 3

$\mu = 5, \sigma^2 = 4$

$(7 - 5) / \sqrt{4} = 1$

Wrong training mode: batch [7, 11]

7 11

$\mu = 9, \sigma^2 = 4$

$(7 - 9) / \sqrt{4} = -1$

Evaluation mode: stored running mean 3, variance 4

→ **(7 minus 3) / 2 = 2 with either companion**

Illustrative numbers; epsilon 0, gamma 1, beta 0, population variance; intermediate activation only

The Purpose of a Regularization Penalty

The training loss rewards fitting labels. A penalty also constrains the weights.

Objective = data loss + λ × weight penalty

L2 penalty = sum of squared weights

λ controls the tradeoff. Choose it using validation data.

The next page lists different choices of weight penalty.

Course schematic. Sources and assumptions in speaker notes.

Regularization: Add term to loss

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_i}\max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \boxed{\lambda R(W)}$$

In common use:

L2 regularization $R(W) = \sum_k \sum_l W_{k,l}^2$ (Weight decay)

L1 regularization $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2) $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

L2 Regularization in the Objective

With equal data loss, the L2 penalty separates two weight vectors by their squared norms.

$$J(w) = \text{data loss} + \lambda \Sigma w_i^2$$

A: (3, 0) $0.4 + 0.2 \times 9 = 2.2$

B: (2, 1) $0.4 + 0.2 \times 5 = 1.4$

Both A = (3, 0) and B = (2, 1) have data loss 0.4, with squared norms 9 and 5.

Read lambda and any factor of one half from the definition; validation judges generalization.

Dropout Changes Which Features Are Available

During training, some hidden activations are randomly set to zero.

Training step A

Some feature activations are kept.
Others are temporarily zeroed.

Training step B

A new random mask keeps a different subset of activations.

The weights remain. The activation mask changes.

Course teaching example. Sources and assumptions in speaker notes.

Dropout Scaling Conventions

Let q be the probability of keeping an activation. Dropping probability is 1 − q.

Classic dropout

Training: keep it unchanged or set it to zero.
Evaluation: multiply it by q.

Inverted dropout

Training: keep it divided by q or set it to zero.
Evaluation: use it unchanged.

Each convention matches training and evaluation scales.

Course schematic. Sources and assumptions in speaker notes.

Inverted Dropout Preserves the Expected Scale

A hidden activation is 2. The keep probability is 0.5.

Kept: output 2 / 0.5 = 4. Dropped: output 0.

Expected training output = 0.5 × 4 + 0.5 × 0 = 2

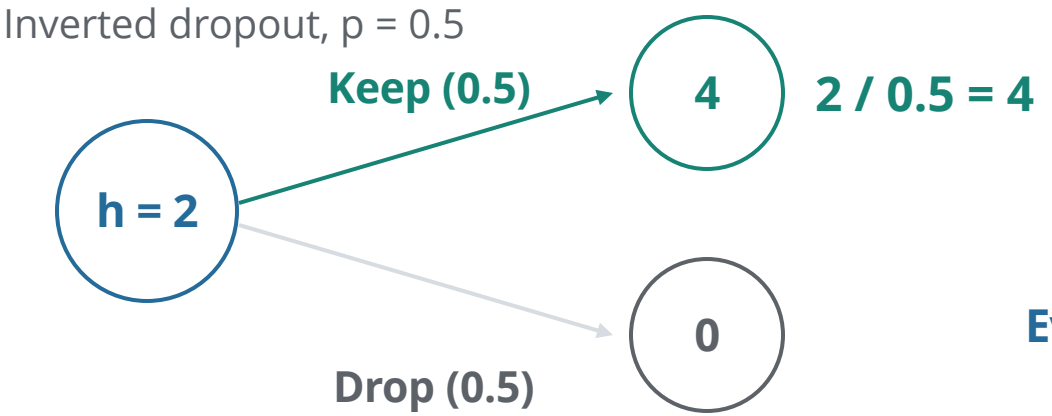
At evaluation: keep the activation as 2, with no dropout mask.

The preserved quantity is an expectation across masks.

Course teaching example. Sources and assumptions in speaker notes.

Inverted Dropout Keeps the Average

With keep probability 0.5, a kept activation is doubled, so the average training output matches evaluation.



Eval: dropout off, output 2

Average = 0.5(4) + 0.5(0) = 2

Same expected scale; one pass gives 0 or 4

Check: Inverted Dropout at Evaluation

A layer uses inverted dropout with keep probability q = 0.8. In training, a random mask zeros some activations and scales kept ones by 1.25. What happens in evaluation mode?

A Apply a random mask that drops 20% and scale the kept activations by 1.25, as in training.

B Disable the dropout mask and pass activations through with no extra scaling.

C Disable the dropout mask and multiply every activation by 0.8.

D Disable the dropout mask and multiply every activation by 1.25.

Answer B: Mask Off, No Extra Scaling

Inverted dropout with keep probability q = 0.8 scales kept activations by 1.25 during training.

Answer: B

A	Wrong: evaluation should be deterministic, so no random dropout mask is applied.
B	**Correct: the 1/q scaling already happened in training, so average sizes already match.**
C	Wrong: scaling by q at test time is the non-inverted convention and would shrink outputs here.
D	Wrong: 1.25 offsets dropped units only in training; at evaluation it would inflate outputs.

An Augmented Example Must Keep a Valid Label

For flower classification, consider a slightly different view of the same flower.



Choose transformations that preserve the task label.

Course teaching example. Sources and assumptions in speaker notes.

The Same Transformation Can Break Another Task

Rotating a digit image can change what the image means.

Flower photo

A rotated view may still show the same flower species.

Digit recognition

A rotated 6 can resemble a 9. Keeping label 6 can be wrong.

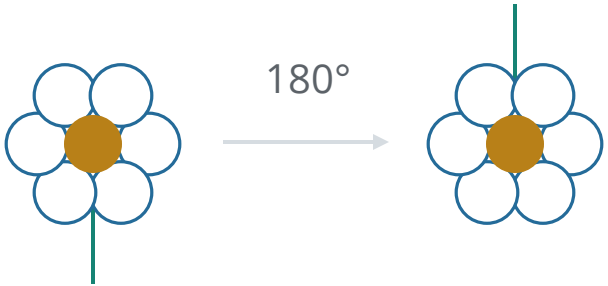
Augmentation depends on the task semantics.

Course teaching example. Sources and assumptions in speaker notes.

Augmentation Must Keep Labels Valid

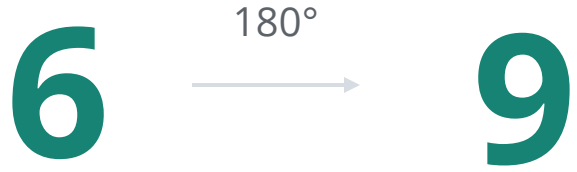
A 180 degree rotation keeps a daisy a daisy, but a rotated 6 can resemble a 9.

Upright daisy



Label: daisy (still valid)

Handwritten 6

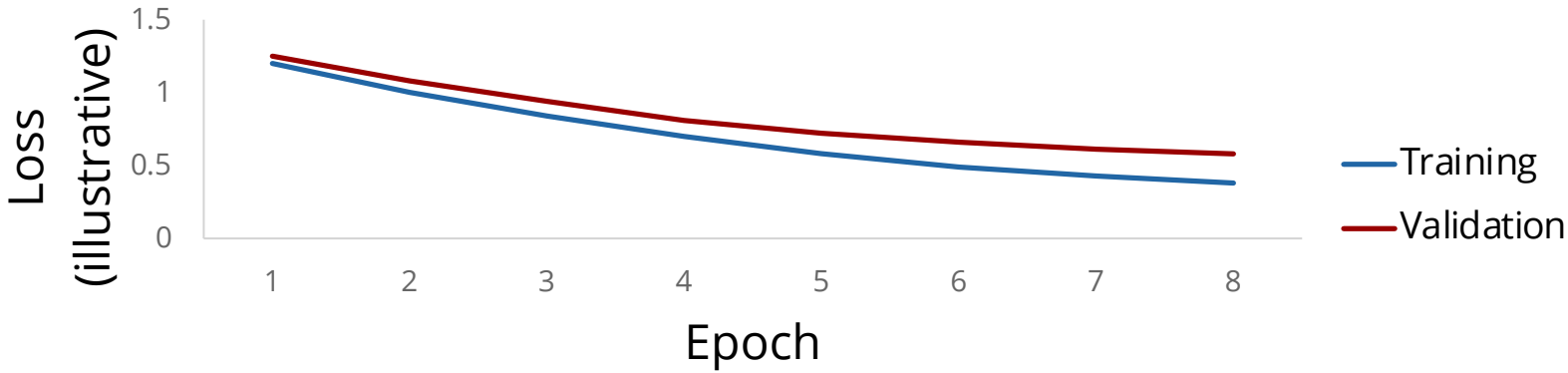


Label 6 can be wrong: may resemble 9

Augment only when the target stays valid for the task

When Training and Validation Improve Together

Evaluate both curves using comparable loss definitions and modes.

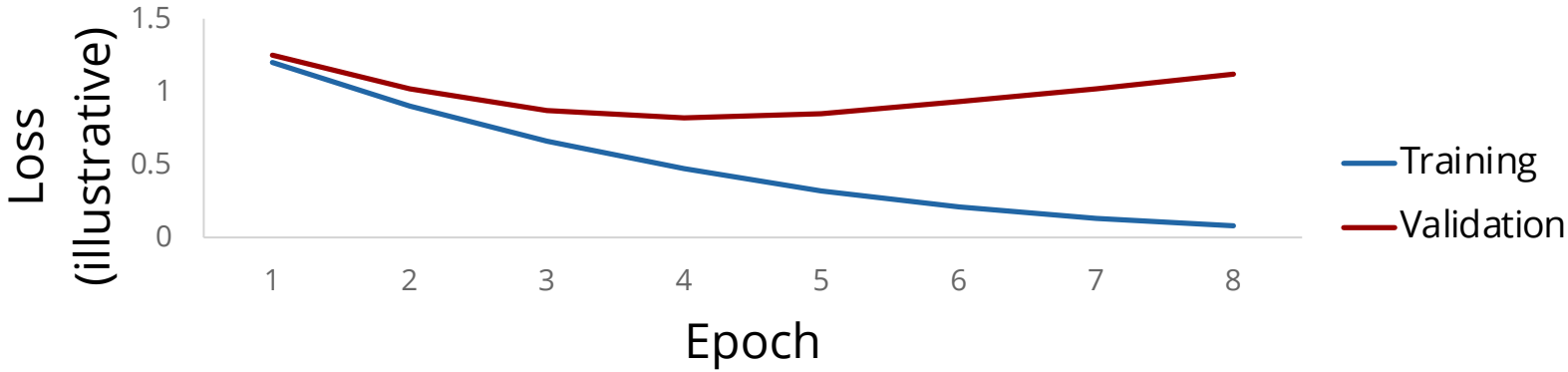


The model is learning patterns that transfer to held-out data.

Course teaching example. Sources and assumptions in speaker notes.

When Training Improves but Validation Worsens

The best training checkpoint may not be the best held-out checkpoint.



Select checkpoints using the chosen validation metric.

Course teaching example. Sources and assumptions in speaker notes.

Checkpoint Selection and Early Stopping

One noisy validation dip does not require stopping immediately.

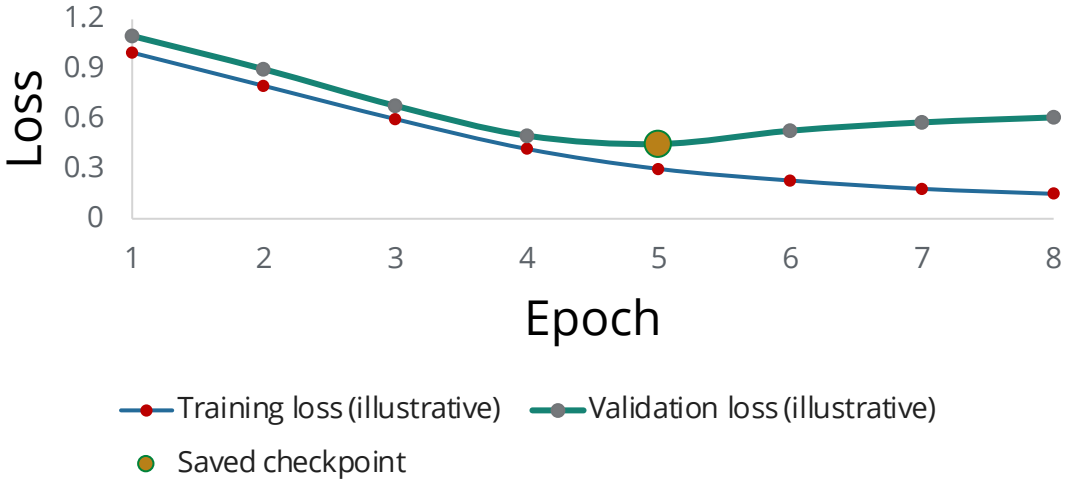
Save the best checkpoint	Keep the best value of the selected validation metric.
Wait for a pattern	A patience window can tolerate short-term noise.
Use a fixed protocol	Keep the test set out of stopping decisions.

Choose the stopping rule before inspecting test performance.

Course teaching example. Sources and assumptions in speaker notes.

Early Stopping With Checkpoints

Save the best validation checkpoint, stop after 3 epochs without improvement, and restore the saved weights.



Best validation: epoch 5, loss 0.45, saved

Epochs 6, 7, 8: no improvement, stop after 8 (patience 3)

Restore epoch 5, not epoch 8

Test set: evaluate once, after all choices are fixed

Limited Data Changes the Starting Point

Our flower dataset is small. A pretrained model already encodes visual patterns.

Train from scratch

Learn all representations from this dataset.

Transfer learning

Start with representations learned from other data.

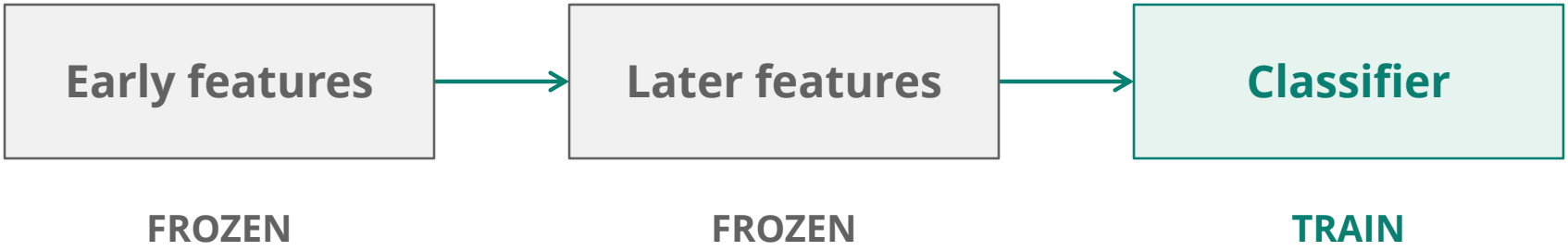
The source and target data determine how useful transfer is.

Course teaching example. Sources and assumptions in speaker notes.

Start by Training a New Classifier

Replace the original label predictor with a three-class flower head.

Stage 1: backbone = the pretrained feature extractor. Keep it fixed.



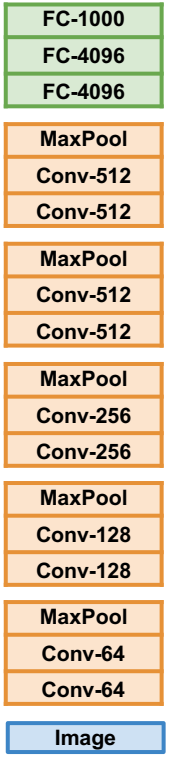
Frozen features still participate in the forward pass.

Course teaching example. Sources and assumptions in speaker notes.

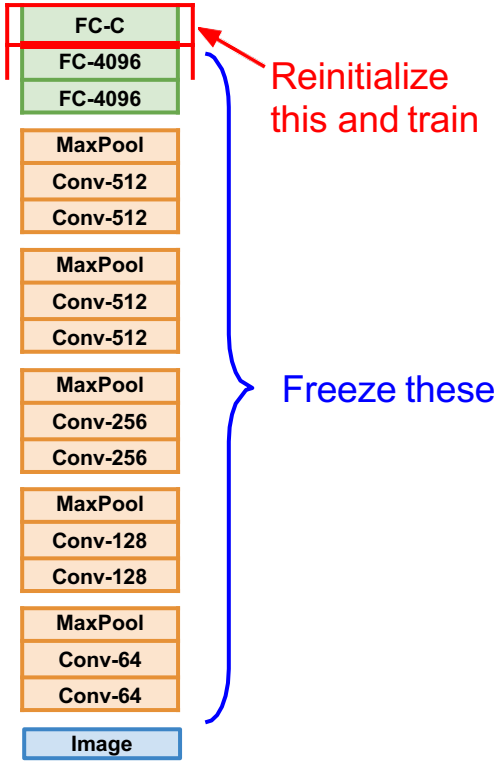
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

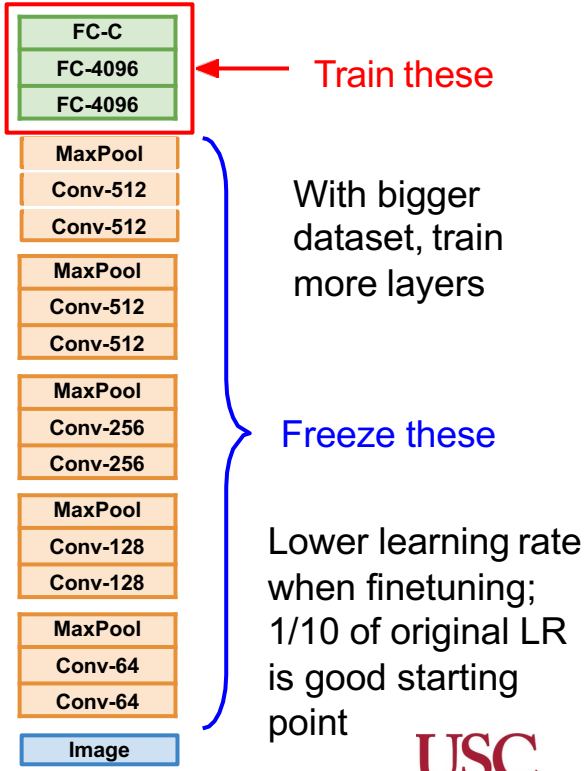
1. Train on Imagenet



2. Small Dataset (C classes)



3. Bigger dataset

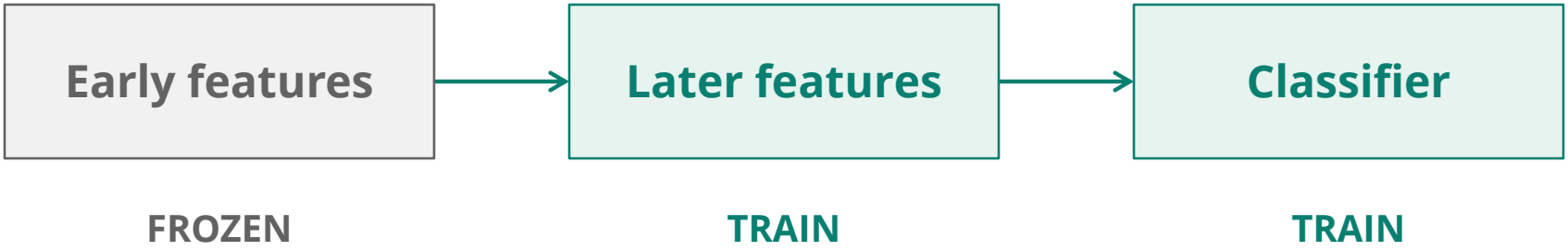


USC Viterbi School of Engineering

Then Adapt Some of the Backbone

If the fixed features are insufficient, unfreeze selected layers.

Stage 2: adapt later features and the classifier.



Fine-tuning changes selected pretrained parameters.

Course teaching example. Sources and assumptions in speaker notes.

Training and Transfer in One View

The same classifier can start from random or pretrained weights.

Starting point	Initialize a new model or load pretrained weights.
Trainable part	Choose the full model, selected layers, or a new head.
Evaluation	Select by validation performance and test the final choice.

Initialization, optimization and transfer fit one training loop.

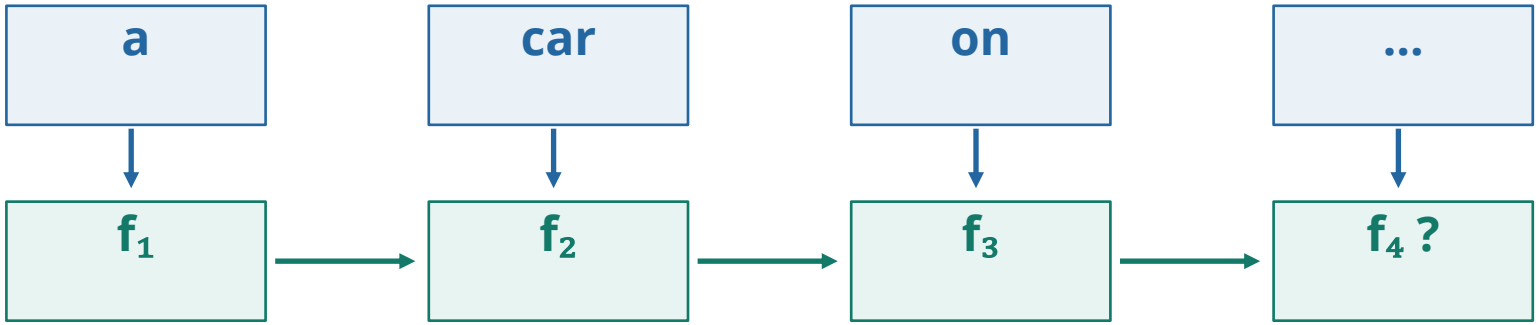
Course teaching example. Sources and assumptions in speaker notes.

Recurrent Memory

Recurrent networks carry a state across time steps using the same weights at every step.

1. The same recurrent weights are reused at every time step, while the hidden state changes.
2. Detaching the state keeps its values as memory but stops gradients from flowing further back.
3. An LSTM uses gates to decide what to retain, what to write, and what to output.
4. In an LSTM, the cell state and the exposed hidden state are separate quantities.

Why Reuse the Same Update?



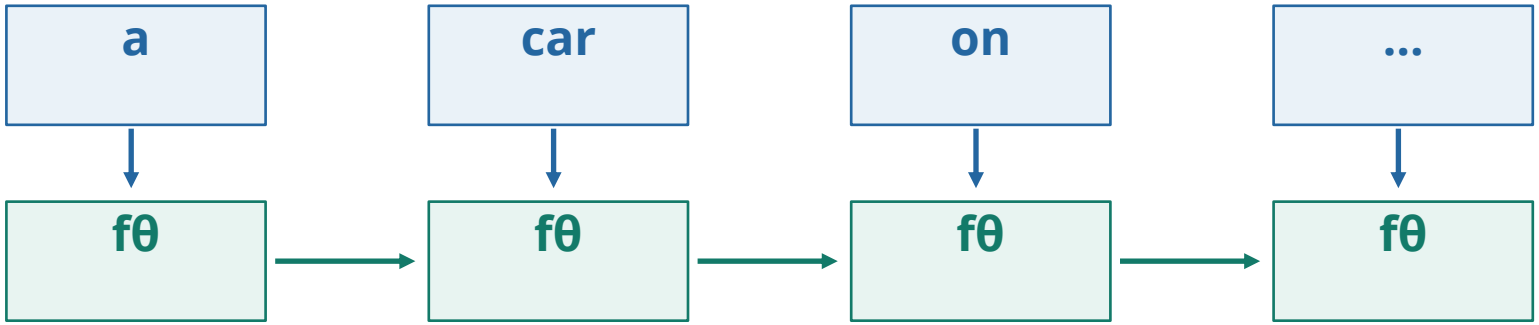
A new position would need another trained network.

Position-specific networks need more parameters as length grows.

RNN parameter sharing; Sutskever (2013). Teaching schematic.

USC Viterbi School of Engineering

One Update Rule, Any Number of Steps



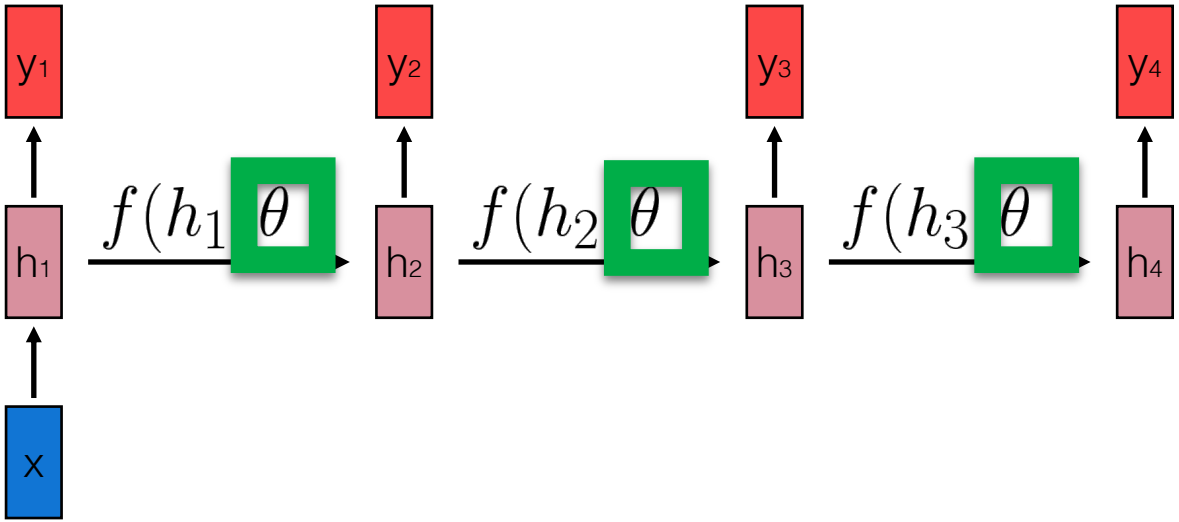
Same weights θ; changing input and previous state.

An RNN reuses a learned transition across time.

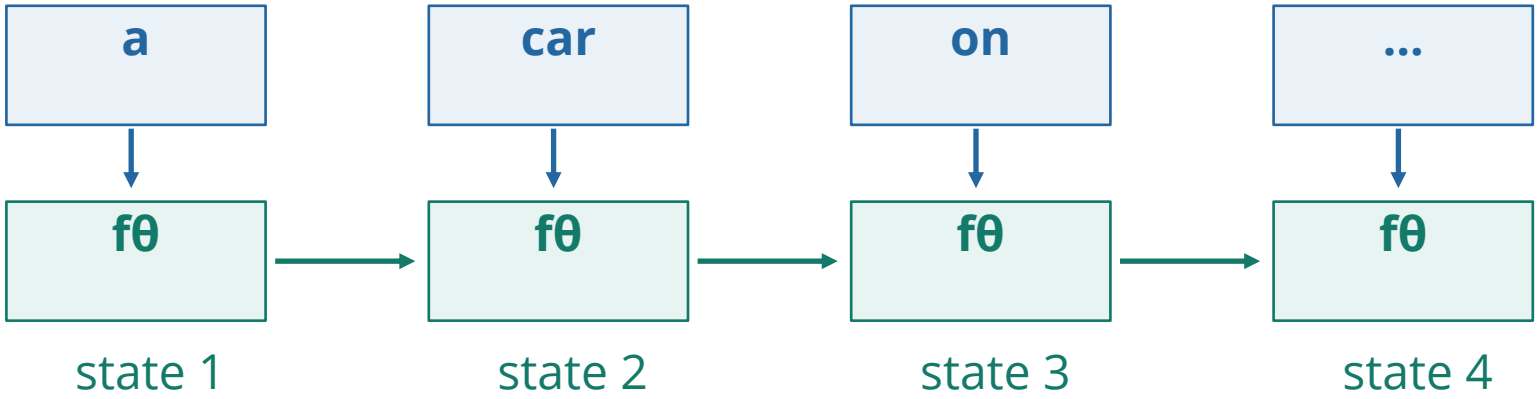
RNN parameter sharing; Sutskever (2013). Teaching schematic.

USC Viterbi School of Engineering

Recurrent Neural Network



Shared Weights, Different States



CNN: reuse a filter across space. RNN: reuse an update across time.

Sharing θ does not make the hidden states or predictions identical.

RNN parameter sharing; Sutskever (2013). Teaching schematic.

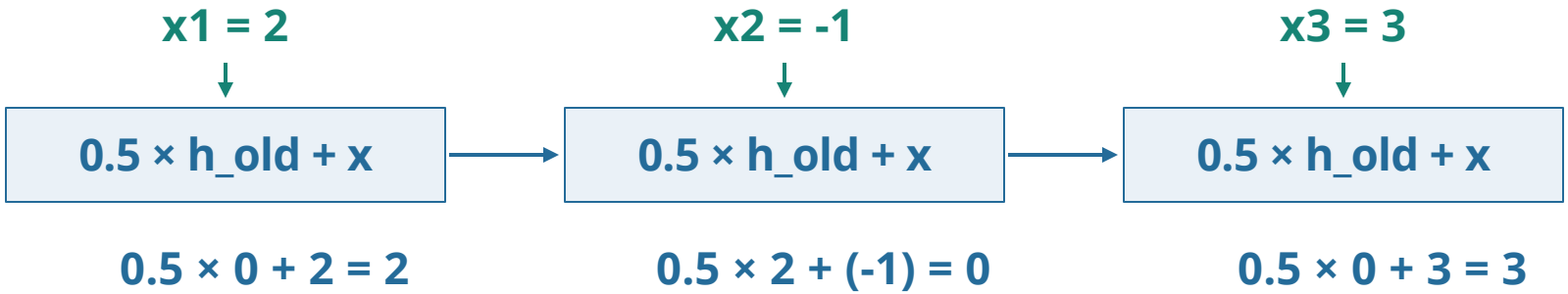
USC Viterbi School of Engineering

One Recurrent Rule, Three Steps

The coefficient 0.5 is reused at every step while the input reading and the carried state change.

$h_t = 0.5 \times h_{(t-1)} + x_t$

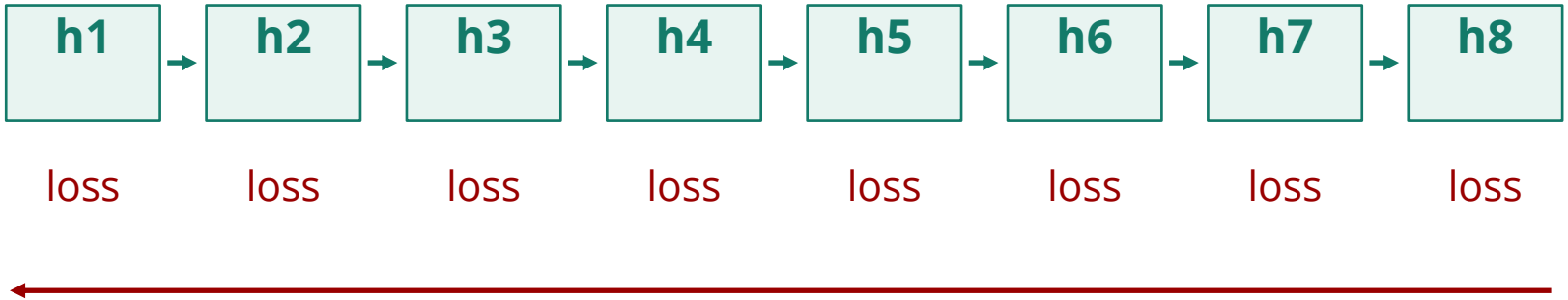
Task: update a state from sensor readings. h0 = 0



Same stored coefficient 0.5 at every step

Simplified: scalar values, no nonlinearity

Long Sequences Make Full BPTT Expensive



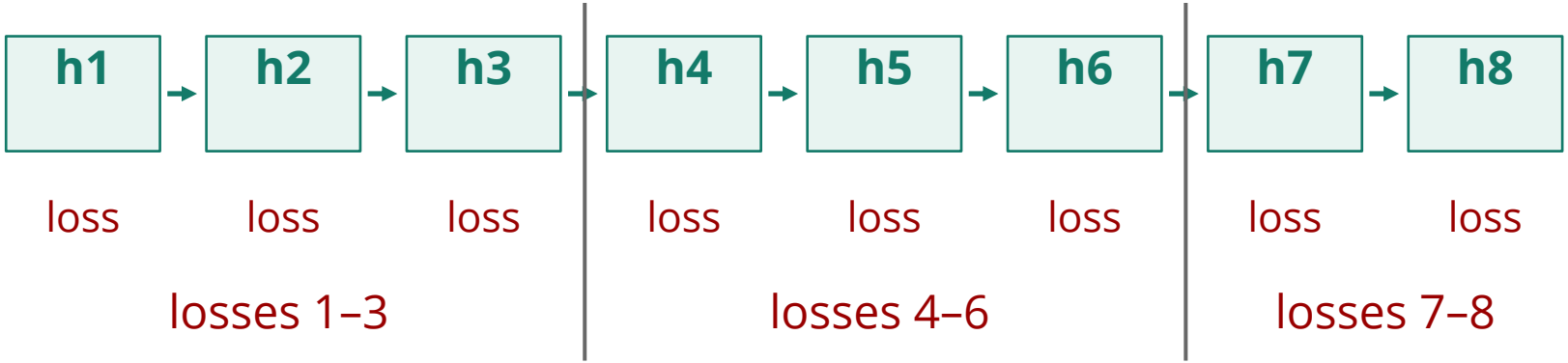
Save intermediate activations across the whole sequence.

A longer backward path needs more memory before an update.

Truncated BPTT; Sutskever (2013). Non-overlapping chunks shown.

USC Viterbi School of Engineering

Train on Shorter Spans, Keep the Token Targets



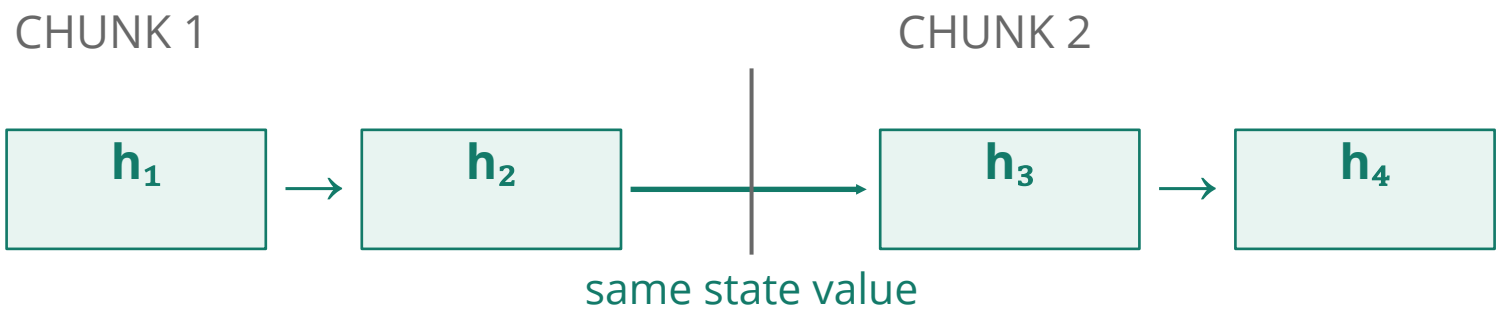
For each chunk: sum/average its losses, backpropagate, update θ.

The targets stay the same; the gradient history is truncated.

Truncated BPTT; Sutskever (2013). Non-overlapping chunks shown.

USC Viterbi School of Engineering

Carry the State Across Chunks

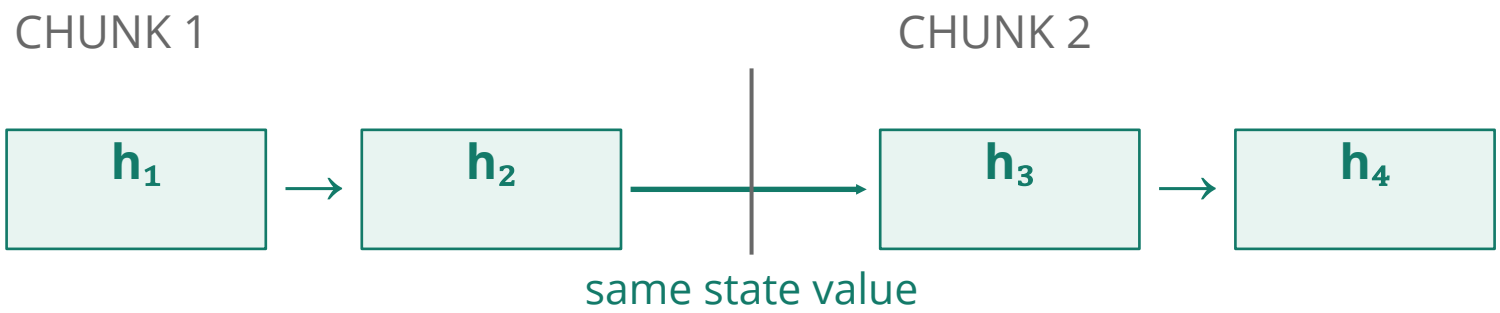


The next chunk starts with information from the previous chunk.

Forward memory can cross the boundary.

Course schematic.

Stop Gradients at the Boundary



STOP ← gradient from chunk-2 loss

detach(h_2)

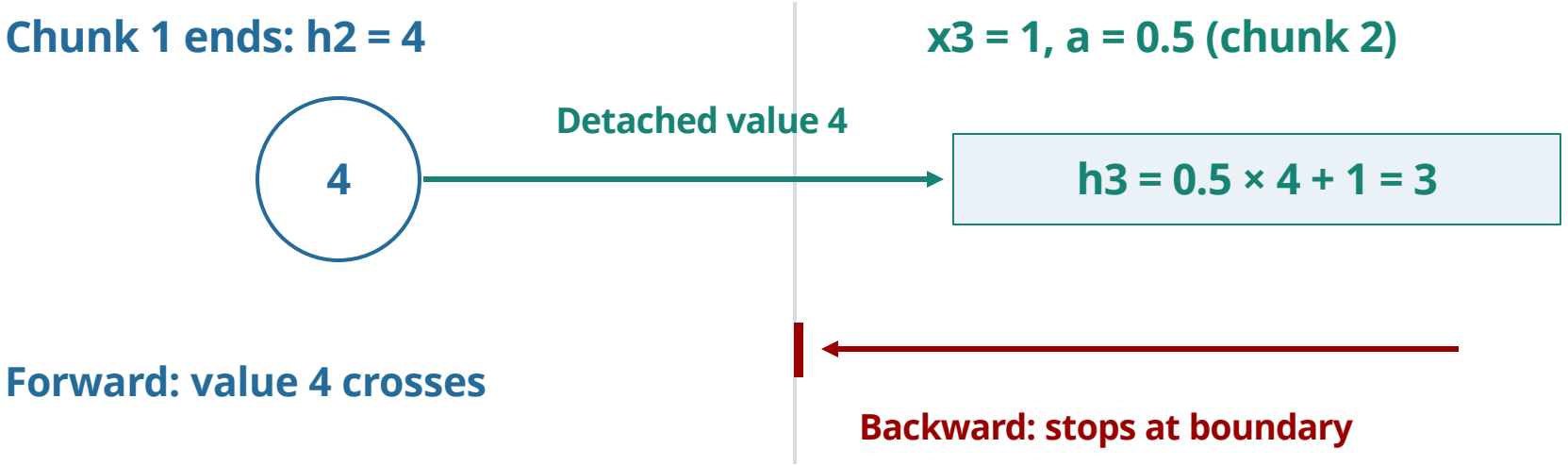
The backpropagation horizon is shorter than the forward history.

Course schematic.

USC Viterbi School of Engineering

Detach Between Sequence Chunks

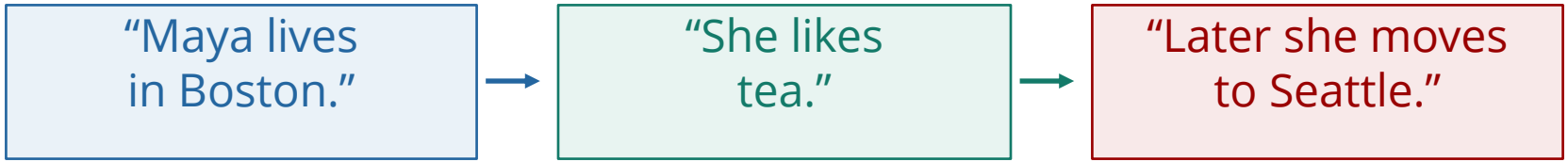
Chunk 2 reuses the value 4 from chunk 1, but its loss sends no gradient back into chunk 1.



a still gets gradients from chunk 2

Where Does Maya Live Now?

A continuing story: track changing facts across context.



Query prompt: “Where does Maya live now?” → Seattle

Teaching analogy: not literal neuron slots; no measured activations.

LSTM must learn when to retain and when to update memory.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Standard RNN: One State for Everything

A standard RNN uses a single recurrent hidden state h_t:

Remember city (earlier context)	Remember tea (local detail)	Predict next (output score)

A single vector mixes distant context and immediate detail. Intervening words can dilute or displace earlier facts.

A single hidden state mixes memory and immediate output.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Step 1: First Fact Sets the City Memory

Step 1: Process the opening sentence of the story.

Evidence 1:
“Maya lives in Boston.”

Memory sketch (analogy)

Boston [active]	Seattle [not seen]

The first sentence adds Boston to the memory sketch.

Initial evidence establishes the current context.

Standard LSTM: PyTorch documentation. Teaching analogy.

Step 2: Unrelated Context Arrives

Step 2: Unrelated preference information arrives.

Evidence 2:
“She likes tea.”

Memory sketch (analogy)

Boston [keep]	Seattle [not seen]

Tea is a preference, not a relocation. Boston remains true!
The model needs to keep older facts when new topics arrive.

New input should not erase unrelated existing memory.

Standard LSTM: PyTorch documentation. Teaching analogy.

A Learned Dimmer: How Much to Retain

A gate acts as a learned dimmer on existing memory:

City Retain Slider: HIGH



Unrelated tea arrives.
Slider near 1 retains old city.

Memory sketch (analogy)

Boston [keep]	Seattle [not seen]

Values near 1 keep most of the old cell content.

A gate acts like a continuous dimmer between 0 and 1.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Why Is It Called a Forget Gate?

Terminology contrast: retention multiplier vs forget gate

Retention scale: near 1 keeps most; near 0 keeps little



Historical Name: FORGET GATE Standard literature term	Actual Math: RETENTION FACTOR Multiplies past cell state c_{t-1}

Gates softly modulate features; values near 0 retain little info.

High forget gate: retain more. Low: retain less.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

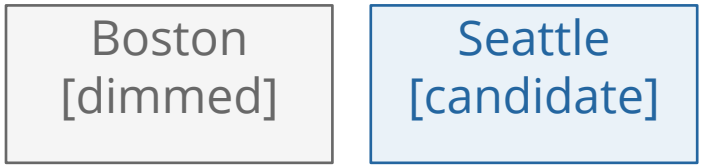
Step 3: A Relocation Changes the Fact

Step 3: A relocation overrides the previous city fact.

Evidence 3:
“Later she moves to Seattle.”

“moves to” signals a change:
Old Boston fact is outdated.

Memory sketch (analogy)



The retain slider for Boston turns low to reduce outdated info.

When facts change, the retain dimmer turns low.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Two Separate Decisions: Retain and Write

Two controls operate at the same step.

1. FORGET GATE: LOW

0 —●— 1

Action on existing memory:
Retain little of old Boston

2. INPUT GATE: HIGH

0 —●— 1

Action on candidate content:
Write new candidate Seattle

Reducing old memory and writing new candidate content are separate.

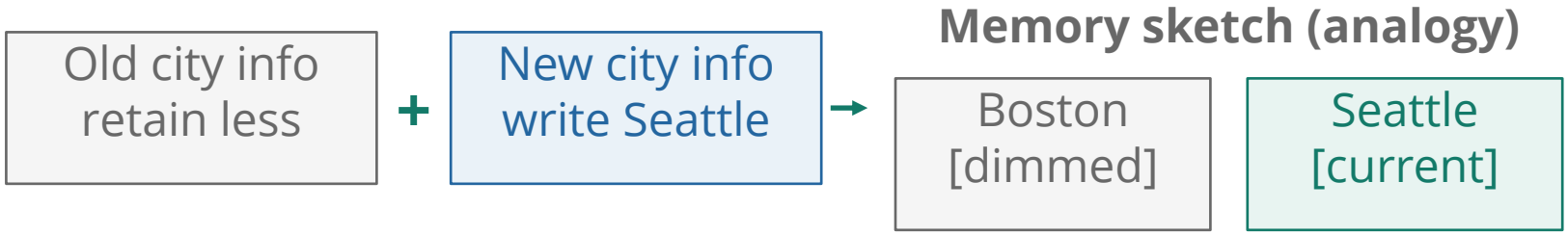
Retaining old memory and writing new memory are separate.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Updating Memory: Retained Plus Added

Cell update: retained past joins new candidate content:



The old-memory path and new-content path meet here.

Updated cell memory combines retained past and new info.

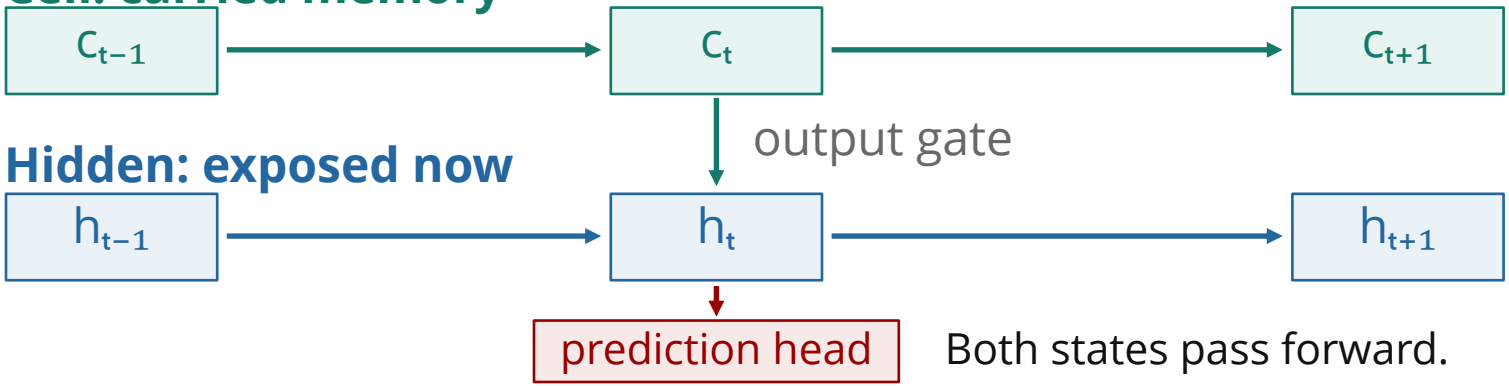
Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Two Lanes: Cell State and Hidden State

Both states move forward as the next token arrives:

Cell: carried memory



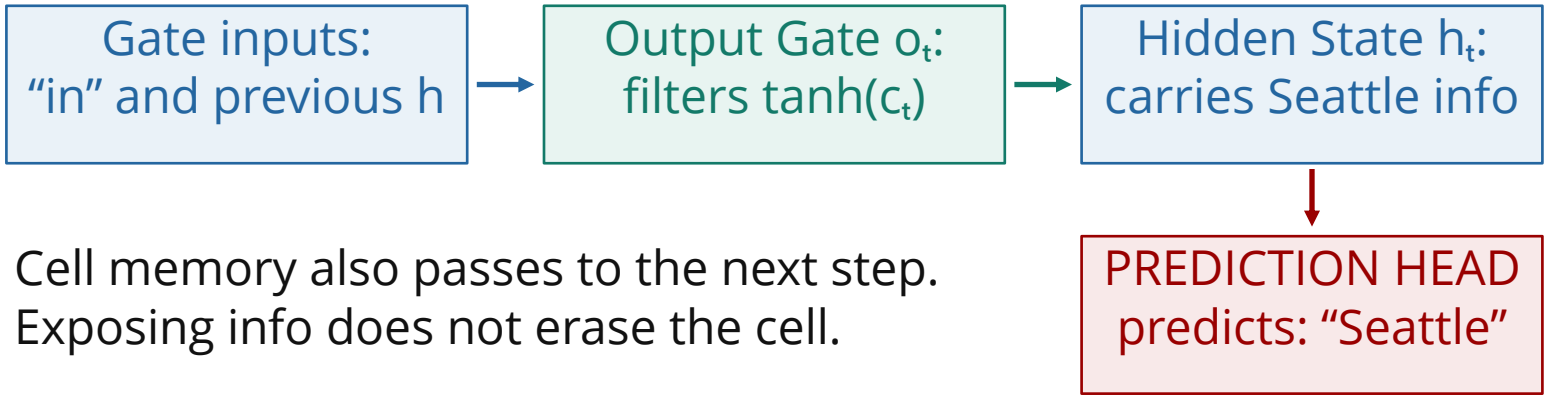
Cell state carries memory; hidden state is exposed now.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Output Gate: Expose What Is Needed Now

Model processes prompt tokens: "...Maya now lives in"



Cell memory also passes to the next step.
Exposing info does not erase the cell.

Exposing memory for an output does not delete the cell.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

LSTM Cell Update

The cell update c_new = f ⊙ c_old + i ⊙ candidate blends old memory with new content.

c_new = f ⊙ c_old + i ⊙ candidate

c_old = 4, candidate = 0.5

f = 0.9, i = 0.1 **0.9×4 + 0.1×0.5 = 3.65**

f = 0.1, i = 0.6 **0.1×4 + 0.6×0.5 = 0.7**

The forget gate f and input gate i are independent LSTM gates, so they need not sum to 1.

h_new = o ⊙ tanh(c_new)

Then h_new = o ⊙ tanh(c_new), so the output gate controls the exposed state separately.

Conceptual Analogy vs Distributed Vectors

Teaching analogy vs real distributed representations:

Teaching Analogy	Real Neural Vectors
Named city slots	Dense vectors in $\mathbb{R}^d$

City names are labels for our sketch, not actual neurons.

Gates are vectors in $[0, 1]^d$: different dimensions dim independently.

Gates operate dimension-by-dimension on real vectors.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Where Do Gate Values Come From?

Current x_t and previous h_{t-1} feed separate gate networks:



At the same layer, weights are shared across time.
Backpropagation through time trains each weight matrix.

Shared trained weights compute context-dependent gates.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

Recap: Forget, Input, and Output Roles

Three distinct gate roles work together at each step:



Retain past + write new candidate → updated cell → expose to hidden.

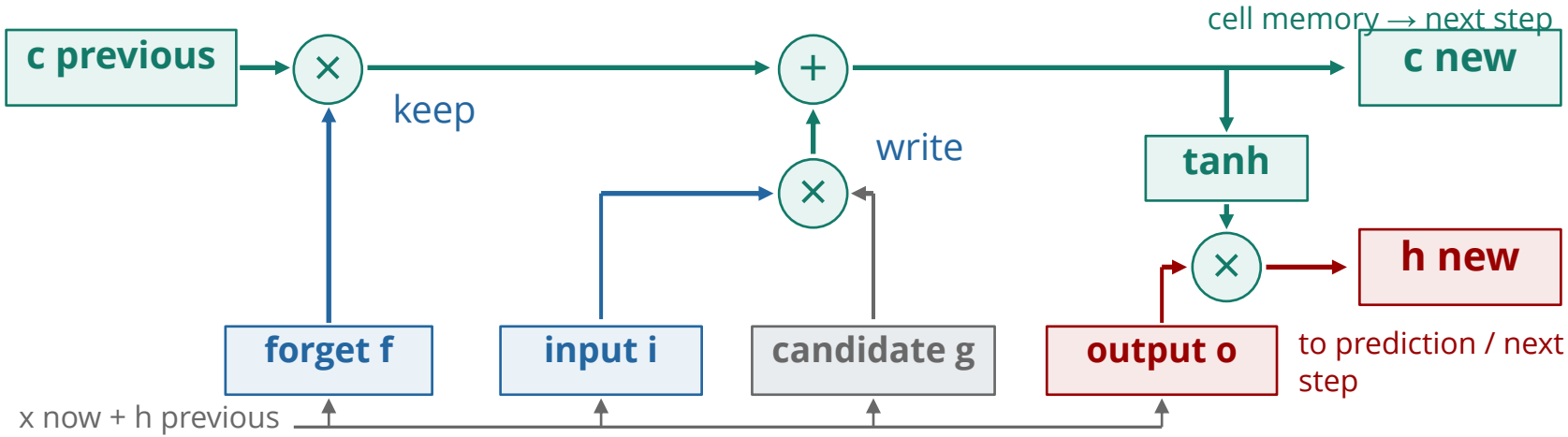
Three learned dimmers control retain, write, and expose.

Standard LSTM: PyTorch documentation. Teaching analogy.

USC Viterbi School of Engineering

The Complete LSTM Cell

One time step: keep, write, and expose information.



The three gates work together inside one repeated LSTM cell.

Standard LSTM cell; PyTorch LSTM equations. Editable course diagram.

USC Viterbi School of Engineering

The Limitation: A Compact State Bottleneck

A fixed-size state pair must compress an arbitrary history:



Can we keep separate vectors and read them when needed?
Next: read earlier token vectors with attention.

A fixed-size state must compress the entire past.

Standard LSTM: PyTorch documentation; Vaswani et al. (2017).

USC Viterbi School of Engineering

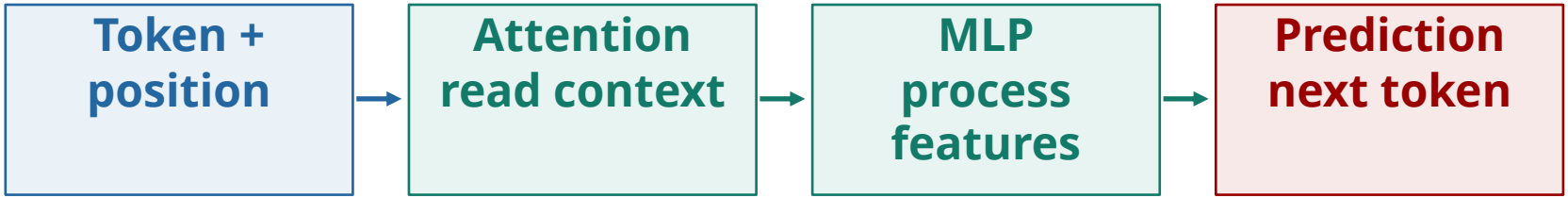
Attention and Transformers

Instead of carrying one state forward, attention lets each position read directly from a context.

1. Recurrent models pass a carried state step by step; attention computes a weighted read over many positions.
2. Position information, masks, and attention weights are distinct parts that should not be confused.
3. In training, all target positions are known; in generation, each step sees only the generated prefix.
4. Weights are shared across positions within a layer, while different layers have distinct weights.

Now Build a Text-Generating Transformer

Task: continue “The dog chased the ball. It …”



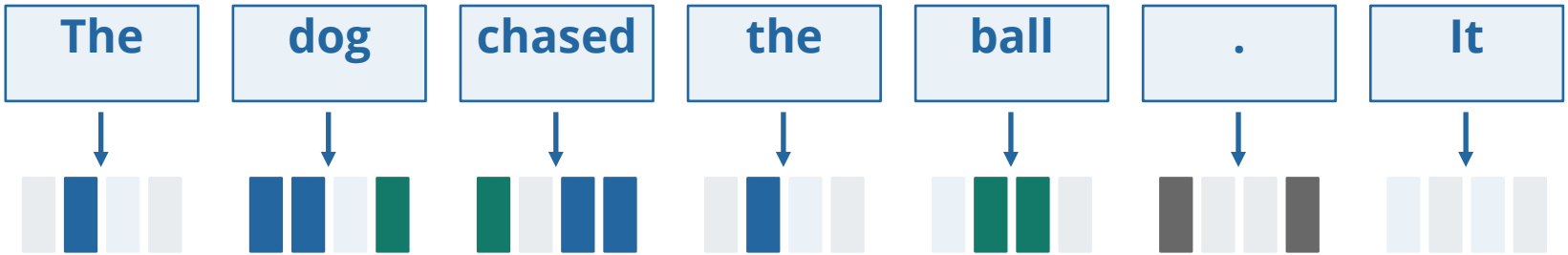
Repeat attention + MLP in several layers.

Follow one path from input words to the next-token prediction.

Vaswani et al. (2017). Course schematic.

Words Enter as Feature Vectors

Start with one learned vector per token.



Simplified token vectors; colored bars are illustrative, not measured.

Each token begins as a learned vector of feature values.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Word Order Changes Sentence Meaning

Same words, different order, different meaning.



Actor: dog | Target: man



Actor: man | Target: dog

Word vectors alone do not specify their order.

Identical words in different orders mean different things.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Add a Position Vector to Each Token

Give each token a signal for its position.

The	dog	chased	the	ball	.	It
p1	p2	p3	p4	p5	p6	p7



Here: word vector + position vector = input vector

Position values preserve word order across the sequence.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

One Token Asks About Its Context

At “It”, read the available context.



“It” could refer to the dog or the ball.

All seven positions, including “It”, can supply information.

A token needs context from other tokens to resolve its meaning.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Step 1: Compute Attention Weights

At “It”: how much should each source contribute?



Illustrative weights for one head; they add to 1.

A different token or context produces a different set of weights.

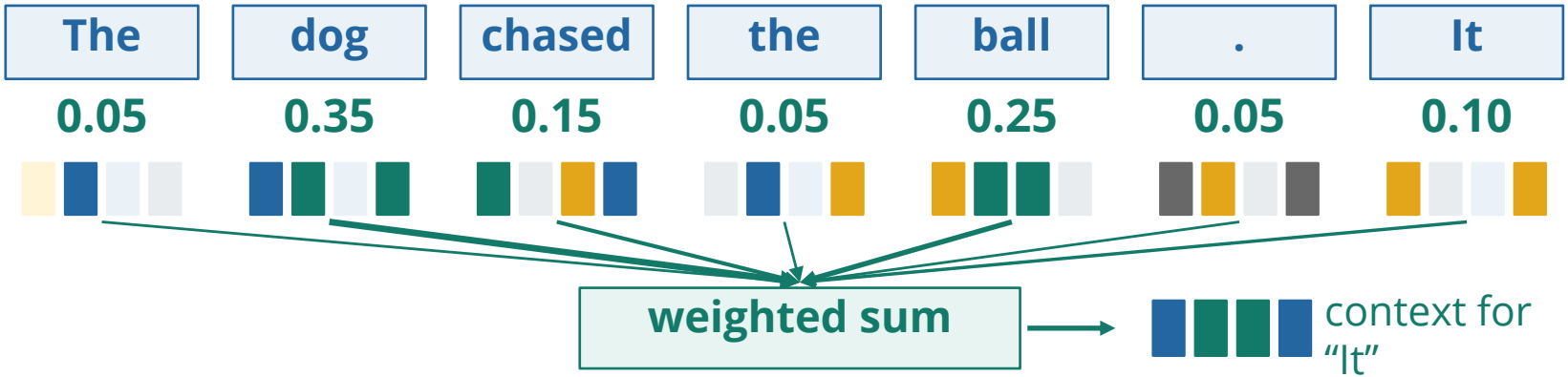
Attention weights are computed from the current input.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Step 2: Mix the Source Information

At “It”: use those weights to combine source vectors.



Illustrative values and weights; Block 9 shows the projections.

The weights control how much each value vector contributes.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Attention as a Weighted Mix

At 'barked', toy attention mixes prefix values 1, 5, 9 with weights 0.2, 0.3, 0.5 into context 6.2.



Context = 0.2 + 1.5 + 4.5 = 6.2

Task: predict the next token after 'The dog barked'.

Projection parameters are learned and fixed at inference; attention weights are recomputed for each input.

Why Future Tokens Must Be Hidden

Hide the word we are trying to predict.



CAUSAL MASK: future words hidden

Seeing “chased” would reveal the answer.

A position cannot see future tokens it is trying to predict.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Causal Attention Across Two Positions

Later positions can read a longer prefix.

At “dog”: read the first 2 positions.

The	dog	chased	the	ball	.	It

At “ball”: read the first 5 positions.

The	dog	chased	the	ball	.	It

Colored: allowed. Gray: future positions, hidden here.

Each token may attend to itself and earlier tokens only.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Order, Visibility, and Contribution Are Different

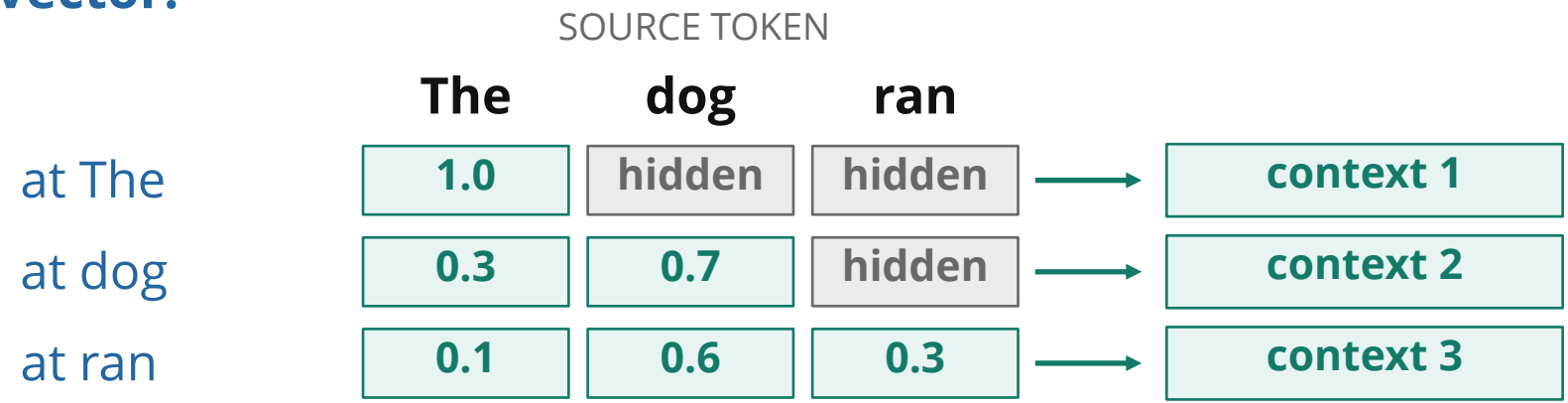
POSITION	CAUSAL MASK	ATTENTION WEIGHTS
dog at position 2	At dog: The + dog	At It: dog gets 0.35
Where is this token?	Which tokens are visible?	How much does each contribute?

Order, access, and contribution are three separate choices.

Vaswani et al. (2017). Course schematic.

Each Position Computes Its Own Mixture

Each query position gets its own weights and context vector.



Illustrative weights. Each row uses the same layer input.

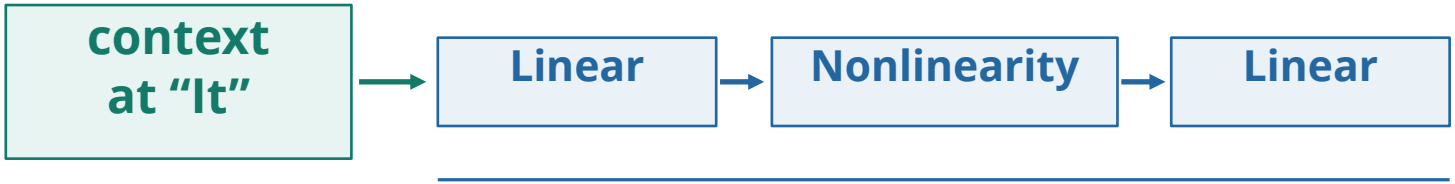
Shared parameters compute a different mixture at each position.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Why Add an MLP After Attention?

After gathering context, learn how to combine its features.



A familiar feedforward network (MLP), applied to one position.

Its output is another feature vector for “It”.

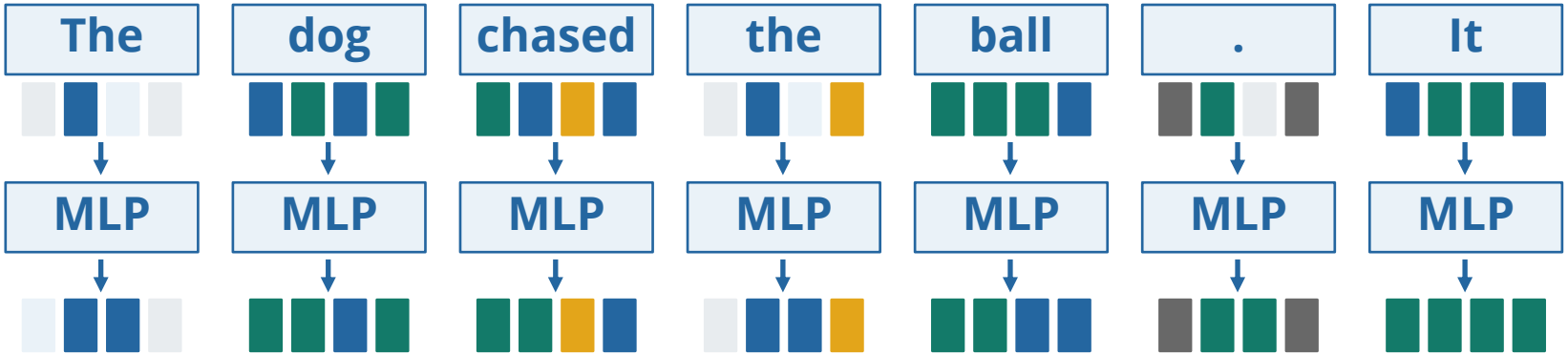
Attention gathers context; the MLP learns new feature combinations.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Same MLP, Different Token Vectors

Reuse this MLP at every position in the layer.



Same MLP weights at every position within this layer.

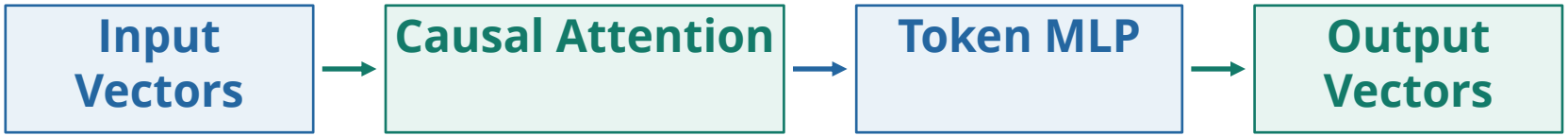
A shared MLP transforms each contextualized vector separately.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Inside One Transformer Block

Put attention and the MLP together.



Attention: exchange information across positions.
MLP: transform the features within each position.

Residual connections and LayerNorm omitted for clarity.

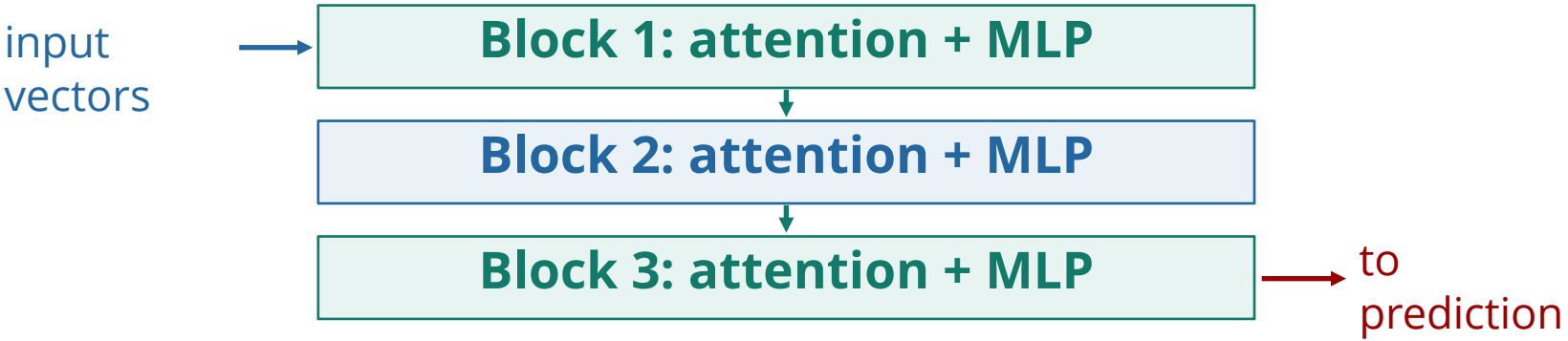
Attention shares information; the MLP refines each vector.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Stack Blocks to Build a Deeper Network

Each block has its own attention and MLP weights.



Same architecture. Different weights. One jointly trained model.

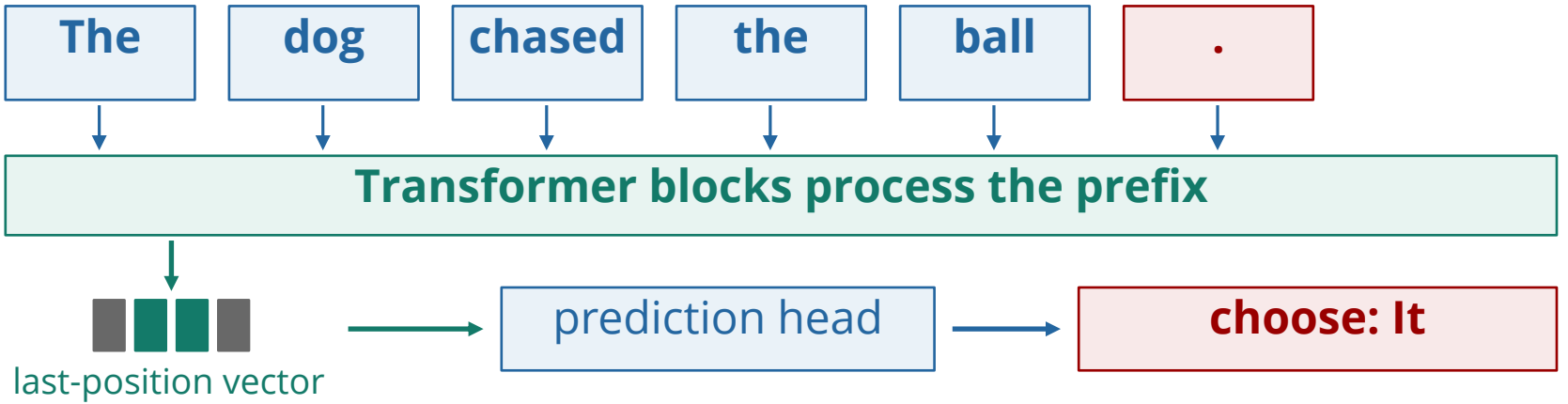
Depth counts blocks; generation length counts tokens.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Generation Step 1: Predict the Next Word

Read the prefix, then choose one more token.



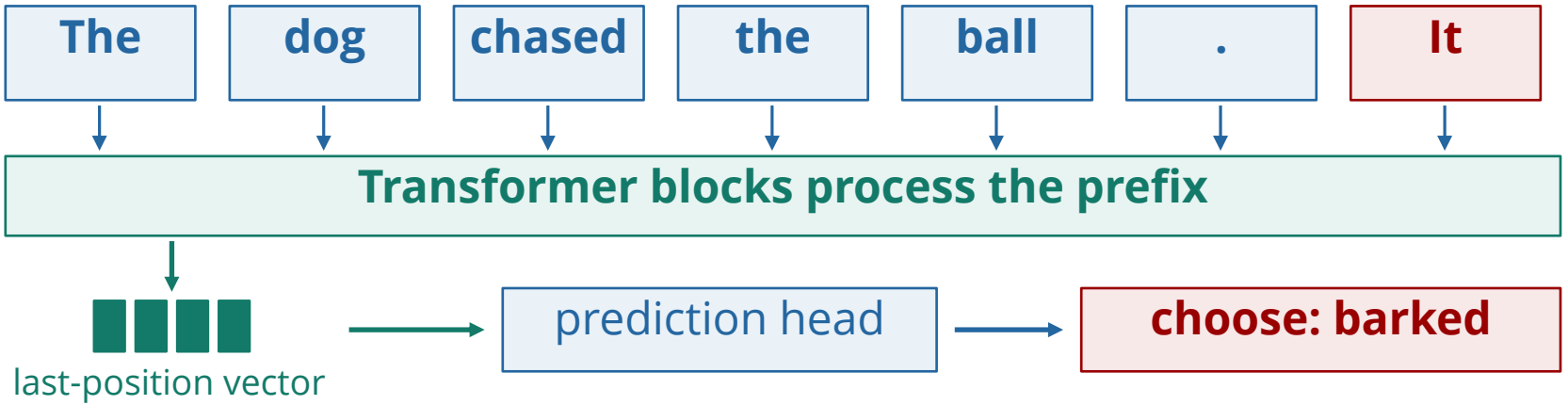
The final transformed vector produces scores for the next word.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Generation Step 2: Append and Predict Again

Read the prefix, then choose one more token.



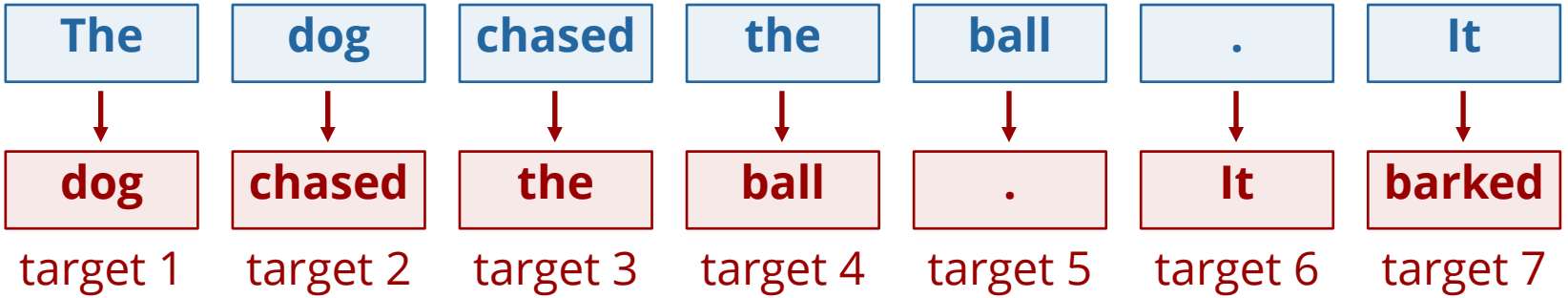
Append the new token to the sequence and repeat the process.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

USC Viterbi School of Engineering

Training Scores All Positions in Parallel

During training, the full example is already known.



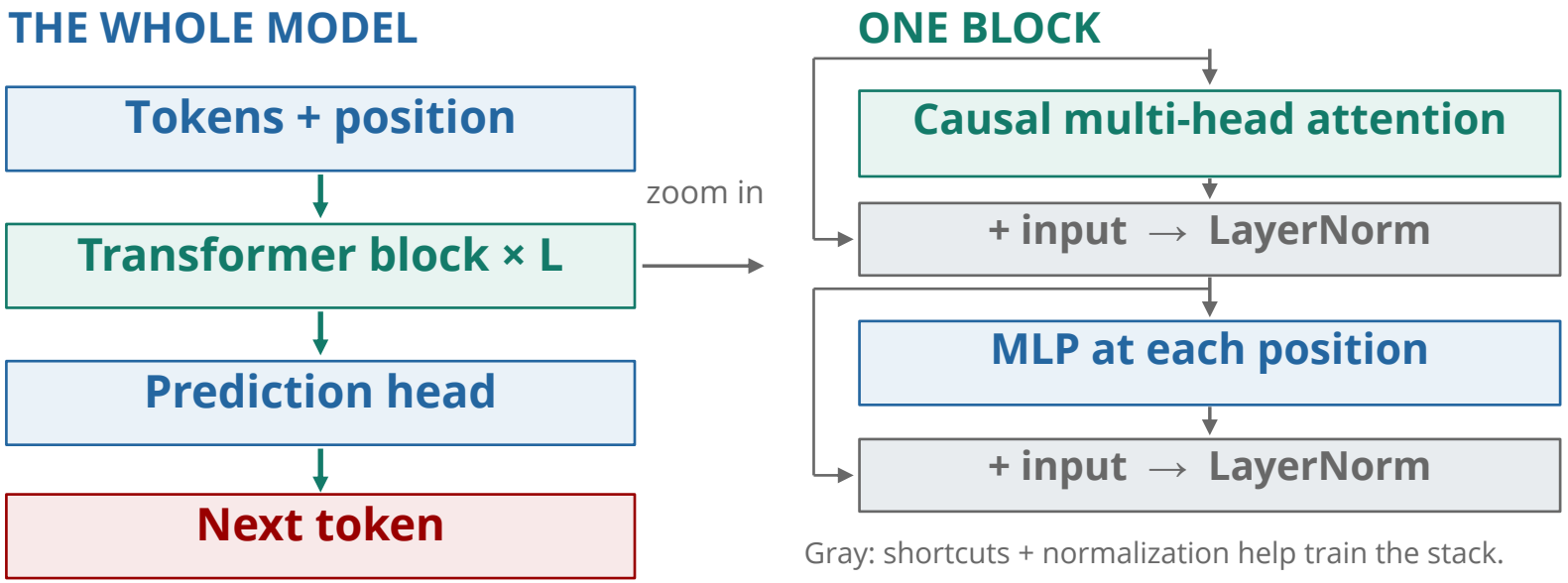
Score every next-token target, with future tokens hidden.

During generation, the next chosen token is not yet known.

Training evaluates all sequence positions in a single pass.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

The Complete Text-Generating Transformer



Gray: shortcuts + normalization help train the stack.

Generate → append the new token → run the model again.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Position and Causal Mask: Different Jobs

A text model uses both position vectors and a causal mask. Which choice explains their different roles?

A Position vectors block future tokens, and the causal mask records the order of the words.

B The causal mask alone tells each token where it sits, so position vectors add nothing new.

C Position vectors alone stop each token from using later tokens, so the mask is not needed.

D Position vectors tell where each token is, and the causal mask blocks access to future tokens.

Answer D: Location and Access Are Separate

Position vectors supply explicit location information. A causal mask controls which tokens may be read.

Answer: D

A Incorrect. The roles are swapped. Position vectors carry order information, and the causal mask limits which tokens can be used.

B Incorrect: the mask controls allowed connections. Position vectors add explicit location information.

C Incorrect. Position vectors describe location but do not restrict access. Only the causal mask keeps future tokens out of view.

D **Correct. The two parts do different jobs: one supplies word order, the other keeps each token from using later tokens.**

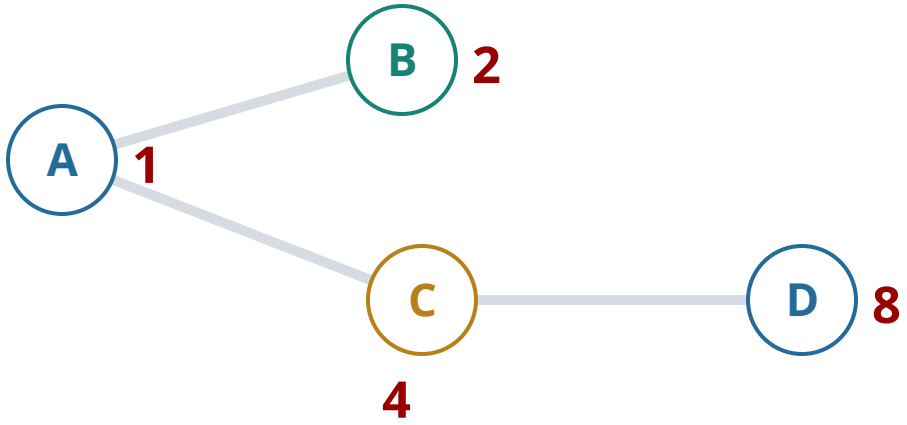
Graphs and Shared Computation

Graph networks update node features using edges, then read out predictions per node or per graph.

1. Start from concrete inputs: the edges that connect nodes and the feature stored at each node.
2. Each layer updates node features by combining information from neighbors.
3. Outputs can be one prediction per node or one pooled prediction for the whole graph.
4. Shared per-node updates with a mean or sum over neighbors respect node reordering and apply to new graphs with compatible features.

Defining a Graph Task

A toy road network shows which numbers are inputs and which outputs a model must return.



Nodes: roads A, B, C, D

Edges: A-B, A-C, C-D

These unitless load indices are input features; the labels are the delay risk classes below.

A node head predicts one delay risk class per road; a graph head pools to one city label.

A Shared Rule Can Also Read Neighbors

Back to our road example: A is the named road with load 1.

Shared node MLP	Apply the same function to each road's own features.
GNN at road A	Read A = 1 and neighbor inputs B = 2, C = 4.
Neighbor order	The mean of 2 and 4 is 3 in either listing order.

One shared update combines each node with its neighbors.

Course example. Sources and assumptions in speaker notes.

A Message Brings a Neighbor Into the Update

Road A reads information from B and C, then updates its vector.



Each node combines its own information with its neighbors.

Course schematic. GNN sources and assumptions in speaker notes.

The Next Four Pages Show Information Flow

We use a fixed toy update so that every number can be followed.

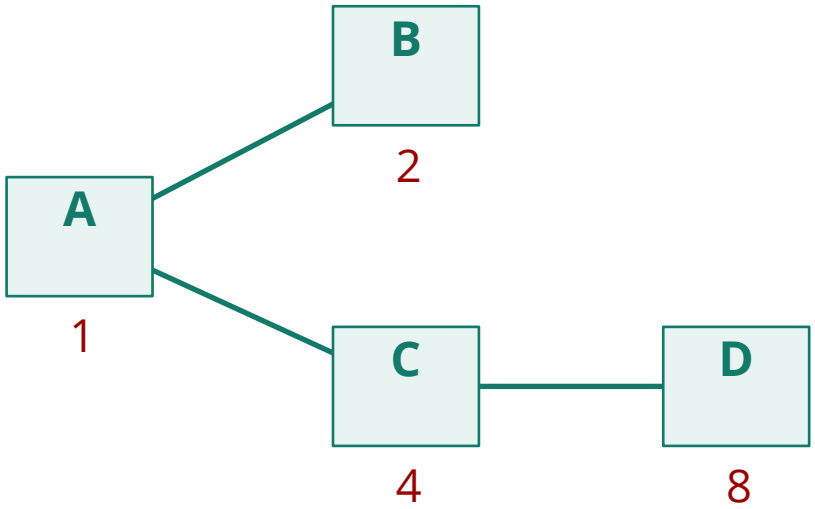
Layer 0	Observed load indices: A = 1, B = 2, C = 4, D = 8.
Toy update	Half of the own value + half of the neighbor mean.
Layer 1 and later	Computed hidden values, rather than observed load indices.

After an update, the number is a hidden value used by the model.

Course schematic. Numbers are illustrative; sources in speaker notes.

Read Only One-Hop Neighbors

All nodes: layer 0



A reads B and C.

Neighbor mean $= (2 + 4)/2 = 3$

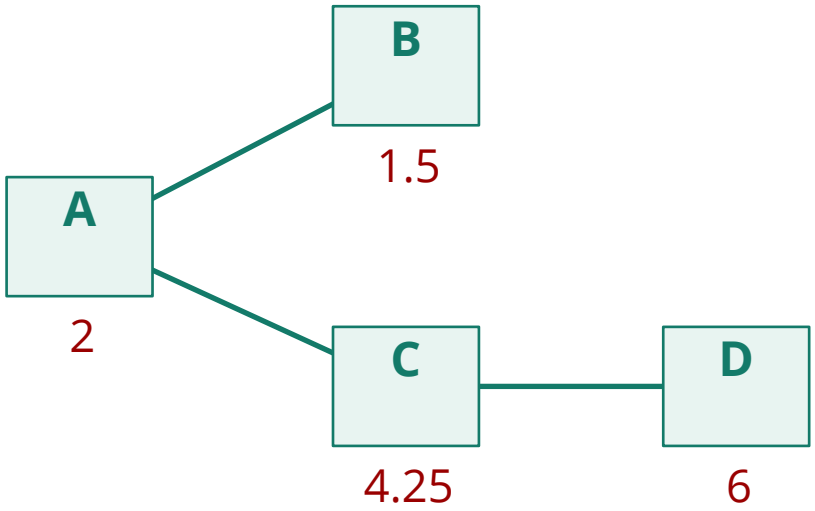
$A^1 = 0.5 \cdot 1 + 0.5 \cdot 3 = 2$

$$h_v^{k+1} = 0.5\, h_v^k + 0.5\, \text{mean}\{h_u^k : u \text{ is a neighbor of } v\}$$

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Update Every Node From Layer 0

All nodes: layer 1



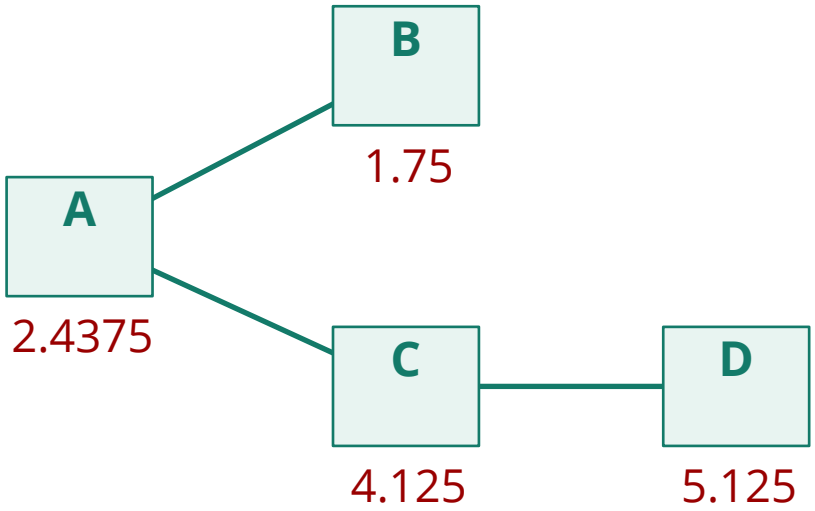
All layer-1 values:
$A^1 = 2, B^1 = 1.5$
$C^1 = 4.25, D^1 = 6$

Every new value uses only the previous layer.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

The Second Layer Reaches Two Hops

All nodes: layer 2



A reads B^1 and C^1.

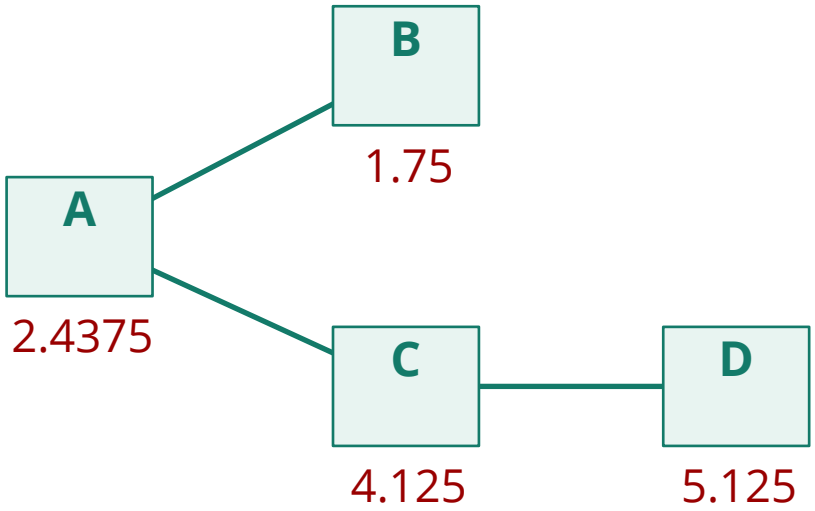
$$A^2 = 0.5 \cdot 2 + 0.5 \cdot (1.5 + 4.25)/2 = 2.4375$$

Two layers let information travel two graph edges.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Keep Updates Synchronous

All nodes: layer 2



Read layer k → write layer k+1

In-place A → B would give
$B^1 = 2$ instead of 1.5.

Node numbering does not define an update schedule.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Which Parts of the Update Can Learn?

Our 0.5 / 0.5 example explains information flow with fixed weights.

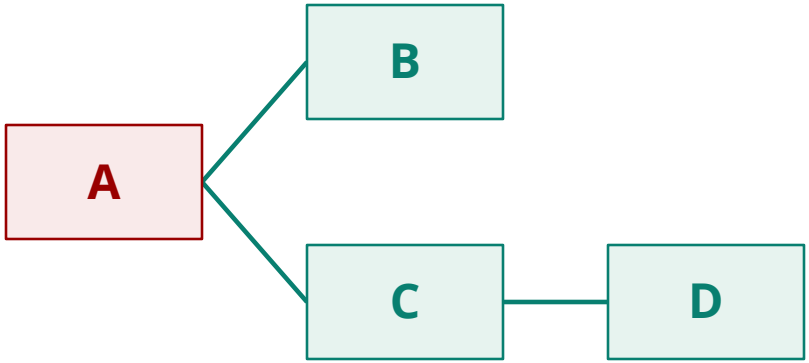
Neighbor information	Transform or weight the messages.
Node information	Combine messages with the node's current vector.
Shared parameters	Reuse the same update rule across nodes.

Training learns shared transforms from the prediction loss.

Course schematic. GNN sources and assumptions in speaker notes.

One Layer and Two Layers Use Different Context

Information moves one graph edge per local aggregation layer.



Every node reads the previous layer.

Layer 1 at A: B and C

Layer 2 at A: D can contribute through C.

Depth expands the neighborhood that can influence a node.

Course schematic. GNN sources and assumptions in speaker notes.

Neighbor Order Should Preserve A's Update

Road A has the same two neighbors whether we list them as B, C or C, B.

Neighbor Listing	Values Read	Mean
B, C	2, 4	3
C, B	4, 2	3

Toy update for A: $0.5 \times 1 + 0.5 \times 3 = 2$ in both cases.

A mean or sum treats the neighbors as a set.

Course schematic. Numbers are illustrative; sources in speaker notes.

Node Outputs Are Equivariant

A GNN gives one hidden vector per road. The output rows follow the input rows.

File Order	Layer-1 Outputs in That Order
A, B, C, D	2, 1.5, 4.25, 6
D, B, A, C	6, 1.5, 2, 4.25

Road A still has output 2 after its row moves.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Whole-Graph Output Can Be Invariant

For a graph task, combine all node outputs into one graph vector.

Sum the layer-1 values as a simple graph representation:

Row Order	Pooled Sum
A, B, C, D	2 + 1.5 + 4.25 + 6 = 13.75
D, B, A, C	6 + 1.5 + 2 + 4.25 = 13.75

The same set of node vectors gives the same pooled result.

Course schematic. Numbers are illustrative; sources in speaker notes.

Invariance and Equivariance Answer Different Questions

Always say what is being reordered and what the output represents.

Operation	Reordering	Required Behavior
Neighbor mean at A	B, C → C, B	Same aggregate
All node outputs	ABCD → DBAC	Rows move together
Whole-graph pooling	ABCD → DBAC	Same graph vector

Reordering the file keeps the same physical prediction.

Course schematic. Numbers are illustrative; sources in speaker notes.

The Node Vector Feeds a Prediction Head

For node A, the head converts h_A into class scores.



A training label supplies the loss for a labeled node.

The head and GNN layers train together when both are trainable.

Course schematic. GNN sources and assumptions in speaker notes.

What Is Shared Across Roads?

At one GNN layer, every road uses the same learned matrices.

Road-specific inputs	A reads its own vector and its neighbor mean. B reads its own.
Shared learned weights	W_self transforms a node vector. W_neighbor transforms a mean.
Repeated computation	Each road runs the same update with different input vectors.

The weights depend on feature width, rather than road IDs.

Course schematic. Numbers are illustrative; sources in speaker notes.

Why Can the Weights Run on a New Graph?

Suppose each node vector has 2 numbers. The learned matrices accept that width.

Input to One Update	Width	Shared Transform
Own road vector	2 numbers	W_self: 2 × 2
Mean of neighbor vectors	2 numbers	W_neighbor: 2 × 2
Updated road vector	2 numbers	Same rule at every road

More nodes mean more uses of the same function.

Course schematic. Numbers are illustrative; sources in speaker notes.

Inductive Means Applying the Learned Rule to Unseen Nodes

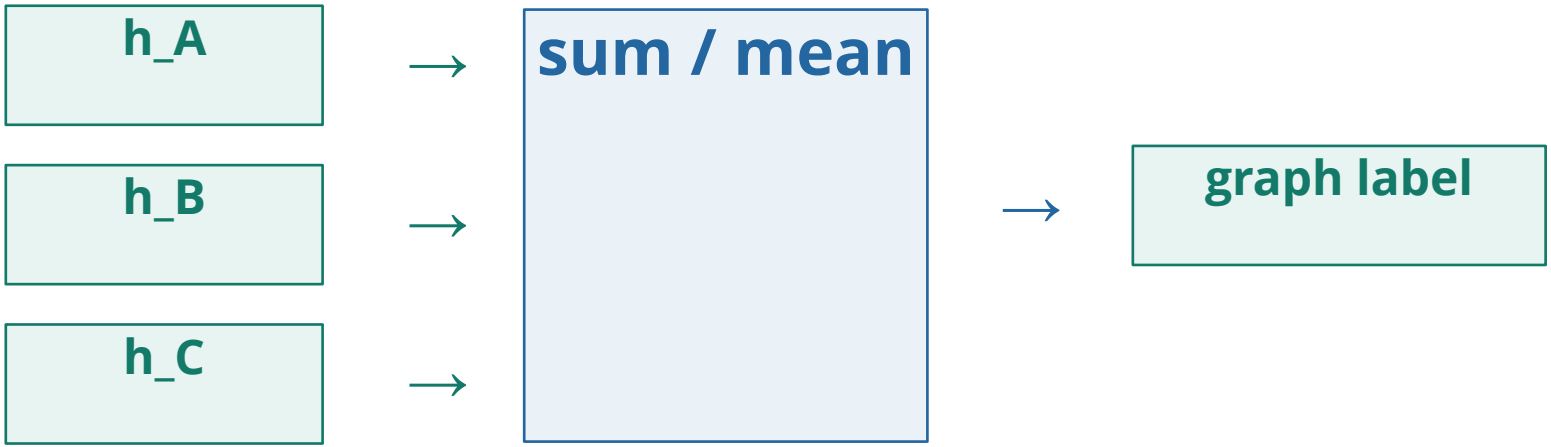
Training fits the encoder. Inference computes outputs for new compatible inputs.

Training city	Use road inputs, links and observed delay labels to fit the weights.
New city	Keep the weights. Compute vectors from its road inputs and links.
Check generalization	Test accuracy, feature units, edge meanings and traffic conditions.

Computing an output and predicting accurately are separate checks.

Course schematic. Numbers are illustrative; sources in speaker notes.

Graph Outputs Pool Across Nodes

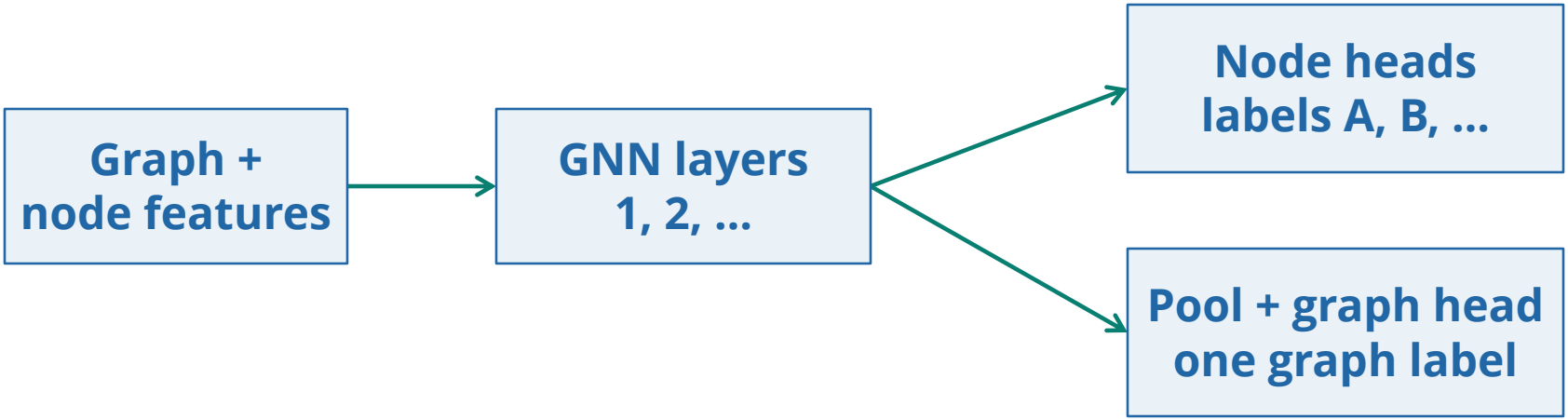


A symmetric readout gives the same graph output after reordering.

Course schematic.

The GNN From Input to Output

Node tasks keep one output per node. Graph tasks pool the node vectors.

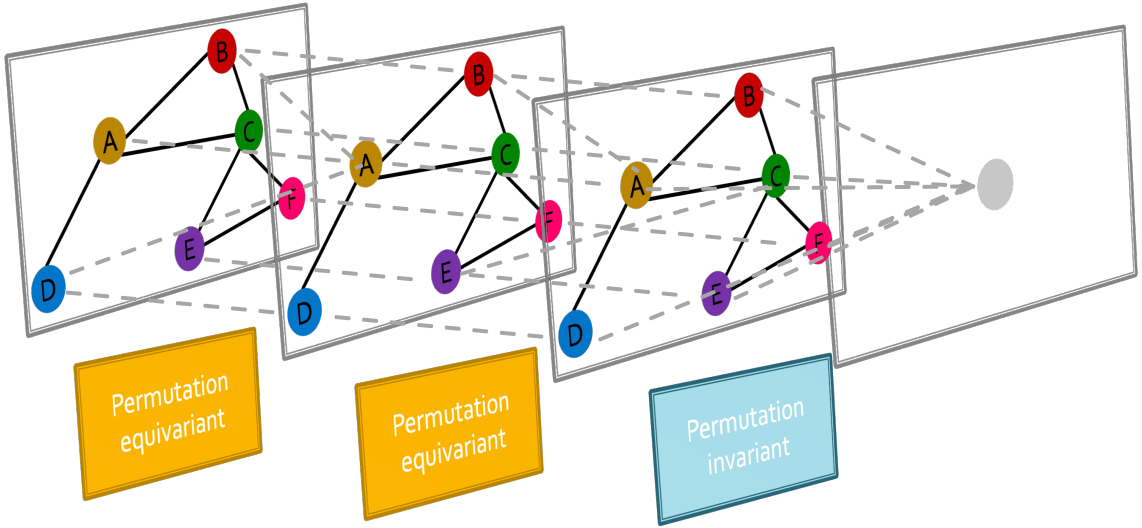


The output task determines whether we keep or pool node vectors.

Course schematic. GNN sources and assumptions in speaker notes.

GNN Layers and the Graph Readout

Two node-update layers feed a graph readout that combines the node vectors.



Equivariant node layers. Invariant graph pooling.

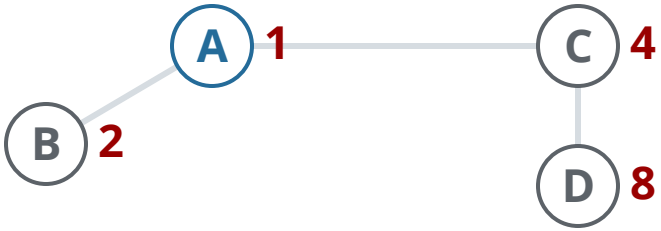
Course example. Sources and assumptions in speaker notes.

One Graph Rule, New Graph

The same two learned weights update nodes in a 4-road graph and an unseen 3-road graph.

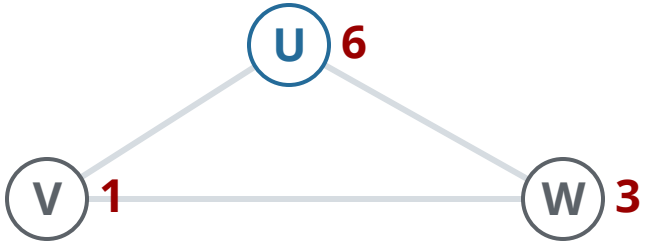
h_v = 0.5 × x_v + 0.5 × mean(neighbor x)

Old graph



A: mean(2, 4) = 3
hA = 0.5 × 1 + 0.5 × 3 = 2

Unseen graph



U: mean(1, 3) = 2
hU = 0.5 × 6 + 0.5 × 2 = 4

Feature: unitless load index, one per road

Runs without retraining, but accuracy must be evaluated; outputs are not probabilities

Review: Feature Order in a Trained GNN

Trained GNN atom features: [atomic number, charge]. A new molecule gives [charge, atomic number]. No learned node IDs. What should be done before using the frozen model?

A No change, because the feature vector width is still 2

B Pad the node count to match the training molecules

C Restore the training feature order and definitions before applying the shared weights

D Reorder atom feature rows while leaving the adjacency matrix unchanged.

Answer: C, Restore the Training Feature Order

Training atom features: [atomic number, charge]; new input: [charge, atomic number]. The edge encoding is unchanged.

Answer: C

A	The width matches, but the weights were learned for what each column means, not just the count.
B	Node count is not the issue; shared weights work on graphs of different sizes.
C	**Each column again carries the meaning that the shared weights were trained on.**
D	This mismatches atom rows with edges and leaves the feature columns swapped.

Compatible features permit computation; accuracy still needs evaluation.

Reordering Nodes Moves Features and Adjacency Together

M[i, j] = M[j, i] = 1 when roads i and j share an edge.

Order: A, B, C, D

	A	B	C	D
A	0	**1**	**1**	0
B	**1**	0	0	0
C	**1**	0	0	**1**
D	0	0	**1**	0

X rows: 1, 2, 4, 8



Order: D, B, A, C

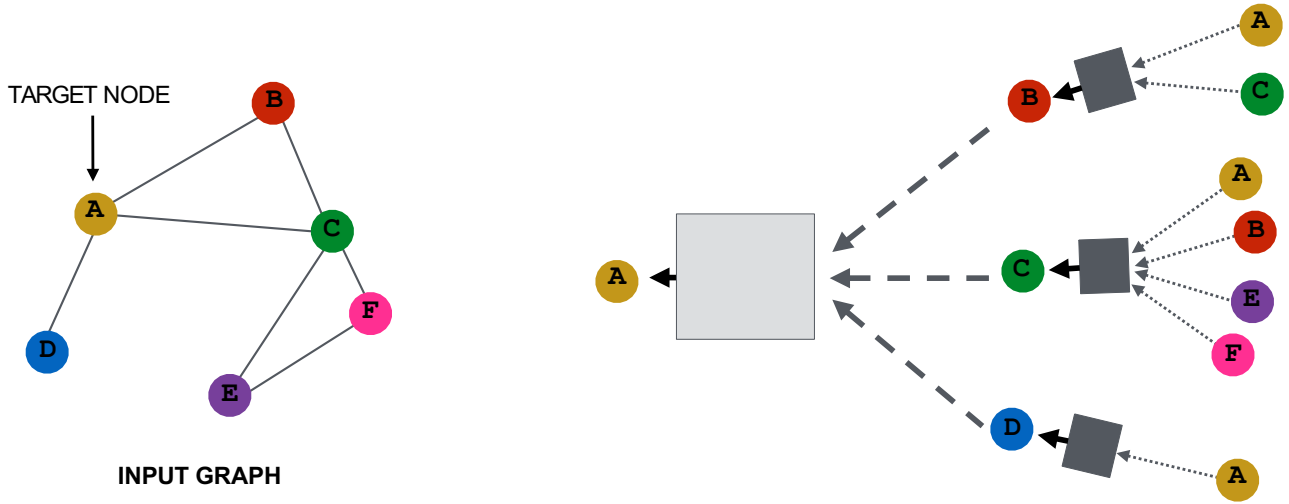
	D	B	A	C
D	0	0	0	**1**
B	0	0	**1**	0
A	0	**1**	0	**1**
C	**1**	0	**1**	0

X rows: 8, 2, 1, 4

Reorder the feature rows and both matrix axes together to keep the same graph.

Idea: Aggregate Neighbors

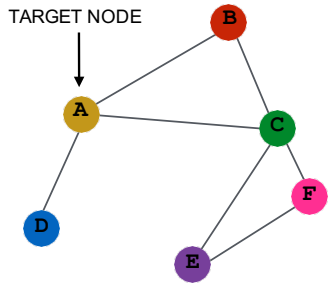
Key idea: Generate node embeddings based on **local network neighborhoods**



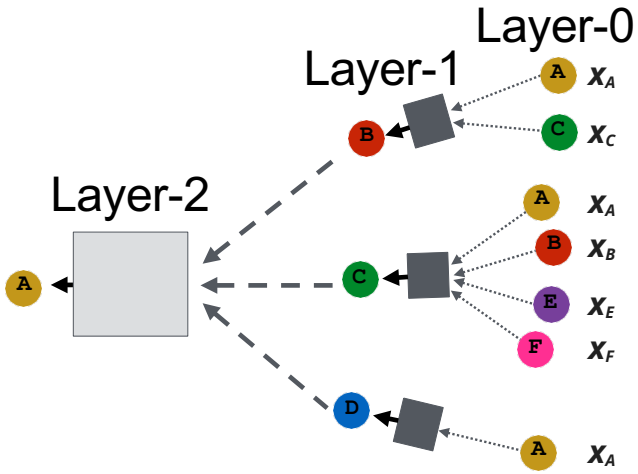
Deep Model: Many Layers

Model can be of arbitrary depth:

- Nodes have embeddings at each layer
- Layer-0 embedding of node v is its input feature, x_v
- Layer-k embeddings can use information from up to k hops away.



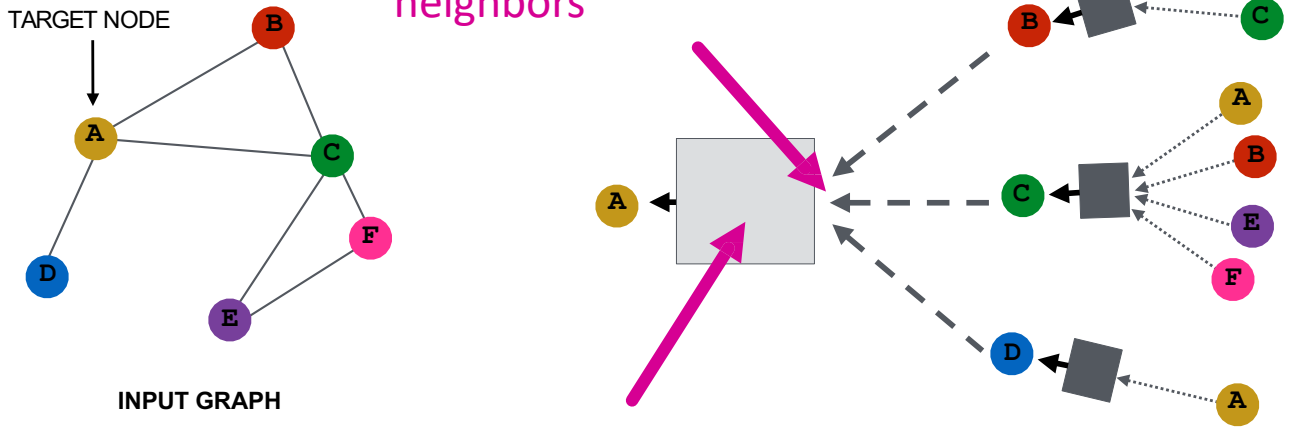
INPUT GRAPH



Neighborhood Aggregation

Basic approach: Average information from neighbors and apply a neural network

(1) average messages from neighbors



(2) apply neural network

How Far Information Travels

With one-hop layers, information moves one edge per layer along V-W-X-Y-Z.



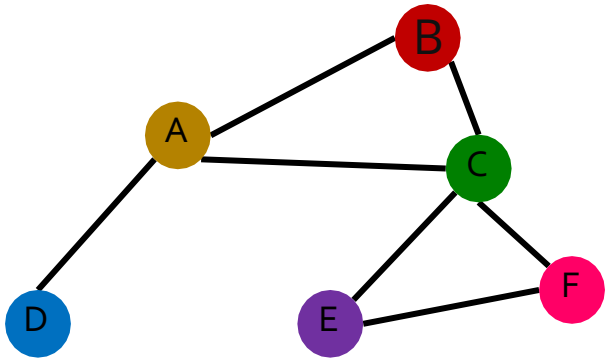
Information from Z reaches Y after 1 layer, X after 2, W after 3, and V after 4.

This assumes each layer reads only neighbors from the previous layer, with no global shortcut.

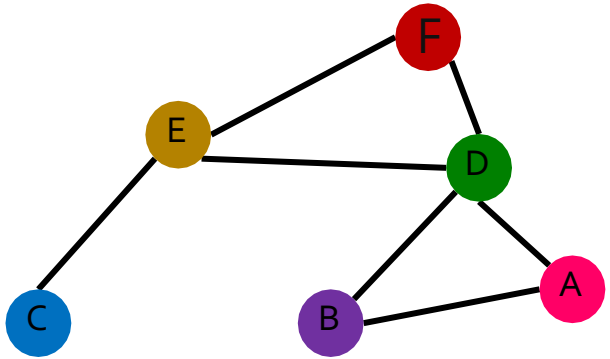
One Graph, Two Node Numberings

In this six-node sketch, colors track the objects. Letters are temporary storage labels.

Order 1



Order 2



Example: the gold object changes label A → E, while its links stay intact.

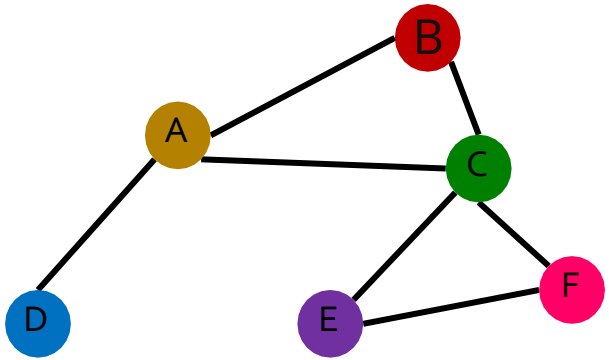
The same colored node keeps the same neighbors.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Invariance: One Answer for the Whole Graph

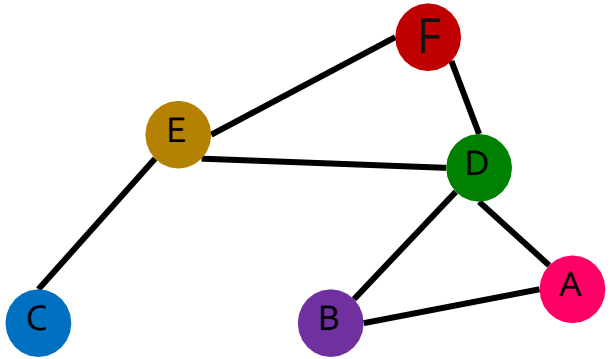
Suppose our task asks whether this entire graph is connected.

Order 1



Connected: Yes

Order 2



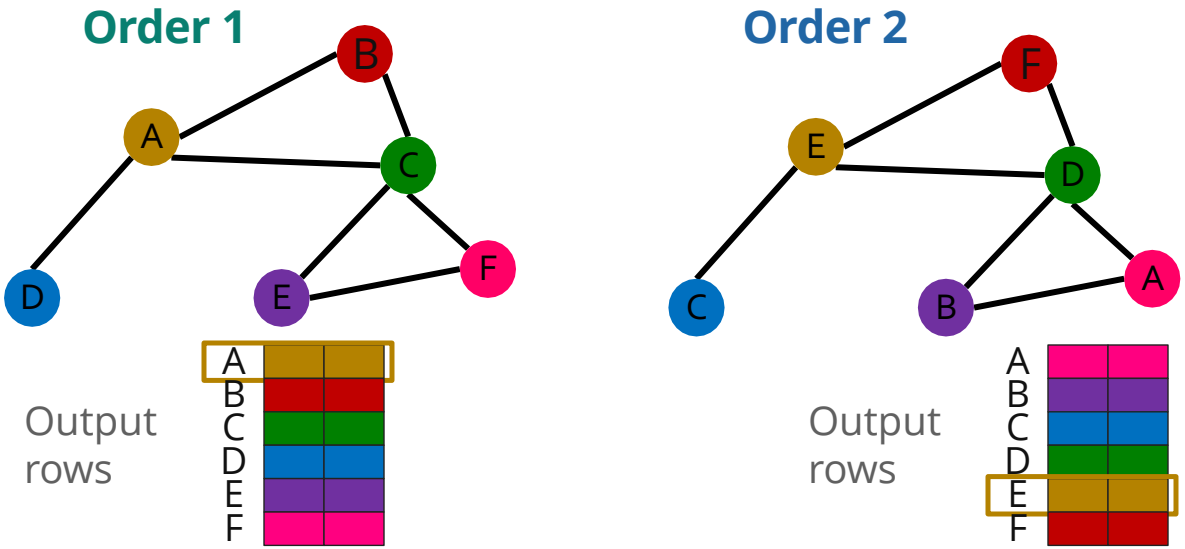
Connected: Yes

Renumbering leaves the whole-graph answer unchanged.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Equivariance: One Vector for Each Node

The colored rows below belong to the colored nodes above.

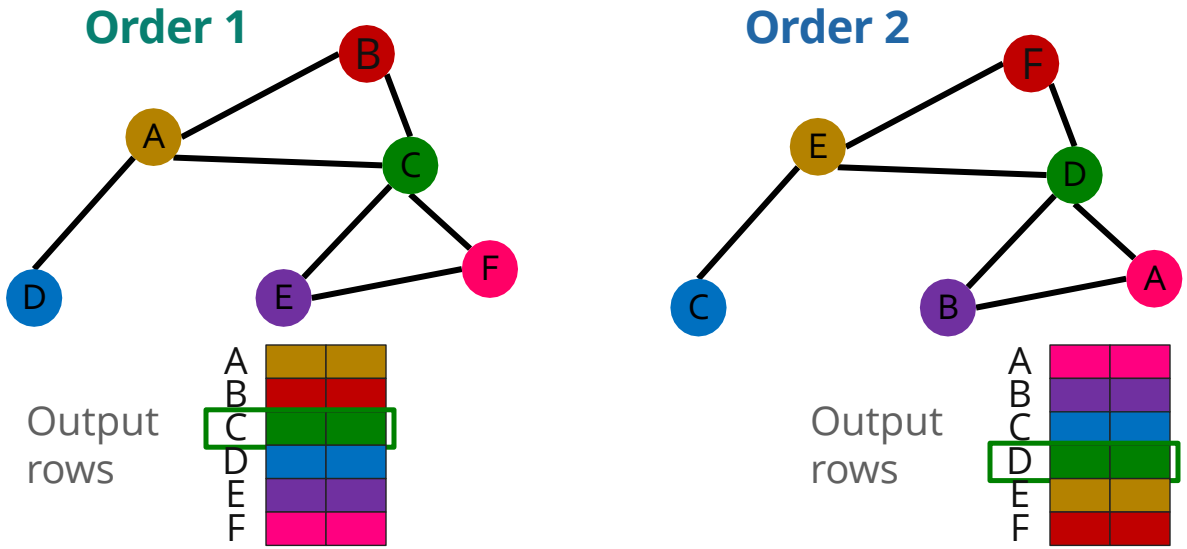


The gold object keeps its vector, moving from row A to row E.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

The Green Object Moves From Row C to Row D

Follow a second node through the same renumbering. Its vector moves with its ID.



Equivariance keeps every node's output attached to that node.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Two Output Types Need Two Properties

What should happen when we reorder the input rows?

One output per road	Output rows follow the same reordering: equivariance.
One output per graph	The graph output stays the same: invariance.
Same graph throughout	We changed its storage order, keeping its connections intact.

The required property depends on what the model outputs.

Course schematic. Numbers are illustrative; sources in speaker notes.

Reordering: Pixels Versus Nodes

After convolution, keep the largest value over all spatial positions in each channel. Rearranging input pixels can change a 3×3 neighborhood.

0	0	0	0	0
0	**1**	**1**	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

One window covers both ones: sum = 2.

0	0	0	0	0
0	**1**	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	**1**

Ones 3 rows and 3 columns apart: every window sum ≤ 1.

3×3 all-ones kernel

Stride 1, zero padding 1

Global max: 2 → 1

1×1 + global max: same pooled vector

The earlier 1 x 1 shared map only moves its outputs when positions move.

Reordering graph features with adjacency keeps the same graph; moving pixels changes neighborhoods.

Shared Computation Across Architectures

CNNs, RNNs, and GNNs reuse one set of weights many times, so uses do not add parameters.

CNN	same kernel at every spatial position
RNN	same update at every time step
GNN	same update at every node

A larger compatible input repeats the rule with no new parameters.

Parameters are fixed at inference; attention weights depend on input.

How to Study These Ideas

Retrace each worked example on your own and check every convention before computing.

1. Trace each example by hand, then check shapes and parameter counts at every layer.
2. Read loss, optimizer, and normalization conventions from their definitions before computing.
3. Note when training and evaluation modes differ and what makes a model compatible with new inputs.